

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
MÔN HỌC: CÁC HỆ THỐNG PHÂN TÁN

**Đề tài: Xây dựng hệ thống đặt đồ ăn trực tuyến dựa trên
kiến trúc Microservice**

Giảng viên hướng dẫn: Kim Ngọc Bách

Lớp: 02

Nhóm bài tập lớn: 14

Họ và tên	Mã sinh viên
Trần Đức Dũng	B22DCCN139
Lê Đức Thiện	B22DCCN823
Trịnh Lê Xuân Bách	B22DCCN056

Hà Nội – 2026

MỤC LỤC

MỤC LỤC	1
DANH MỤC BẢNG BIỂU	5
DANH MỤC HÌNH ẢNH	6
BẢNG PHÂN CÔNG CÔNG VIỆC	7
CHƯƠNG 1. GIỚI THIỆU CHUNG	8
1.1. Lý do chọn đề tài	8
1.2. Mục tiêu của đề tài.....	9
1.3. Đối tượng và phạm vi nghiên cứu	10
1.3.1. Đối tượng nghiên cứu	10
1.3.2. Phạm vi nghiên cứu	11
1.4. Phương pháp nghiên cứu và cấu trúc báo cáo.....	11
1.4.1. Phương pháp nghiên cứu	11
1.4.2. Cấu trúc báo cáo	12
CHƯƠNG 2. TỔNG QUAN VỀ HỆ THỐNG VÀ KIẾN TRÚC PHÂN TÁN	14
2.1. Mô tả bài toán nghiệp vụ	14
2.1.1. Quy trình quản lý tài khoản và phân quyền người dùng	14
2.1.2. Quy trình quản lý danh mục món ăn	15
2.1.3. Quy trình tiếp nhận đơn hàng và tính toán hóa đơn.....	16
2.1.4. Quy trình xử lý và gửi thông báo tự động	17
2.2. Kiến trúc tổng thể hệ thống.....	19
2.2.1. Thành phần API Gateway	19
2.2.2. Khối các dịch vụ độc lập.....	20
2.2.3. Hệ thống Message Broker trung gian	21
2.3. Các công nghệ và công cụ phát triển sử dụng.....	22

2.3.1. Ngôn ngữ Python và Framework FastAPI.....	22
2.3.2. Hệ quản trị cơ sở dữ liệu quan hệ MySQL	24
2.3.3. Cơ chế giao tiếp hàng đợi thông điệp với RabbitMQ	25
2.3.4. Công nghệ đóng gói và ảo hóa Container.....	26
2.3.5. Công nghệ phát triển giao diện người dùng.....	27
2.3.6. Hạ tầng thu thập Nhật ký tập trung và Giám sát dịch vụ	28
CHƯƠNG 3. THIẾT KẾ CHI TIẾT CÁC MICROSERVICES VÀ CƠ CHẾ GIAO TIẾP	30
3.1. Thiết kế các dịch vụ thành phần.....	30
3.1.1. API Gateway	30
3.1.2. Auth Service	31
3.1.3. Product Service.....	33
3.1.4. Order Service.....	34
3.1.5. Notification Service	36
3.1.6. Payment Service	38
3.2. Thiết kế Cơ sở dữ liệu độc lập	39
3.2.1. Cơ sở dữ liệu auth_db.....	39
3.2.2. Cơ sở dữ liệu product_db.....	40
3.2.3. Cơ sở dữ liệu order_db	42
3.2.4. Cơ sở dữ liệu notification_db.....	44
3.2.5. Cơ sở dữ liệu payment_db	45
3.3. Thiết kế giao tiếp đồng bộ bằng RESTful API	47
3.3.1. Nguyên tắc thiết kế tài nguyên và ma trận phương thức HTTP	47
3.3.2. Danh sách cấu trúc các API Endpoint chi tiết.....	51
3.3.3. Cơ chế giao tiếp đồng bộ nội bộ giữa Order Service và Product Service ..	60
3.4. Thiết kế giao tiếp bất đồng bộ qua Broker RabbitMQ	62

3.4.1. Cấu hình kiến trúc RabbitMQ	62
3.4.2. Sơ đồ trình tự luồng xử lý nghiệp vụ bất đồng bộ	63
3.4.3. Đánh giá tính nhất quán dữ liệu giữa các dịch vụ	66
3.5. Thiết kế cơ chế xử lý lỗi cơ bản và ghi nhận nhật ký	68
3.5.1. Giải pháp xử lý khi dịch vụ liên quan tạm thời không phản hồi	68
3.5.2. Giải pháp xử lý khi message trên queue bị lỗi hoặc không thể tiêu thụ	69
3.5.3. Định dạng cấu trúc log theo dõi Request HTTP và Message Event	71
CHƯƠNG 4: TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	75
4.1. Môi trường triển khai và cấu hình Docker	75
4.1.1. Cấu hình tệp điều phối hệ thống docker-compose.yml	75
4.1.2. Hiện thực hóa việc cô lập các dịch vụ chạy độc lập (Ảnh chụp các container vận hành)	77
4.2. Kiểm thử luồng nghiệp vụ liên dịch vụ (End-to-End Testing qua Swagger UI)	78
4.2.1. Kịch bản xác thực: Đăng ký tài khoản và Đăng nhập lấy mã JWT thành công	78
4.2.2. Kịch bản phân quyền: Thực hiện quyền Admin để cập nhật danh mục sản phẩm	81
4.2.3. Kịch bản liên dịch vụ đồng bộ: Đặt hàng thành công	82
4.3. Kiểm thử và Giám sát luồng Message Queue kết hợp Log hệ thống	85
4.3.1. Giám sát trạng thái hàng đợi thực tế trên giao diện RabbitMQ Dashboard	85
4.3.2. Kiểm tra Log Console chứng minh quá trình ghi nhận request và xử lý tin nhắn thành công	87
4.3.3. Kiểm tra kết quả lưu trữ thông báo cuối cùng	90
4.4. Thử nghiệm thực tế trên giao diện Frontend và Dashboard giám sát	93
4.4.1. Minh chứng vận hành tương tác trực quan và nhận thông báo đẩy trên UI	93

4.4.2. Kết quả truy vấn Log tập trung và trực quan hóa tài nguyên phần cứng ...	94
4.5. Đánh giá kết quả đạt được so với mức tối thiểu của yêu cầu bài toán.....	95
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	97
TÀI LIỆU THAM KHẢO.....	100

DANH MỤC BẢNG BIỂU

Bảng 3.1: Bảng auth_db.....	40
Bảng 3.2: Bảng product_db	41
Bảng 3.3: Bảng order_db	43
Bảng 3.4: Bảng notification_db	45
Bảng 3.5: Bảng payment_db	47
Bảng 3.6. Ma trận phân bố phương thức HTTP và tài nguyên hệ thống.....	51
Bảng 3.7. Cấu trúc Request/Response của API POST /api/auth/register	52
Bảng 3.8. Cấu trúc Request/Response của API POST /api/auth/login.....	53
Bảng 3.9. Cấu trúc Response của API GET /api/products	54
Bảng 3.10. Cấu trúc Request/Response của API POST /api/products	55
Bảng 3.11. Cấu trúc Request/Response của API POST /api/orders.....	55
Bảng 3.12. Cấu trúc Response của API GET /api/notifications	56
Bảng 3.13. Cấu trúc Request/Response của API POST /api/payments.....	58
Bảng 3.14. Cấu trúc Request/Response của API PATCH /api/payments/{id}/status ...	59
Bảng 3.15. Cấu trúc Response của API GET /api/payments/order/{order_id}	60

DANH MỤC HÌNH ẢNH

Hình 2.1: Sơ đồ kiến trúc hệ thống.....	19
Hình 4.1. Cấu hình container trong tệp điều phối hệ thống.....	77
Hình 4.2: Hình ảnh các container đã chạy thành công.....	78
Hình 4.3: Giao diện giám sát các service.....	95

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và tên	Mã sinh viên	Công việc
Trịnh Lê Xuân Bách	B22DCCN056	Auth service + Config + FE
Trần Đức Dũng	B22DCCN139	Product Service + API gateway
Lê Đức Thiện	B22DCCN823	Order Service + Payment Service + Notification

CHƯƠNG 1. GIỚI THIỆU CHUNG

1.1. Lý do chọn đề tài

Trong kỷ nguyên số hóa hiện nay, xu hướng mua sắm và đặt đồ ăn trực tuyến đã trở thành một phần thiết yếu trong đời sống hằng ngày của người tiêu dùng. Sự bùng nổ về số lượng người dùng cùng với tính chất biến động theo khung giờ của dịch vụ này đặt ra những thách thức rất lớn cho các hệ thống phần mềm backend. Hệ thống không chỉ đòi hỏi khả năng xử lý lượng lớn truy cập đồng thời tại các thời điểm cao điểm (giờ trưa, giờ tối), mà còn phải duy trì tính sẵn sàng cao, tốc độ phản hồi nhanh và khả năng mở rộng linh hoạt theo nhu cầu thực tế.

Đối với các phương pháp phát triển phần mềm truyền thống, kiến trúc Nguyên khối (Monolithic) thường là lựa chọn phổ biến do tính đơn giản trong giai đoạn đầu triển khai. Tuy nhiên, khi hệ thống đặt đồ ăn phát triển mở rộng với nhiều luồng nghiệp vụ phức tạp đan xen — như xác thực người dùng, quản lý thực đơn, xử lý đơn hàng và gửi thông báo — kiến trúc Monolithic bắt đầu lộ rõ những hạn chế chí tử. Việc tất cả các thành phần bị đóng gói chặt chẽ trong một khối duy nhất khiến hệ thống đối mặt với nguy cơ đổ vỡ dây chuyền (Single Point of Failure); chỉ cần một module xảy ra lỗi hoặc quá tải (ví dụ: luồng gửi thông báo bị nghẽn), toàn bộ ứng dụng sẽ bị tê liệt. Đồng thời, việc nâng cấp mở rộng hay bảo trì một tính năng nhỏ cũng đòi hỏi phải tái triển khai lại toàn bộ hệ thống, gây lãng phí tài nguyên và làm giảm tính linh hoạt của doanh nghiệp.

Để giải quyết triệt để bài toán trên, kiến trúc Vi dịch vụ (Microservices) kết hợp với mô hình Hướng sự kiện (Event-driven Architecture) đã ra đời và trở thành chuẩn mực công nghệ mới. Bằng cách phân rã hệ thống thành các dịch vụ nhỏ, hoạt động độc lập và sở hữu vùng dữ liệu riêng biệt (Database-per-service), kiến trúc này cho phép mỗi phân hệ nghiệp vụ tự tối ưu hóa hiệu năng và mở rộng quy mô một cách biệt lập. Bên cạnh đó, việc kết hợp giao tiếp đồng bộ qua RESTful API cho các tác vụ cần phản hồi ngay (như kiểm tra giá món ăn) và giao tiếp bất đồng bộ qua hàng đợi thông điệp Message Queue (RabbitMQ) cho các tác vụ nền (như gửi thông báo) giúp giải phóng tài nguyên xử lý của hệ thống, tăng cường khả năng chịu lỗi và đảm bảo tính nhất quán sau cùng của dữ liệu trong môi trường phân tán.

Nhận thức được tầm quan trọng của các nguyên lý thiết kế hệ thống phân tán hiện đại, đồng thời nhằm bám sát các yêu cầu kỹ thuật cốt lõi của môn học, nhóm chúng em đã quyết định lựa chọn đề tài: *"Xây dựng hệ thống đặt đồ ăn trực tuyến dựa trên kiến trúc Microservices"*. Việc hiện thực hóa đề tài này không chỉ dừng lại ở việc giải quyết một bài toán thực tế sinh động, mà còn là cơ hội để nhóm trực tiếp áp dụng, thử nghiệm và đánh giá các công nghệ cốt lõi bao gồm Framework FastAPI, hệ quản trị MySQL biệt lập, hệ thống điều phối thông điệp RabbitMQ và giải pháp đóng gói hạ tầng container hóa với Docker Compose.

1.2. Mục tiêu của đề tài

Mục tiêu cốt lõi của đề án này là nghiên cứu, thiết kế và xây dựng hoàn chỉnh một ứng dụng đặt đồ ăn trực tuyến (Food Ordering System) thực tế dựa trên việc áp dụng thực tiễn các nguyên lý thiết kế của kiến trúc phân tán. Đề án hướng tới việc hiện thực hóa mô hình Vi dịch vụ (Microservices) kết hợp cơ chế Hướng sự kiện (Event-driven) nhằm tối ưu hóa hiệu năng, tăng cường tính sẵn sàng và khả năng mở rộng của hệ thống backend trước bài toán tải cao và xử lý đồng thời.

Để đạt được mục tiêu tổng thể trên, đề án tập trung hoàn thành các mục tiêu cụ thể sau đây:

Trước hết, về mặt kiến trúc và dữ liệu, đề án hướng đến việc phân rã bài toán nghiệp vụ phức tạp thành 05 dịch vụ thành phần hoạt động hoàn toàn độc lập bao gồm: API Gateway, Auth Service, Product Service, Order Service và Notification Service. Đi kèm với sự phân tách này là việc áp dụng triệt để nguyên lý "Database-per-service", cấu hình cho mỗi dịch vụ sở hữu một phân vùng cơ sở dữ liệu MySQL vật lý riêng biệt (auth_db, product_db, order_db, notification_db). Mục tiêu của việc thiết kế này nhằm loại bỏ hoàn toàn tình trạng nghẽn cổ chai dữ liệu, đảm bảo tính đóng gói và cho phép các dịch vụ có thể triển khai, bảo trì hoặc nâng cấp độc lập mà không gây ảnh hưởng đến các thành phần còn lại.

Thứ hai, về cơ chế giao tiếp liên dịch vụ, đề án đặt mục tiêu kết hợp nhuần nhuyễn hai phương thức giao tiếp đồng bộ và bất đồng bộ nhằm tối ưu hóa tài nguyên. Hệ thống ứng dụng chuẩn RESTful API với đầy đủ các phương thức HTTP (GET, POST, PUT, DELETE) cho cổng kết nối API Gateway và cho luồng giao tiếp đồng bộ nội bộ khi Order Service gọi trực tiếp sang Product Service để xác thực đơn giá món ăn. Song song

với đó, hệ thống tích hợp thành công Message Broker (RabbitMQ) để xử lý luồng giao tiếp bất đồng bộ; khi đơn hàng được khởi tạo thành công, một sự kiện (order.created) sẽ được đẩy vào hàng đợi thông điệp để Notification Service tiêu thụ và lưu trữ, giúp giải phóng luồng xử lý chính của dịch vụ đặt hàng, nâng cao trải nghiệm người dùng và đảm bảo tính nhất quán sau cùng của dữ liệu (Eventual Consistency).

Cuối cùng, về mặt triển khai và vận hành, đồ án hướng tới việc container hóa toàn bộ mã nguồn của các dịch vụ backend cùng các hạ tầng phụ trợ (MySQL, RabbitMQ) bằng công nghệ Docker. Thông qua việc cấu hình tệp điều phối docker-compose.yml, mục tiêu đề ra là đóng gói toàn bộ hệ thống thành một hạ tầng nhất quán, cho phép khởi chạy, cô lập môi trường và giám sát hệ thống một cách tự động chỉ với một lệnh thực thi duy nhất. Đồng thời, đồ án thiết kế cơ chế ghi nhận nhật ký (Logging) cơ bản ở mức console cho từng container, giúp quản trị viên dễ dàng theo dõi luồng request HTTP và tiến trình xử lý message event thực tế trong môi trường phân tán.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu trọng tâm của đồ án này bao gồm cả hai khía cạnh: nền tảng lý thuyết về kiến trúc hệ thống và giải pháp hiện thực hóa phần mềm thực tế. Về mặt lý thuyết, đồ án tập trung nghiên cứu sâu vào mô hình kiến trúc Vi dịch vụ (Microservices Architecture) và mô hình tương tác Hướng sự kiện (Event-driven Architecture) trong môi trường phân tán. Trong đó, các cơ chế giao tiếp liên dịch vụ bao gồm giao tiếp đồng bộ qua giao thức RESTful API và giao tiếp bất đồng bộ qua hàng đợi thông điệp Message Queue là những đối tượng được đặt lên hàng đầu. Song song với đó, đồ án cũng nghiên cứu nguyên lý cô lập dữ liệu theo mô hình "Database-per-service", các giải pháp xử lý lỗi cơ bản và cơ chế giám sát hệ thống thông qua việc lưu vết nhật ký hoạt động (Logging) của các thành phần trong mạng phân tán.

Về mặt thực nghiệm, đối tượng ứng dụng cụ thể của đồ án là "Hệ thống đặt đồ ăn trực tuyến (Food Ordering System)". Hệ thống này được hiện thực hóa thông qua các công nghệ backend và hạ tầng hiện đại bao gồm ngôn ngữ lập trình Python, framework FastAPI, hệ quản trị cơ sở dữ liệu quan hệ MySQL, hệ thống điều phối thông điệp trung gian RabbitMQ và công nghệ container hóa Docker kết hợp công cụ điều phối Docker Compose.

1.3.2. Phạm vi nghiên cứu

Về mặt không gian nghiệp vụ, đồ án giới hạn phạm vi trong các miền chức năng cốt lõi, đại diện cho quy trình vận hành cơ bản của một ứng dụng đặt đồ ăn trực tuyến. Phạm vi chức năng được khoanh vùng trong 05 cấu phần dịch vụ chạy độc lập bao gồm: dịch vụ cổng điều hướng trung tâm (API Gateway) , dịch vụ quản lý tài khoản và xác thực mã thông báo người dùng (Auth Service) , dịch vụ quản lý danh mục và thực đơn món ăn (Product Service) , dịch vụ tiếp nhận giỏ hàng và tính toán tạo hóa đơn đặt hàng (Order Service) , và dịch vụ xử lý lưu vết thông báo tự động (Notification Service). Các luồng nghiệp vụ phức tạp khác ngoài thực tế như tích hợp cổng thanh toán trực tuyến, theo dõi định vị tài xế theo thời gian thực hay quản lý tối ưu hóa kho nguyên liệu (Inventory) tạm thời nằm ngoài phạm vi xử lý của đồ án này.

Về mặt kỹ thuật và vận hành, hệ thống được cấu hình, triển khai và đánh giá hoàn toàn trên môi trường giả lập cục bộ (Local Environment) sử dụng hạ tầng ảo hóa được cung cấp bởi Docker và Docker Compose. Cơ chế giao tiếp hàng đợi thông điệp được tập trung thử nghiệm và chứng minh độ tin cậy thông qua một luồng nghiệp vụ liên dịch vụ duy nhất: sự kiện khởi tạo đơn hàng từ Order Service được truyền tải và tiêu thụ bởi Notification Service qua RabbitMQ. Việc cấu hình phân tán nâng cao như triển khai trên các nền tảng đám mây (Cloud) hay quản lý container tự động bằng Kubernetes (K8s) được giới hạn trong phần định hướng phát triển tương lai.

1.4. Phương pháp nghiên cứu và cấu trúc báo cáo

1.4.1. Phương pháp nghiên cứu

Để hoàn thành các mục tiêu đề ra, đồ án này kết hợp đồng thời giữa phương pháp nghiên cứu lý thuyết và phương pháp thực nghiệm phát triển phần mềm, cụ thể:

- **Phương pháp nghiên cứu lý thuyết:** Nhóm tiến hành thu thập, phân tích và tổng hợp các tài liệu khoa học, bài báo và tài liệu kỹ thuật chính thống liên quan đến kiến trúc hệ thống phân tán, mô hình Microservices và kiến trúc Hướng sự kiện (Event-driven). Trọng tâm lý thuyết hướng vào việc làm rõ cơ chế định tuyến của API Gateway , nguyên lý phân tách dữ liệu "Database-per-service" , các phương thức giao tiếp liên dịch vụ (đồng bộ qua RESTful API và bất đồng bộ qua Message Queue), cùng các giải pháp xử lý lỗi cơ bản trong môi trường phân tán.

- **Phương pháp thực nghiệm phát triển:** Trên cơ sở lý thuyết đã nghiên cứu, nhóm áp dụng quy trình phát triển thực nghiệm để xây dựng hệ thống phần mềm thực tế. Quá trình này bao gồm việc phân rã nghiệp vụ bài toán đặt đồ ăn, thiết kế đặc tả các API Endpoint đúng chuẩn RESTful, và cấu hình hệ thống hàng đợi thông điệp RabbitMQ. Toàn bộ mã nguồn được hiện thực hóa bằng ngôn ngữ Python (FastAPI), lưu trữ dữ liệu trên hệ quản trị MySQL, đóng gói hạ tầng bằng Docker Compose và tiến hành kiểm thử liên dịch vụ (End-to-End) thông qua giao diện Swagger UI để đánh giá tính đúng đắn của hệ thống.

1.4.2. Cấu trúc báo cáo

Nội dung của báo cáo đồ án được cấu trúc hóa một cách logic thành 04 chương chính như sau:

- **Chương 1: Giới thiệu chung:** Giới thiệu tổng quan về lý do lựa chọn đề tài, mục tiêu cần đạt được, đối tượng, phạm vi nghiên cứu cùng phương pháp tiếp cận xuyên suốt của đồ án.
- **Chương 2: Tổng quan về hệ thống và kiến trúc phân tán:** Tập trung mô tả chi tiết bài toán nghiệp vụ của hệ thống đặt đồ ăn; phân tích và phác thảo sơ đồ kiến trúc tổng thể của hệ thống dựa trên mô hình Microservices kết hợp với Message Broker; đồng thời giới thiệu các công nghệ, công cụ nền tảng được lựa chọn để hiện thực hóa ứng dụng.
- **Chương 3: Thiết kế chi tiết các Microservices và cơ chế giao tiếp:** Đây là chương nội dung trọng tâm, trình bày chi tiết việc phân rã chức năng của 05 dịch vụ thành phần; thiết kế cấu trúc dữ liệu độc lập (Database-per-service); đặc tả ma trận RESTful API và sơ đồ trình tự (Sequence Diagram) luồng thông điệp bất đồng bộ qua RabbitMQ; ngoài ra chương này cũng mô tả giải pháp thiết kế cơ chế lưu vết nhật ký (Logging) và xử lý lỗi cơ bản của hệ thống.
- **Chương 4: Triển khai, thử nghiệm và đánh giá kết quả:** Trình bày chi tiết quá trình cấu hình tệp điều phối Docker Compose để vận hành hệ thống chạy độc lập; thực hiện chạy thử nghiệm thực tế theo các kịch bản liên nghiệp vụ; minh chứng kết quả giao tiếp đồng bộ và bất đồng bộ thông qua ảnh chụp màn hình Swagger UI, RabbitMQ Dashboard cùng Log Console; từ đó đưa ra những đánh giá khách

quan về kết quả đạt được, các hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2. TỔNG QUAN VỀ HỆ THỐNG VÀ KIẾN TRÚC PHÂN TÁN

2.1. Mô tả bài toán nghiệp vụ

2.1.1. Quy trình quản lý tài khoản và phân quyền người dùng

Trong cấu trúc của một hệ thống phân tán theo kiến trúc Microservices, việc quản lý định danh và phân quyền người dùng đặt ra một bài toán phức tạp do tính chất phân tách của các dịch vụ backend. Để giải quyết bài toán này mà không làm mất đi tính độc lập của từng phân hệ, hệ thống ứng dụng quy trình quản lý tài khoản tập trung kết hợp cơ chế xác thực phi trạng thái (Stateless Authentication) dựa trên chuẩn mã thông báo JSON Web Token (JWT). Quy trình này đảm bảo rằng thông tin định danh của người dùng chỉ cần được khởi tạo một lần duy nhất tại dịch vụ xác thực, sau đó được lan truyền an toàn qua môi trường mạng đến các dịch vụ nghiệp vụ khác.

Giai đoạn đầu tiên của quy trình là khởi tạo tài khoản và thiết lập phiên truy cập. Khi người dùng thực hiện đăng ký thông qua API Gateway, yêu cầu sẽ được điều phối đến auth-service. Tại đây, thông tin tài khoản được kiểm tra tính hợp lệ và lưu trữ vào cơ sở dữ liệu `auth_db`. Khi người dùng tiến hành đăng nhập bằng tài khoản hợp lệ, auth-service sẽ tiến hành mã hóa các thông tin định danh cốt lõi (chính là các thông tin như `user_id`, `username`, và vai trò hệ thống `role`) cùng với thời gian hết hạn thành một chuỗi mã thông báo JWT duy nhất, được ký bằng một khóa bí mật (Secret Key) theo thuật toán mã hóa đối xứng. Chuỗi JWT này sau đó được trả về cho phía ứng dụng khách (Client) để làm minh chứng xác thực cho các yêu cầu giao dịch tiếp theo.

Giai đoạn tiếp theo là quá trình xác thực và phân quyền dựa trên mã thông báo trong luồng nghiệp vụ. Khi Client gửi yêu cầu thực hiện một tác vụ — ví dụ như khởi tạo đơn hàng hay cập nhật thực đơn — chuỗi mã thông báo JWT bắt buộc phải được đính kèm vào phần tiêu đề của yêu cầu HTTP (HTTP Authorization Header). API Gateway đóng vai trò là chốt chặn đầu tiên, tiếp nhận yêu cầu và kiểm tra sự hiện diện của mã thông báo. Sau khi vượt qua Gateway, yêu cầu được định tuyến đến dịch vụ backend mục tiêu (như `product-service` hoặc `order-service`). Tại đây, dịch vụ đích sẽ tự giải mã (decode) chuỗi JWT bằng khóa bí mật chung mà không cần phải thực hiện một truy vấn mạng ngược trở lại auth-service, giúp giảm thiểu tối đa độ trễ hệ thống và loại bỏ tình trạng nghẽn cổ chai tại dịch vụ xác thực.

Cuối cùng, cơ chế phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) được thực thi trực tiếp tại các endpoint của từng vi dịch vụ. Sau khi giải mã thành công thông tin từ JWT, hệ thống sẽ trích xuất trường dữ liệu role để quyết định cấp độ quyền hạn của tài khoản hiện tại. Cụ thể, đối với các tác vụ thông thường như xem danh mục món ăn hay khởi tạo hóa đơn, hệ thống cho phép tất cả người dùng có vai trò khách hàng truy cập. Ngược lại, đối với các tác vụ quản trị có tính chất thay đổi dữ liệu nghiêm trọng như thêm mới, sửa đổi hoặc xóa bỏ các món ăn trong hệ thống, dịch vụ product-service sẽ thực hiện kiểm tra điều kiện nghiêm ngặt; nếu trường role không phải là admin, hệ thống sẽ lập tức từ chối và phản hồi lỗi phân quyền đến người dùng, đảm bảo tính toàn vẹn và an toàn thông tin cho toàn bộ hệ thống đặt đồ ăn.

2.1.2. Quy trình quản lý danh mục món ăn

Trong hệ thống đặt đồ ăn trực tuyến, danh mục thực đơn đóng vai trò là luồng dữ liệu cốt lõi, cung cấp thông tin hiển thị cho khách hàng và là đối tượng quản lý chính của quản trị viên (Admin). Nhằm đảm bảo tính độc lập và tối ưu hóa hiệu năng truy xuất dữ liệu món ăn, quy trình quản lý danh mục thực đơn được đóng gói hoàn toàn bên trong dịch vụ product-service. Dịch vụ này chịu trách nhiệm thực thi trọn vẹn các tác vụ nghiệp vụ cơ bản bao gồm: Khởi tạo (Create), Đọc (Read), Cập nhật (Update) và Xóa bỏ (Delete) thông tin sản phẩm, đồng thời phối hợp chặt chẽ với cơ chế phân quyền hệ thống để bảo vệ an toàn dữ liệu.

Quy trình đọc dữ liệu (Read) là tác vụ có tần suất khai thác cao nhất từ phía người dùng, cho phép mọi đối tượng khách hàng truy cập mà không cần xác thực nâng cao. Khi người dùng truy cập ứng dụng để duyệt thực đơn, một yêu cầu HTTP GET sẽ được gửi qua API Gateway và định tuyến đến product-service. Tại đây, dịch vụ thực hiện truy vấn vào phân vùng cơ sở dữ liệu product_db để trích xuất danh sách món ăn, bao gồm thông tin chi tiết về tên món, mô tả, đơn giá và trạng thái khả dụng. Nhờ cơ chế bất đồng bộ (Asynchronous) của framework FastAPI, luồng xử lý này có thể tiếp nhận và phản hồi đồng thời lượng lớn request từ các Client khác nhau với độ trễ tối thiểu, đảm bảo trải nghiệm mượt mà cho khách hàng trong quá trình chọn món.

Ngược lại với tác vụ đọc, các quy trình thay đổi trạng thái dữ liệu bao gồm Khởi tạo (Create), Cập nhật (Update) và Xóa bỏ (Delete) thực đơn yêu cầu một quy trình kiểm soát nghiêm ngặt nhằm đảm bảo tính toàn vẹn. Khi quản trị viên thực hiện thêm mới

một món ăn (HTTP POST), chỉnh sửa giá cả/thông tin (HTTP PUT) hoặc gỡ bỏ sản phẩm khỏi hệ thống (HTTP DELETE), yêu cầu bắt buộc phải đi kèm mã thông báo JWT hợp lệ trong tiêu đề request. Dịch vụ product-service sau khi tiếp nhận thông tin sẽ tiến hành giải mã mã thông báo để xác minh vai trò (role); nếu định danh không phải là admin, hệ thống sẽ lập tức từ chối thực thi và trả về mã lỗi phân quyền. Quy trình này giúp ngăn chặn hoàn toàn các hành vi thay đổi dữ liệu trái phép từ phía người dùng thông thường.

Khi một yêu cầu thay đổi dữ liệu từ Admin được xác thực thành công, dịch vụ sẽ tiến hành kiểm tra tính hợp lệ của dữ liệu đầu vào (Data Validation) thông qua các mô hình Pydantic được định nghĩa sẵn. Dữ liệu sau khi vượt qua bước kiểm tra (ví dụ: tên món ăn không được để trống, đơn giá phải là số dương) mới chính thức được ghi nhận và đồng bộ trực tiếp vào các bảng tương ứng trong product_db. Việc khu biệt toàn bộ quy trình CRUD sản phẩm trong một dịch vụ và một cơ sở dữ liệu riêng biệt không chỉ giúp cô lập các rủi ro phát sinh lỗi dữ liệu, mà còn tạo nền tảng vững chắc để hệ thống dễ dàng tích hợp các giải pháp tối ưu hóa hiệu năng như bộ nhớ đệm (Caching) cho danh mục món ăn trong tương lai.

2.1.3. Quy trình tiếp nhận đơn hàng và tính toán hóa đơn

Quy trình tiếp nhận đơn hàng và tính toán giá trị hóa đơn là nghiệp vụ trung tâm, kết nối trực tiếp các thực thể dữ liệu và quyết định tính đúng đắn trong vận hành của hệ thống đặt đồ ăn trực tuyến. Do đặc thù của kiến trúc Microservices, thông tin giỏ hàng do người dùng gửi lên chỉ chứa các định danh cơ bản như mã món ăn (product_id) và số lượng đặt mua, nhằm giảm tải dung lượng truyền tải qua mạng. Trách nhiệm tiếp nhận, xác thực thực tế, tính toán tài chính và khởi tạo trạng thái đơn hàng được giao cho dịch vụ order-service phối hợp điều phối cùng cơ sở dữ liệu độc lập order_db.

Giai đoạn đầu tiên của quy trình khởi đầu khi khách hàng gửi một yêu cầu khởi tạo đơn hàng (HTTP POST) kèm theo danh sách giỏ hàng và mã thông báo JWT hợp lệ thông qua API Gateway. Sau khi order-service tiếp nhận request, dịch vụ sẽ trích xuất thông tin định danh user_id từ JWT để làm căn cứ xác định chủ thể đơn hàng. Tuy nhiên, vì order-service không sở hữu thông tin về đơn giá hay trạng thái tồn kho của món ăn, dịch vụ không thể tự tính toán tổng tiền một cách biệt lập. Lúc này, hệ thống bắt buộc phải thực hiện một luồng giao tiếp đồng bộ (Synchronous Communication), gửi yêu cầu

HTTP GET nội bộ trực tiếp sang dịch vụ product-service để truy xuất thông tin đơn giá chính xác và mới nhất của các sản phẩm có trong giỏ hàng.

Sau khi nhận được dữ liệu phản hồi từ product-service, order-service tiến hành giai đoạn tính toán hóa đơn và kiểm tra tính toàn vẹn của dữ liệu. Hệ thống thực hiện vòng lặp để đối chiếu mã sản phẩm, nhân số lượng đặt mua với đơn giá thực tế của từng món ăn nhằm xác định tổng số tiền của từng mục (item_total), từ đó cộng dồn lại thành tổng giá trị cuối cùng của toàn bộ hóa đơn (total_price). Quá trình tính toán này được bảo vệ bởi các mô hình kiểm định dữ liệu đầu vào; nếu có bất kỳ sai lệch nào về logic dữ liệu (chẳng hạn như món ăn không tồn tại hoặc số lượng đặt mua nhỏ hơn hoặc bằng không), hệ thống sẽ ngay lập tức hủy bỏ tiến trình và phản hồi mã lỗi tương ứng về cho người dùng.

Cuối cùng, khi dữ liệu hóa đơn đã được tính toán chính xác, order-service tiến hành đồng bộ và lưu trữ vĩnh viễn thông tin vào cơ sở dữ liệu order_db. Tiến trình ghi dữ liệu được thực hiện đồng thời trên hai bảng có mối quan hệ chặt chẽ: thông tin tổng quan của đơn hàng (bao gồm order_id, user_id, total_price và trạng thái ban đầu) được lưu vào bảng orders, trong khi chi tiết từng món ăn cùng số lượng và đơn giá tại thời điểm đặt được ghi nhận vào bảng order_items. Việc hoàn tất lưu trữ xuống cơ sở dữ liệu vật lý này không chỉ đánh dấu một đơn hàng được tiếp nhận thành công về mặt hệ thống, mà còn là điều kiện tiên quyết để kích hoạt luồng giao tiếp bất đồng bộ kế tiếp qua hàng đợi thông điệp RabbitMQ.

2.1.4. Quy trình xử lý và gửi thông báo tự động

Trong kiến trúc vi dịch vụ, luồng xử lý thông báo cho khách hàng sau khi đặt hàng thành công là một minh chứng điển hình cho việc áp dụng mô hình giao tiếp bất đồng bộ (Asynchronous Communication) nhằm tối ưu hóa hiệu năng hệ thống. Quy trình này được thiết kế tách biệt hoàn toàn khỏi luồng nghiệp vụ chính của đơn hàng và được đảm nhiệm bởi dịch vụ notification-service. Mục tiêu tối cao của quy trình là giải phóng thời gian phản hồi cho luồng đặt hàng, đảm bảo rằng trải nghiệm người dùng không bị chậm trễ do các tác vụ phụ trợ và tăng cường khả năng chịu lỗi của toàn bộ hệ thống.

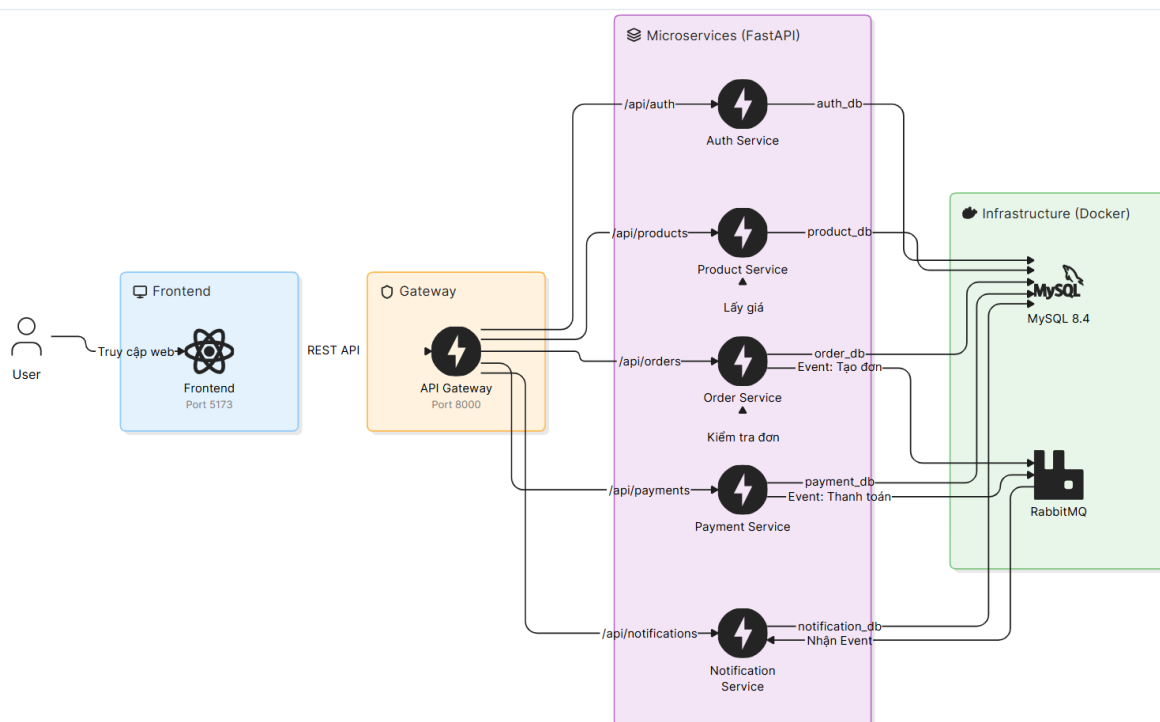
Quy trình được kích hoạt ngay sau khi order-service hoàn tất việc ghi nhận và lưu trữ hóa đơn hợp lệ vào cơ sở dữ liệu order_db. Thay vì phải gửi một yêu cầu trực tiếp và đứng đợi phản hồi từ dịch vụ thông báo (gây tổn tài nguyên và tăng độ trễ mạng),

order-service chỉ đóng vai trò là một nhà xuất bản (Publisher). Dịch vụ này tiến hành đóng gói các thông tin cơ bản của đơn hàng thành một thông điệp dữ liệu dạng JSON — chứa các trường thông tin cốt lõi như mã đơn hàng `order_id` và mã người dùng `user_id` — rồi xuất bản (Publish) thông điệp đó lên hệ thống hàng đợi trung gian RabbitMQ với một khóa định tuyến xác định mang tên `order.created`. Ngay sau khi thông điệp được gửi đi, order-service lập tức trả về phản hồi "Đặt hàng thành công" cho phía khách hàng.

Ở phía bên kia của hệ thống hàng đợi, notification-service vận hành độc lập và đóng vai trò là một thực thể tiêu thụ thông điệp (Consumer). Dịch vụ này thiết lập một tiến trình chạy ngầm liên tục lắng nghe tại một hàng đợi (`notification_queue`) được liên kết chặt chẽ với Exchange của RabbitMQ thông qua khóa định tuyến `order.created`. Ngay khi có một sự kiện tạo đơn hàng mới được đẩy vào hàng đợi, RabbitMQ sẽ tự động phân phối thông điệp này đến cho notification-service xử lý. Cơ chế này đảm bảo rằng ngay cả trong trường hợp dịch vụ thông báo tạm thời bị quá tải hoặc gặp sự cố ngắt kết nối mạng ngắn hạn, thông điệp vẫn được lưu trữ an toàn trong hàng đợi của RabbitMQ và sẽ được tiêu thụ ngay khi dịch vụ phục hồi, ngăn chặn hoàn toàn nguy cơ mất dữ liệu.

Khi tiếp nhận được thông điệp từ hàng đợi, notification-service tiến hành giải mã gói tin JSON để trích xuất các trường thông tin và thực hiện logic nghiệp vụ tự động. Dịch vụ sẽ tự động biên dịch và định dạng một chuỗi nội dung thông báo trực quan dựa trên mã đơn hàng nhận được, ví dụ: *"Đơn hàng số #123 của bạn đã được đặt thành công"*. Chuỗi nội dung này, cùng với thông tin định danh `user_id` và trạng thái ban đầu là "chưa đọc" (`unread`), sẽ được đồng bộ và lưu trữ trực tiếp vào bảng `notifications` thuộc phân vùng cơ sở dữ liệu độc lập `notification_db`. Quy trình kết thúc khi dữ liệu được ghi nhận thành công xuống database, sẵn sàng cung cấp thông tin qua các API Endpoint để người dùng có thể truy vấn và xem lại lịch sử thông báo cá nhân của mình bất cứ lúc nào.

2.2. Kiến trúc tổng thể hệ thống



Hình 2.1: Sơ đồ kiến trúc hệ thống

2.2.1. Thành phần API Gateway

Trong mô hình kiến trúc Vi dịch vụ (Microservices), hệ thống được chia nhỏ thành nhiều dịch vụ độc lập chạy trên các tiến trình và cổng kết nối (Port) khác nhau. Nếu để ứng dụng khách (Client) tương tác trực tiếp với từng dịch vụ đơn lẻ này, hệ thống sẽ đối mặt với nhiều rủi ro về bảo mật, làm tăng độ phức tạp ở phía Client khi phải quản lý quá nhiều địa chỉ URL, đồng thời gây khó khăn cho việc kiểm soát luồng dữ liệu. Để giải quyết triệt để vấn đề này, đồ án đã triển khai thành phần api-gateway đóng vai trò là Cổng kết nối duy nhất (Single Entry Point) trung gian, đứng trước toàn bộ các dịch vụ backend nhằm quản lý và điều phối mọi lưu lượng mạng đi vào hệ thống.

Chức năng cốt lõi và quan trọng nhất của api-gateway là cơ chế định tuyến yêu cầu (Request Routing). Khi vận hành thực tế, API Gateway thiết lập một ma trận định tuyến cố định tại cổng mạng 8000. Khi nhận được bất kỳ yêu cầu HTTP nào từ Client, Gateway sẽ phân tích cấu trúc đường dẫn (URL Path) để nhận diện dịch vụ đích, sau đó thực hiện chuyển tiếp (Proxy) yêu cầu đó đến đúng dịch vụ backend tương ứng đang chạy ngầm ở các cổng nội bộ (ví dụ: định tuyến các yêu cầu có tiền tố `/api/auth` về `auth-service` ở cổng 8001, `/api/products` về `product-service` ở cổng 8002, hay `/api/orders` về `order-service` ở cổng 8003). Cơ chế này giúp che giấu hoàn toàn cấu trúc mạng và sơ đồ

phân bổ công kết nối nội bộ của hệ thống, bảo vệ các dịch vụ backend khỏi các nguy cơ tấn công trực tiếp từ môi trường mạng bên ngoài.

Bên cạnh nhiệm vụ định tuyến, api-gateway còn đóng vai trò là chốt chặn trung tâm để thực hiện kiểm soát và giảm tải biểu mẫu xử lý cho các dịch vụ phía sau. Cụ thể, Gateway hỗ trợ việc tập trung hóa tài liệu giao tiếp API thông qua việc tích hợp giao diện Swagger UI tại một đầu mối duy nhất. Thay vì phải truy cập vào từng dịch vụ riêng lẻ để kiểm thử, giảng viên hoặc người vận hành chỉ cần truy cập vào cổng 8000 của Gateway là có thể bao quát và thực nghiệm toàn bộ luồng nghiệp vụ liên dịch vụ của hệ thống đặt đồ ăn. Ngoài ra, việc đặt Gateway làm điểm tiếp nhận duy nhất tạo điều kiện lý tưởng để hệ thống có thể tích hợp và mở rộng các tính năng chung như giới hạn tốc độ truy cập (Rate Limiting), kiểm tra mã thông báo JWT tập trung, hoặc ghi vết nhật ký hệ thống (Centralized Logging) trong các giai đoạn phát triển nâng cao mà không cần phải chỉnh sửa mã nguồn của từng vi dịch vụ thành phần.

2.2.2. Khối các dịch vụ độc lập

Trọng tâm lớp xử lý nghiệp vụ của hệ thống bao gồm khối **05 dịch vụ cốt lõi (Core Services)** hoạt động hoàn toàn độc lập và tách biệt nhau. Mỗi dịch vụ được thiết kế để đảm nhiệm một phạm vi nghiệp vụ duy nhất (*Single Responsibility Principle*), sở hữu mã nguồn riêng, môi trường thực thi biệt lập và cô lập hoàn toàn về mặt dữ liệu dưới mô hình *Database-per-service*. Sự phân tách này giúp giảm thiểu tối đa mức độ phụ thuộc lẫn nhau (*Loose Coupling*), cho phép nâng cấp cục bộ một phân hệ mà không ảnh hưởng đến tính ổn định của toàn bộ hệ thống đặt đồ ăn trực tuyến.

Chi tiết về chức năng và vai trò vận hành của từng dịch vụ độc lập bao gồm:

- **Auth Service (Dịch vụ xác thực - Cổng 8001):** Chịu trách nhiệm quản lý toàn bộ vòng đời tài khoản của người dùng. Các chức năng chính bao gồm: tiếp nhận đăng ký, mã hóa bảo mật mật khẩu một chiều, kiểm tra thông tin đăng nhập và cấp phát mã thông báo phi trạng thái JSON Web Token (JWT) cho các phiên làm việc của Client.
- **Product Service (Dịch vụ sản phẩm - Cổng 8002):** Chịu trách nhiệm lưu trữ và quản lý thông tin của toàn bộ danh mục thực đơn. Dịch vụ cung cấp các API công khai cho khách hàng tìm kiếm, duyệt món ăn và các API bảo mật (yêu cầu quyền Admin) để thực hiện các thao tác quản trị như thêm, sửa, xóa sản phẩm.

- **Order Service (Dịch vụ đơn hàng - Cổng 8003):** Trung tâm điều phối luồng nghiệp vụ mua sắm của ứng dụng. Dịch vụ tiếp nhận yêu cầu đặt hàng, gọi đồng bộ REST API sang product-service để kiểm tra giá gốc, tính tổng tiền hóa đơn và quản lý trạng thái đơn hàng. Sau khi khởi tạo đơn hàng thành công, dịch vụ lập tức xuất bản sự kiện `order.created` lên hàng đợi RabbitMQ để kích hoạt các tác vụ hậu mãi ngầm.
- **Notification Service (Dịch vụ thông báo - Cổng 8004):** Vận hành theo cơ chế hướng sự kiện dưới vai trò một Consumer ngầm, không tiếp nhận request trực tiếp từ API Gateway. Dịch vụ tập trung lắng nghe, tiêu thụ thông điệp từ hàng đợi RabbitMQ (như sự kiện tạo đơn hàng, thanh toán thành công) để tự động biên dịch, lưu vết và đẩy thông báo thời gian thực tới khách hàng, giúp giảm tải hoàn toàn luồng xử lý hậu mãi ra khỏi tiến trình đặt hàng chính.
- **Payment Service (Dịch vụ thanh toán - Cổng 8005):** Đóng vai trò là "Thủ quỹ" trung tâm, chịu trách nhiệm khởi tạo, xác thực và quản lý các giao dịch tài chính. Nhằm chống gian lận đổi giá, dịch vụ tự động gọi REST API ngầm sang order-service để đối chiếu thông tin chủ đơn hàng và lấy tổng số tiền thực tế trước khi phê duyệt trạng thái pending. Khi nhận kết quả thanh toán thành công (paid), dịch vụ lập tức phát sự kiện bất đồng bộ `payment.paid` lên RabbitMQ để kích hoạt luồng tự động chuyển trạng thái đơn hàng và gửi thông báo xác nhận.

2.2.3. Hệ thống Message Broker trung gian

Trong kiến trúc Vi dịch vụ, nếu tất cả các dịch vụ đều giao tiếp với nhau theo cơ chế đồng bộ (Synchronous), hệ thống sẽ rất dễ rơi vào trạng thái thắt nút cổ chai và làm giảm khả năng chịu lỗi toàn cục. Để khắc phục nhược điểm này và hiện thực hóa mô hình kiến trúc Hướng sự kiện (Event-driven Architecture), đồ án đã tích hợp hệ thống trung gian điều phối thông điệp RabbitMQ đóng vai trò làm Message Broker. Thành phần này hoạt động như một bus cục trung tâm an toàn, chịu trách nhiệm tiếp nhận, lưu trữ tạm thời và phân phối các thông điệp nghiệp vụ giữa các dịch vụ một cách hoàn toàn bất đồng bộ.

Hệ thống RabbitMQ trong đồ án được cấu hình vận hành tại cổng mã hóa dữ liệu mặc định 5672 (AMQP) và cổng hiển thị quản trị 15672. Vai trò của thành phần này được thể hiện rõ nét qua 03 đặc tính kỹ thuật cốt lõi sau:

- **Giải phóng liên kết trực tiếp (Decoupling):** RabbitMQ phá vỡ sự phụ thuộc vật lý giữa dịch vụ gửi và dịch vụ nhận. Khi một đơn hàng được tạo lập thành công, order-service (Publisher) chỉ cần đóng gói dữ liệu và đẩy lên RabbitMQ rồi kết thúc tiến trình. Dịch vụ này hoàn toàn không cần biết notification-service (Consumer) đang ở đâu, trạng thái hoạt động ra sao, giúp mã nguồn của các dịch vụ không bị ràng buộc lẫn nhau.
- **Cân bằng tải và đệm dữ liệu (Buffering/Throttling):** Trong các khung giờ cao điểm, lượng đơn hàng đổ về hệ thống có thể tăng đột biến. Nếu notification-service không thể xử lý kịp thời các yêu cầu lưu vết và gửi thông báo, RabbitMQ sẽ đóng vai trò là một bộ nhớ đệm, lưu trữ an toàn các thông điệp này trong hàng đợi (notification_queue) theo cơ chế FIFO (First In, First Out). Các thông điệp sẽ được xếp hàng và phân phối dần theo đúng năng lực xử lý của dịch vụ tiêu thụ, ngăn chặn hoàn toàn nguy cơ sập dịch vụ do quá tải.
- **Nâng cao tính chịu lỗi (Fault Tolerance):** Một trong những yêu cầu quan trọng của hệ thống phân tán là khả năng hoạt động ổn định ngay cả khi có thành phần gặp sự cố. Nhờ có RabbitMQ, nếu notification-service tạm thời bị ngắt kết nối hoặc dừng hoạt động để bảo trì, luồng đặt hàng của khách hàng trên ứng dụng vẫn diễn ra hoàn toàn bình thường. Thông điệp tạo đơn hàng không bị mất đi mà được giữ lại trên hàng đợi, sẵn sàng phân phối ngay khi dịch vụ thông báo khởi chạy trở lại.

Cơ chế phân phối thông điệp của hệ thống được tổ chức chặt chẽ thông qua việc thiết lập một Exchange trung gian mang tên order_exchange kết nối với hàng đợi notification_queue bằng khóa định tuyến (Routing Key) có tên là order.created. Sự xuất hiện của RabbitMQ không chỉ giải quyết trọn vẹn yêu cầu giao tiếp bất đồng bộ của đề tài, mà còn là nền tảng cốt lõi giúp hệ thống đạt được trạng thái nhất quán sau cùng của dữ liệu (Eventual Consistency) mà không làm suy giảm hiệu năng tổng thể.

2.3. Các công nghệ và công cụ phát triển sử dụng

2.3.1. Ngôn ngữ Python và Framework FastAPI

Để xây dựng lớp xử lý nghiệp vụ backend cho một hệ thống phân tán đòi hỏi hiệu năng cao và khả năng mở rộng linh hoạt, đồ án đã lựa chọn ngôn ngữ lập trình Python kết hợp cùng framework FastAPI. Python từ lâu đã khẳng định được vị thế là một ngôn

ngữ lập trình mạnh mẽ, sở hữu cú pháp tường minh, dễ bảo trì và có một hệ sinh thái thư viện hỗ trợ phong phú. Tuy nhiên, đối với các ứng dụng phân tán có mật độ request lớn, việc lựa chọn một framework phù hợp là yếu tố quyết định để tối ưu hóa tài nguyên phần cứng. FastAPI xuất hiện như một giải pháp công nghệ hiện đại, đáp ứng trọn vẹn cả yêu cầu về tốc độ phát triển mã nguồn lẫn hiệu năng vận hành thực tế.

Đặc tính kỹ thuật đắt giá nhất của FastAPI chính là hỗ trợ toàn diện cơ chế lập trình bất đồng bộ (Asynchronous Programming) dựa trên các từ khóa `async` và `await` của Python. Trong một hệ thống đặt đồ ăn, phần lớn thời gian xử lý của các dịch vụ bị tiêu tốn vào các tác vụ I/O Bound như đợi truy vấn cơ sở dữ liệu MySQL, gọi RESTful API liên dịch vụ hoặc tương tác với Message Broker RabbitMQ. Với các framework đồng bộ truyền thống, mỗi request sẽ chiếm dụng một thread của hệ thống, gây lãng phí tài nguyên khi thread phải đứng đợi phản hồi mạng. Ngược lại, FastAPI tận dụng cơ chế Vòng lặp sự kiện (Event Loop), cho phép một tiến trình duy nhất có thể tiếp nhận và xử lý hàng ngàn yêu cầu đồng thời. Khi một request đang đứng đợi dữ liệu từ cơ sở dữ liệu hoặc dịch vụ khác, Event Loop sẽ lập tức chuyển quyền xử lý sang cho request kế tiếp, giúp tối ưu hóa hiệu suất luồng mạng và giảm thiểu tối đa độ trễ.

Bên cạnh hiệu năng vượt trội, FastAPI còn tích hợp sẵn thư viện Pydantic để thực hiện cơ chế kiểm định dữ liệu (Data Validation) và quản lý biểu mẫu dữ liệu một cách tự động. Khi Client gửi các yêu cầu tạo mới sản phẩm hay khởi tạo đơn hàng, FastAPI sẽ tự động đối chiếu cấu trúc gói tin JSON đầu vào với các mô hình dữ liệu (Pydantic Models) được định nghĩa sẵn trong mã nguồn. Nếu dữ liệu truyền lên sai định dạng hoặc thiếu các trường bắt buộc, framework sẽ tự động trả về lỗi mã trạng thái HTTP thích hợp (như lỗi 422 Unprocessable Entity) kèm theo mô tả chi tiết chi tiết mà nhà phát triển không cần phải viết thêm các câu lệnh kiểm tra thủ công. Tính năng này giúp mã nguồn của hệ thống trở nên sạch sẽ, an toàn và giảm thiểu tối đa các lỗi hổng logic dữ liệu.

Cuối cùng, một ưu điểm thực tế giúp FastAPI hỗ trợ đắc lực cho việc phát triển và kiểm thử hệ thống phân tán là khả năng tự động sinh tài liệu định chuẩn OpenAPI (Swagger UI). Ngay khi các endpoint được khởi tạo, FastAPI sẽ tự động biên dịch cấu trúc và hiển thị giao diện kiểm thử trực quan tại cổng mạng của từng dịch vụ. Trong đồ án này, tính năng này đã được tận dụng tối đa để tích hợp vào thành phần API Gateway ở cổng 8000. Người vận hành hay giảng viên chấm điểm chỉ cần thông qua một giao

diện Swagger duy nhất là có thể bao quát toàn bộ đặc tả kỹ thuật, cấu trúc đầu vào/đầu ra của các API và trực tiếp thực nghiệm luồng chạy liên dịch vụ của hệ thống một cách nhanh chóng và chính xác.

2.3.2. Hệ quản trị cơ sở dữ liệu quan hệ MySQL

Trong thiết kế kiến trúc hệ thống phân tán, việc lưu trữ và quản trị dữ liệu đòi hỏi một giải pháp vừa đảm bảo tính toàn vẹn cấu trúc nghiệp vụ, vừa hỗ trợ khả năng phân tách độc lập giữa các dịch vụ thành phần. Để đáp ứng yêu cầu này, đồ án đã lựa chọn hệ quản trị cơ sở dữ liệu quan hệ MySQL. Là một mã nguồn mở phổ biến, MySQL cung cấp hiệu năng truy xuất mạnh mẽ, khả năng bảo mật dữ liệu cao và hỗ trợ toàn diện cơ chế giao dịch ACID (Atomicity, Consistency, Isolation, Durability) – một yếu tố quan trọng để đảm bảo tính chính xác cho các thực thể dữ liệu như hóa đơn tài chính và thông tin người dùng trong hệ thống đặt đồ ăn.

Để hiện thực hóa nguyên lý cô lập dữ liệu tối cao trong kiến trúc Vi dịch vụ là "Database-per-service", hệ thống không sử dụng một cơ sở dữ liệu tập trung duy nhất. Thay vào đó, hạ tầng MySQL được cấu hình để phân rã hoàn toàn thành 04 phân vùng cơ sở dữ liệu độc lập về mặt vật lý bao gồm: `auth_db`, `product_db`, `order_db` và `notification_db`. Mỗi vi dịch vụ chỉ được cấp quyền kết nối và thao tác trực tiếp với phân vùng dữ liệu do chính nó quản lý. Sự phân tách này triệt tiêu hoàn toàn rủi ro nghẽn cổ chai dữ liệu (Database Bottleneck) của kiến trúc nguyên khối, đồng thời nâng cao tính chịu lỗi; nếu một phân vùng dữ liệu gặp sự cố (ví dụ: `notification_db` bị khóa), các dịch vụ còn lại vẫn truy xuất database bình thường để phục vụ luồng chọn món và đặt hàng của khách hàng.

Về mặt cấu hình triển khai và kết nối mạng nội bộ, hạ tầng MySQL được đóng gói bên trong một container Docker độc lập. Để tối ưu hóa việc quản trị và tránh xung đột với các tiến trình database có sẵn trên máy chủ (Host), cổng kết nối của MySQL được thiết lập theo cơ chế ánh xạ thông minh. Cụ thể, cổng nội bộ bên trong mạng Docker được giữ ở cổng tiêu chuẩn 3306 để các dịch vụ backend kết nối trực tiếp với nhau, trong khi cổng xuất bản ra môi trường máy chủ bên ngoài được cấu hình thành 3307. Điều này cho phép người vận hành có thể sử dụng các công cụ quản trị trực quan từ bên ngoài để truy vấn dữ liệu một cách an toàn mà không làm ảnh hưởng đến đường truyền nội bộ của hệ thống.

Nhằm tự động hóa quy trình khởi tạo hệ thống và đảm bảo tính nhất quán của cấu trúc dữ liệu trên mọi môi trường triển khai, đề án tích hợp một tệp kịch bản cấu hình mang tên `init-mysql.sql`. Tệp kịch bản này được gắn kết (Mount) trực tiếp vào tiến trình khởi chạy ban đầu của container MySQL. Khi hệ thống được kích hoạt thông qua Docker Compose, kịch bản sẽ tự động quét, khởi tạo toàn bộ 04 cơ sở dữ liệu thành phần và thiết lập cấu trúc các bảng (Tables) cùng các ràng buộc khóa ngoại cần thiết. Giải pháp này giúp loại bỏ hoàn toàn các bước cấu hình thủ công phức tạp, hiện thực hóa mục tiêu triển khai hệ thống phân tán một cách nhanh chóng, đồng bộ và tin cậy.

2.3.3. Cơ chế giao tiếp hàng đợi thông điệp với RabbitMQ

Song song với việc sử dụng giao thức RESTful API cho các tác vụ đồng bộ, cơ chế giao tiếp hàng đợi thông điệp (Message Queue) là thành phần cốt lõi được đề án tích hợp để hiện thực hóa luồng xử lý bất đồng bộ giữa các dịch vụ độc lập. Bằng việc sử dụng hệ thống trung gian RabbitMQ đóng vai trò làm Message Broker, hệ thống cho phép các dịch vụ truyền tải thông tin dưới dạng các sự kiện (Event-driven) mà không cần duy trì kết nối trực tiếp. Cơ chế này đóng vai trò quyết định trong việc tối ưu hóa hiệu năng luồng mạng, giảm thiểu thời gian chờ cho người dùng và nâng cao tính ổn định toàn cục của hệ thống phân tán.

Để tổ chức luồng thông điệp một cách khoa học và chính xác, kiến trúc RabbitMQ nội bộ của đề án được cấu hình chặt chẽ thông qua ba thành phần cấu phần kỹ thuật sau:

- **Exchange trung gian (order_exchange):** Đóng vai trò là điểm tiếp nhận thông điệp đầu tiên từ phía dịch vụ xuất bản. Khi người dùng hoàn tất quy trình đặt món ăn, order-service sẽ đóng gói dữ liệu đơn hàng và xuất bản sự kiện này lên order_exchange. Exchange này có nhiệm vụ phân tích nhãn của thông điệp để quyết định hướng phân phối dữ liệu, thay vì đẩy trực tiếp vào một hàng đợi cố định.
- **Hàng đợi thông điệp (notification_queue):** Đóng vai trò là bộ nhớ đệm an toàn lưu trữ dữ liệu vĩnh viễn. Hàng đợi này được liên kết chặt chẽ với order_exchange thông qua một khóa định tuyến đặc trưng (Routing Key) mang tên order.created. Khi có một thông điệp mang khóa định tuyến trùng khớp rơi vào Exchange, RabbitMQ sẽ tự động định hướng và xếp thông điệp đó vào notification_queue theo cơ chế FIFO (First In, First Out).

- **Thực thể tiêu thụ (Consumer):** Tiến trình ngầm nằm bên trong dịch vụ notification-service liên tục duy trì một kết nối mở đến notification_queue. Ngay khi nhận thấy hàng đợi có sự xuất hiện của thông điệp mới, RabbitMQ sẽ chủ động phân phối gói tin JSON này về cho Consumer để giải mã, biên dịch nội dung và đồng bộ dữ liệu thông báo xuống database của khách hàng.

Việc vận hành cơ chế hàng đợi RabbitMQ mang lại cho hệ thống khả năng xử lý phi tập trung vượt trội. Trong các kịch bản thực tế, khi hệ thống phải đối mặt với lượng đơn hàng lớn vượt quá năng lực xử lý tức thời của các dịch vụ hậu mãi, RabbitMQ sẽ giữ vai trò điều tiết lưu lượng (Throttling). Thông điệp sẽ được lưu trữ an toàn trong hàng đợi của Broker và được tiêu thụ một cách tuần tự. Điều này đảm bảo luồng đặt hàng chính của khách hàng luôn diễn ra mượt mà với độ trễ thấp nhất, đồng thời chứng minh trọn vẹn đặc tính độc lập và phối hợp đúng đắn của một hệ thống vi dịch vụ chuẩn mực.

2.3.4. Công nghệ đóng gói và ảo hóa Container

Trong một hệ thống phân tán phát triển theo kiến trúc Vi dịch vụ, một trong những thách thức lớn nhất của giai đoạn vận hành là sự khác biệt về môi trường triển khai (Matrix of Hell). Việc cấu hình thủ công các phiên bản ngôn ngữ Python, cài đặt thư viện phụ thuộc, hay thiết lập hạ tầng MySQL và RabbitMQ trên từng máy tính khác nhau rất dễ dẫn đến xung đột hệ thống và phát sinh các lỗi không đồng nhất. Để giải quyết triệt để bài toán này, đồ án đã áp dụng công nghệ ảo hóa ở cấp độ hệ điều hành thông qua Docker và công cụ điều phối Docker Compose. Giải pháp này giúp đóng gói toàn bộ mã nguồn và môi trường chạy thành các khối Container độc lập, nhất quán và có khả năng vận hành linh hoạt trên mọi hạ tầng phần cứng.

Công nghệ Docker hoạt động dựa trên cơ chế chia sẻ chung nhân hệ điều hành của máy chủ (Host OS), giúp các Container khởi chạy với tốc độ cực nhanh và chiếm dụng rất ít tài nguyên phần cứng so với các máy ảo (Virtual Machines) truyền thống. Trong đồ án, mỗi dịch vụ vi mô (như auth-service, product-service...) đều được định nghĩa một tệp cấu hình Dockerfile riêng. Tệp này chịu trách nhiệm thiết lập môi trường Python 3.11 chuẩn hóa, sao chép mã nguồn, cài đặt chính xác các thư viện phụ thuộc thông qua tệp requirements.txt và thiết lập lệnh khởi chạy cho máy chủ Uvicorn. Quá trình này

giúp cô lập hoàn toàn môi trường thực thi của từng dịch vụ, ngăn chặn tình trạng xung đột thư viện giữa các cấu phần phân tán.

Khi số lượng Container tăng lên (bao gồm 05 dịch vụ nghiệp vụ cùng 02 hạ tầng hỗ trợ là MySQL và RabbitMQ), việc khởi chạy và kết nối thủ công từng Container đơn lẻ sẽ trở nên vô cùng phức tạp. Do đó, đồ án ứng dụng Docker Compose thông qua việc xây dựng một tệp điều phối trung tâm mang tên `docker-compose.yml`. Tệp cấu hình này cho phép định nghĩa toàn bộ kiến trúc hạ tầng của hệ thống dưới dạng mã nguồn (Infrastructure as Code). Tại đây, các thuộc tính kỹ thuật như cổng mạng xuất bản (Port Mapping), biến môi trường cấu hình (`.env`) và đặc biệt là cơ chế phân tách mạng nội bộ (Docker Network) được thiết lập một cách tự động.

Cơ chế mạng ảo độc lập (bridge network) do Docker Compose khởi tạo đóng vai trò là mạch máu giao tiếp an toàn của toàn hệ thống. Tất cả 07 Container đều được gắn kết vào một vùng mạng nội bộ biệt lập, cho phép các dịch vụ backend giao tiếp trực tiếp với nhau và kết nối với MySQL hay RabbitMQ thông qua cơ chế phân giải tên miền nội bộ (Container Name) thay vì sử dụng địa chỉ IP tĩnh. Nhờ có Docker Compose, toàn bộ hệ thống phân tán phức tạp của đồ án có thể được khởi chạy, thiết lập liên kết mạng và đưa vào trạng thái sẵn sàng phục vụ chỉ thông qua một câu lệnh thực thi duy nhất (`docker-compose up -d`). Điều này không chỉ đáp ứng xuất sắc điểm cộng khuyến khích của đề bài, mà còn chứng minh năng lực tự động hóa hạ tầng triển khai theo đúng xu hướng công nghệ hiện đại.

2.3.5. Công nghệ phát triển giao diện người dùng

Trong kiến trúc vi dịch vụ (Microservices), tầng giao diện (Frontend) vận hành độc lập và tách biệt hoàn toàn với tầng xử lý nghiệp vụ Backend. Frontend kết nối phi trạng thái với hệ thống thông qua việc gửi các yêu cầu HTTP API tới cửa ngõ trung tâm API Gateway.

Để xây dựng giao diện người dùng tương tác trực quan cho ứng dụng khách và trang quản trị Dashboard, các công nghệ cốt lõi được áp dụng bao gồm:

- **HTML5 (HyperText Markup Language 5):** Định nghĩa cấu trúc nền tảng và các thành phần hiển thị trên trang web như: biểu mẫu đăng ký/đăng nhập, thẻ danh mục món ăn, giỏ hàng và bảng dữ liệu hóa đơn.

- **CSS3 (Cascading Style Sheets 3):** Đảm nhiệm việc định dạng ngôn ngữ hình ảnh, bố cục và hiệu ứng chuyển động. Sử dụng giải pháp thiết kế đáp ứng (Responsive Web Design) giúp giao diện hiển thị tối ưu trên cả thiết bị di động (Mobile) lẫn máy tính (Desktop).
- **JavaScript (ES6+) & Axios / Fetch API:** Sử dụng mô hình Ứng dụng trang đơn (Single Page Application - SPA) giúp người dùng tương tác mượt mà không cần tải lại toàn bộ trang web. Thư viện *Axios* (hoặc *Fetch API*) chịu trách nhiệm gửi các yêu cầu HTTP ngầm (GET, POST,...) kèm theo mã thông báo JWT bảo mật lên API Gateway.

Lợi ích kỹ thuật mang lại cho hệ thống của nhóm:

- **Tách biệt trách nhiệm:** Đội ngũ phát triển Frontend có thể tập trung tối ưu hóa trải nghiệm người dùng (UX/UI) mà không phụ thuộc vào mã nguồn backend của các dịch vụ như order-service hay auth-service.
- **Tối ưu hóa hiệu năng Backend:** Toàn bộ tài nguyên giao diện được tải và xử lý trực tiếp tại trình duyệt của thiết bị khách, giảm thiểu tối đa áp lực tải và băng thông mạng cho máy chủ backend.

2.3.6. Hạ tầng thu thập Nhật ký tập trung và Giám sát dịch vụ

Để giải quyết bài toán giám sát tính toàn vẹn và theo dõi sức khỏe hệ thống trong môi trường phân tán vi dịch vụ, hạ tầng ghi nhật ký và giám sát được cấu hình tập trung nhằm thay thế phương pháp kiểm tra log thủ công trên từng container riêng lẻ.

Hạ tầng này được xây dựng trên sự phối hợp của các cấu phần công nghệ và thư viện chuyên dụng sau:

- **Thư viện Ghi nhật ký cấu trúc (Python logging / json-logging):** Tích hợp trực tiếp vào mã nguồn của 05 dịch vụ thành phần (auth-service, product-service, order-service, notification-service, api-gateway). Công cụ này tự động chuẩn hóa mọi sự kiện vận hành, thông số HTTP Request, và lỗi ngoại lệ (Exceptions) từ dạng văn bản thô (Plain Text) sang định dạng gói tin cấu trúc JSON để máy chủ thu thập dễ dàng bóc tách thông tin, lập chỉ mục (Indexing) và phân loại theo các trường dữ liệu cố định (như timestamp, level, service_name, trace_id).
- **Hệ thống lưu trữ Nhật ký tập trung (Centralized Logging Base Engine):** Sử dụng cơ chế chia sẻ vùng đệm dữ liệu (Docker Volumes) kết hợp với tiến trình

gom log chạy ngầm. Hệ thống chịu trách nhiệm quét, trích xuất và đồng bộ liên tục các tệp tin .log phát sinh từ tất cả các container vi dịch vụ về một kho lưu trữ tập trung duy nhất trên ổ đĩa cứng của máy chủ giám sát. Nhờ đó, việc truy vết phân tán (Distributed Tracing) trở nên dễ dàng, kết nối chuỗi hành trình của một request đi qua nhiều biên giới mạng dựa trên mã định danh duy nhất trace_id.

- **Bảng điều khiển Giám sát dịch vụ (Service Monitoring Dashboard):** Phân hệ cung cấp giao diện đồ họa trực quan (Web-based UI) cho quản trị viên hệ thống. Dashboard chịu trách nhiệm chuyển hóa các luồng số liệu phần cứng thô thành các biểu đồ động theo thời gian thực, bao gồm: tỷ lệ chiếm dụng CPU (%), dung lượng bộ nhớ RAM (MB), tốc độ đọc/ghi đĩa (Disk I/O) và băng thông mạng (Network Traffic) của từng container độc lập, đảm bảo phát hiện sớm các hiện tượng quá tải phần cứng hoặc nghẽn mạch hệ thống.

Lợi ích kỹ thuật mang lại cho hệ thống:

- **Tập trung hóa quản trị nhật ký:** Gom toàn bộ dữ liệu ghi vết của hệ thống về một mối duy nhất, triệt tiêu hoàn toàn thao tác truy cập thủ công vào từng container để đọc log khi hệ thống phát sinh lỗi liên dịch vụ.
- **Chủ động kiểm soát sức khỏe hạ tầng:** Cung cấp góc nhìn trực quan, toàn cảnh về hiệu năng vận hành thực tế của các container và trạng thái luân chuyển thông điệp trên hàng đợi RabbitMQ, đảm bảo tính sẵn sàng cao cho ứng dụng đặt đồ ăn trực tuyến.

CHƯƠNG 3. THIẾT KẾ CHI TIẾT CÁC MICROSERVICES VÀ CƠ CHẾ GIAO TIẾP

3.1. Thiết kế các dịch vụ thành phần

3.1.1. API Gateway

Trong kiến trúc tổng thể của hệ thống đặt đồ ăn trực tuyến, api-gateway là thành phần kỹ thuật đầu tiên tiếp xúc với mọi lưu lượng truy cập từ môi trường bên ngoài. Được phát triển dựa trên nền tảng framework FastAPI và máy chủ Uvicorn hiệu năng cao, API Gateway vận hành tại cổng 8000 đóng vai trò là một điểm đầu mối duy nhất (Single Entry Point) để quản lý luồng dữ liệu. Chức năng thiết kế chi tiết của API Gateway tập trung vào ba nhiệm vụ cốt lõi: Tiếp nhận/Phân tích yêu cầu (Request Interception), Định tuyến phân luồng động (Dynamic Routing) và Tập trung hóa tài liệu kiểm thử (Centralized API Documentation).

Đối với nhiệm vụ tiếp nhận và phân luồng dữ liệu, API Gateway được cấu hình một ma trận định tuyến nghiêm ngặt dựa trên tiền tố của đường dẫn URL (Path-based Routing). Khi một request từ phía ứng dụng khách (Client) gửi đến cổng 8000, Gateway sẽ ngay lập tức bắt giữ (intercept) gói tin và phân tích chuỗi định vị tài nguyên. Dựa trên các quy tắc chuyển tiếp (Reverse Proxy) được thiết lập sẵn trong mã nguồn, Gateway sẽ thực hiện phân luồng như sau:

- Các request có tiền tố `/api/auth` sẽ được gỡ bỏ đầu bọc và chuyển tiếp nguyên bản đến dịch vụ xác thực auth-service (vận hành tại cổng nội bộ 8001).
- Các request có tiền tố `/api/products` sẽ được chuyển hướng chính xác về dịch vụ quản lý thực đơn product-service (vận hành tại cổng nội bộ 8002).
- Các request có tiền tố `/api/orders` được điều phối đến dịch vụ xử lý hóa đơn order-service (vận hành tại cổng nội bộ 8003).
- Các request có tiền tố `/api/notifications` được phân luồng về dịch vụ thông báo notification-service (vận hành tại cổng nội bộ 8004).

Quá trình chuyển tiếp này được xử lý hoàn toàn bằng cơ chế bất đồng bộ (Asynchronous Proxy), giúp Gateway không bị nghẽn mạch khi hệ thống đối mặt với hàng ngàn request đồng thời trong các khung giờ cao điểm đặt đồ ăn.

Đối với cơ chế xác thực JWT đầu vào, thiết kế của API Gateway tập trung vào việc tối ưu hóa hiệu năng và bảo vệ an toàn cho các dịch vụ tuyến sau. Thay vì bắt buộc Gateway phải giải mã chuyên sâu từng gói tin — một tác vụ có thể gây tăng độ trễ (latency) tại cửa ngõ trung tâm — Gateway đóng vai trò kiểm tra tính hiện diện và định dạng hợp lệ của mã thông báo (Bearer Token Validation) trong tiêu đề Authorization Header đối với các endpoint yêu cầu bảo mật. Sau khi xác định yêu cầu có đính kèm chuỗi JWT, Gateway sẽ chuyển tiếp nguyên vẹn ngữ cảnh xác thực này xuống các dịch vụ backend mục tiêu như product-service hay order-service. Tại các dịch vụ đích, việc giải mã chi tiết và thực thi phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) mới chính thức diễn ra. Giải pháp thiết kế phân tán này giúp phân rã áp lực tính toán từ Gateway xuống các vi dịch vụ, đảm bảo tính chịu lỗi và giữ vững nguyên lý phi trạng thái (Stateless) của kiến trúc hệ thống phân tán.

Cuối cùng, một thành phần thiết kế quan trọng tích hợp bên trong API Gateway là cơ chế hợp nhất tài liệu giao tiếp OpenAPI (Swagger UI). Gateway được cấu hình để tự động thu thập cấu trúc schema, các endpoint, tham số đầu vào và định dạng phản hồi đầu ra từ cả 04 dịch vụ backend chạy ngầm thông qua mạng ảo nội bộ của Docker. Sau đó, toàn bộ dữ liệu tài liệu này được biên dịch và hiển thị tập trung tại duy nhất một giao diện trực quan tại địa chỉ cổng 8000. Thiết kế này cho phép người vận hành, lập trình viên hoặc giảng viên chấm điểm có thể dễ dàng theo dõi toàn bộ đặc tả API của hệ thống đặt đồ ăn, đồng thời thực hiện kiểm thử luồng nghiệp vụ liên dịch vụ một cách đồng bộ mà không cần phải truy cập riêng lẻ vào từng dịch vụ thành phần.

3.1.2. Auth Service

Trong kiến trúc phân tán của hệ thống đặt đồ ăn trực tuyến, dịch vụ auth-service đóng vai trò là nền tảng quản lý định danh (Identity Provider) và là chốt chặn bảo mật cho toàn bộ tài nguyên. Vận hành tại cổng nội bộ 8001 và sở hữu phân vùng dữ liệu biệt lập auth_db, dịch vụ này chịu trách nhiệm quản lý toàn bộ vòng đời của tài khoản người dùng và thực thi cơ chế xác thực an toàn. Hai chức năng kỹ thuật cốt lõi được thiết lập chi tiết bên trong auth-service bao gồm: Lưu trữ thông tin người dùng có mã hóa và Quy trình cấp phát mã thông báo xác thực Access Token (JWT).

Đối với chức năng lưu trữ thông tin người dùng, dịch vụ thực hiện tương tác trực tiếp với bảng users bên trong cơ sở dữ liệu auth_db. Để tuân thủ các tiêu chuẩn về an

toàn thông tin, mật khẩu của người dùng do Client gửi lên dưới dạng văn bản thuần (Plain Text) qua API đăng ký (POST /api/auth/register) tuyệt đối không được ghi nhận trực tiếp vào database. Thay vào đó, auth-service sử dụng thuật toán băm một chiều (hashing) có độ an toàn cao để biến đổi chuỗi mật khẩu thành một chuỗi mã hóa cố định (password_hash) trước khi lưu trữ. Các thông tin đi kèm cấu trúc tài khoản bao gồm mã người dùng (id), thư điện tử (email), họ tên (full_name), số điện thoại (phone) và đặc biệt là vai trò hệ thống (role). Trường dữ liệu role này (nhận giá trị customer hoặc admin) chính là cơ sở dữ liệu gốc để các dịch vụ tuyến sau thực thi cơ chế phân quyền kiểm soát truy cập (RBAC).

Đối với quy trình xác thực và cấp phát Access Token, auth-service triển khai theo chuẩn giao thức mã hóa phi trạng thái (Stateless Authentication) thông qua mã thông báo JSON Web Token (JWT). Tiến trình này được kích hoạt khi Client gửi yêu cầu đăng nhập thông qua API Gateway đến endpoint POST /api/auth/login. Nhằm tương thích với các tiêu chuẩn bảo mật hiện đại, endpoint này được thiết kế để tiếp nhận dữ liệu đầu vào dưới dạng biểu mẫu OAuth2 (OAuth2 Form Data) bao gồm hai trường thông tin bắt buộc là username (được ánh xạ từ email người dùng) và password. Quy trình xử lý nội bộ của dịch vụ diễn ra qua ba bước nghiêm ngặt:

1. **Xác thực danh tính:** Dịch vụ truy vấn vào bảng users dựa trên thông tin email để tìm kiếm tài khoản tồn tại. Nếu tìm thấy tài khoản, hệ thống sẽ tiến hành đối chiếu chuỗi mật khẩu đầu vào với chuỗi password_hash được lưu trong database bằng thư viện mã hóa chuyên dụng.
2. **Khởi tạo thông tin mã hóa (Payload):** Khi thông tin đăng nhập hoàn toàn trùng khớp, dịch vụ sẽ đóng gói các trường dữ liệu định danh cốt lõi bao gồm mã người dùng (user_id), email và vai trò (role) thành một phân đoạn dữ liệu dạng JSON gọi là nội dung mã thông báo (Payload). Thời gian hết hạn của phiên làm việc (Expiration Time) cũng được tích hợp vào Payload để đảm bảo mã thông báo tự động vô hiệu hóa sau một khoảng thời gian quy định nhằm giảm thiểu rủi ro bảo mật.
3. **Ký số và xuất bản JWT:** Phân đoạn Payload sau đó được ký số (Sign) bằng một khóa bí mật (JWT_SECRET) theo thuật toán băm đối xứng an toàn HS256 đã được cấu hình thống nhất trên toàn hệ thống. Chuỗi ký số này kết hợp với phần

đầu (Header) tạo thành một chuỗi mã thông báo JWT hoàn chỉnh để trả về cho Client.

Thông qua việc đóng gói toàn bộ quy trình định danh và cấp phát token vào một dịch vụ phi trạng thái, auth-service cho phép Client lưu trữ token cục bộ và sử dụng nó để tự động chứng minh quyền truy cập khi giao tiếp với các dịch vụ nghiệp vụ khác. Thiết kế này loại bỏ hoàn toàn việc các dịch vụ khác phải liên tục truy vấn ngược lại database auth_db, giúp tối ưu hóa hiệu năng luồng mạng nội bộ và đảm bảo tính chịu lỗi cao của hệ thống vi dịch vụ.

3.1.3. Product Service

Trong hệ thống vi dịch vụ đặt đồ ăn trực tuyến, product-service giữ vai trò là phân hệ quản lý thông tin gốc của toàn bộ thực đơn món ăn. Vận hành tại cổng nội bộ 8002 và kết nối trực tiếp với phân vùng cơ sở dữ liệu product_db, dịch vụ này cung cấp các giao diện lập trình ứng dụng (API Endpoints) cho phép thực hiện trọn vẹn các thao tác CRUD dữ liệu sản phẩm. Điểm cốt lõi trong thiết kế kỹ thuật của product-service là sự kết hợp chặt chẽ giữa hiệu năng truy xuất dữ liệu bất đồng bộ và cơ chế phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) để bảo vệ an toàn cho tài nguyên hệ thống.

Đối với các tác vụ không làm thay đổi trạng thái hệ thống, cụ thể là API lấy danh sách toàn bộ món ăn (GET /api/products) và lấy chi tiết món ăn theo định danh (GET /api/products/{product_id}), dịch vụ được thiết kế theo cơ chế mở (Public API). Khách hàng khi truy cập ứng dụng để duyệt thực đơn hoàn toàn không cần phải đính kèm mã thông báo xác thực. Khi tiếp nhận yêu cầu, dịch vụ sử dụng cơ chế Asynchronous của FastAPI để thực hiện truy vấn ngầm vào bảng products bên trong product_db, trích xuất các trường thông tin bao gồm mã sản phẩm (id), tên món (name), mô tả (description), đơn giá (price), danh mục (category), đường dẫn hình ảnh (image_url) và trạng thái kinh doanh (is_available). Việc xử lý bất đồng bộ tại tầng này giúp dịch vụ giải phóng luồng mạng cực nhanh, chịu được áp lực truy cập đồng thời lớn từ hàng ngàn khách hàng duyệt món cùng lúc mà không gây nghẽn hệ thống.

Ngược lại, đối với các tác vụ thay đổi cấu trúc dữ liệu thực đơn — bao gồm thêm món ăn mới (POST /api/products), cập nhật thông tin/giá cả (PUT /api/products/{product_id}) và xóa sản phẩm (DELETE /api/products/{product_id}) —

product-service thiết lập một quy trình kiểm soát phân quyền vô cùng nghiêm ngặt. Quy trình xác thực và phân quyền Admin tại dịch vụ được thực thi chi tiết qua các bước kỹ thuật sau:

1. **Trích xuất ngữ cảnh bảo mật:** Khi Admin gửi yêu cầu thay đổi dữ liệu qua API Gateway, mã thông báo JWT bắt buộc phải được đính kèm trong tiêu đề HTTP. Sau khi Gateway kiểm tra sự hiện diện của token, product-service sẽ tiếp nhận và tiến hành giải mã (Decode) chuỗi JWT này bằng khóa bí mật chung (JWT_SECRET).
2. **Kiểm tra phân quyền dựa trên vai trò (RBAC):** Sau khi giải mã thành công, dịch vụ sẽ trích xuất trường dữ liệu role từ Payload của mã thông báo. Dịch vụ thực hiện đối chiếu điều kiện logic; nếu giá trị của trường role không trùng khớp với chuỗi mã admin, hệ thống sẽ lập tức từ chối thực thi tác vụ, hủy bỏ tiến trình và phản hồi về lỗi phân quyền (HTTP 403 Forbidden) cho Client.
3. **Kiểm định cấu trúc dữ liệu đầu vào (Data Validation):** Khi quyền admin được xác chuẩn, dữ liệu của món ăn (Payload JSON gửi lên từ Admin) sẽ được đưa qua tầng lọc kiểm định tự động bằng các mô hình Pydantic. Các ràng buộc kỹ thuật như đơn giá phải là số dương lớn hơn không, tên món không được để trống sẽ được framework tự động kiểm tra. Nếu phát hiện dữ liệu sai lệch, hệ thống trả về mã lỗi HTTP 422 Unprocessable Entity.
4. **Đồng bộ hóa dữ liệu xuống database:** Dữ liệu sau khi vượt qua tất cả các chốt chặn an toàn mới chính thức được chuyển đổi thành câu lệnh SQL để ghi nhận, cập nhật hoặc xóa bỏ tương ứng trong bảng products của product_db.

Việc cô lập toàn bộ logic quản lý thực đơn và phân quyền an toàn bên trong product-service giúp hệ thống đạt được tính đóng gói tối đa. Các dịch vụ khác (như order-service khi cần lấy đơn giá món ăn) chỉ có thể tương tác với dữ liệu sản phẩm thông qua các endpoint do product-service cung cấp, tuyệt đối không được phép can thiệp trực tiếp vào database product_db, bảo đảm tính độc lập vật lý và an toàn cấu trúc của toàn bộ hệ thống vi dịch vụ.

3.1.4. Order Service

Dịch vụ order-service vận hành tại cổng nội bộ 8003 là hạt nhân điều phối toàn bộ luồng nghiệp vụ mua sắm và khởi tạo giao dịch bên trong hệ thống đặt đồ ăn trực tuyến.

Sở hữu phân vùng dữ liệu biệt lập `order_db`, dịch vụ này có trách nhiệm tiếp nhận yêu cầu đặt hàng, thực hiện liên kết liên dịch vụ để xác thực thông tin tài chính và tính toán giá trị hóa đơn một cách chính xác. Do tính chất phi trạng thái và dữ liệu phân rã của kiến trúc vi dịch vụ, quy trình thiết kế kỹ thuật của `order-service` đòi hỏi một cơ chế kiểm tra dữ liệu nghiêm ngặt kết hợp với giao tiếp đồng bộ (Synchronous Communication) qua giao thức RESTful API.

Quy trình xử lý chi tiết tại `order-service` được kích hoạt khi người dùng gửi một yêu cầu HTTP POST đến endpoint `/api/orders` thông qua cửa ngõ API Gateway. Yêu cầu này bắt buộc phải đính kèm mã thông báo JWT hợp lệ tại tiêu đề HTTP và một gói tin JSON chứa cấu trúc giỏ hàng cơ bản. Cấu trúc dữ liệu đầu vào từ Client chỉ bao gồm mã người dùng (`user_id`) và một danh sách các mục chọn (`order_items`), trong đó mỗi mục được định nghĩa tối giản bằng hai trường thông tin: mã sản phẩm (`product_id`) và số lượng đặt mua (`quantity`). Việc tinh giản dữ liệu đầu vào này giúp giảm thiểu tối đa băng thông truyền tải qua mạng, đồng thời ngăn chặn rủi ro Client gian lận bằng cách tự gửi kèm đơn giá món ăn.

Sau khi tiếp nhận yêu cầu và trích xuất thành công `user_id` từ mã thông báo JWT để định danh chủ thể, `order-service` tiến hành giai đoạn liên kết liên dịch vụ đồng bộ nhằm thu thập thông tin giá cả thực tế. Tiến trình giao tiếp nội bộ này được hiện thực hóa qua các bước kỹ thuật sau:

1. **Khởi tạo yêu cầu RESTful nội bộ:** `order-service` sử dụng một thư viện gửi truy cập HTTP bất đồng bộ để thiết lập một kết nối trực tiếp đến endpoint `GET /api/products/{product_id}` của `product-service` đang chạy ngầm tại cổng nội bộ 8002.
2. **Xác thực trạng thái sản phẩm tuyến sau:** Đối với từng mã `product_id` có trong giỏ hàng, `order-service` sẽ đợi phản hồi từ dịch vụ sản phẩm. Nếu `product-service` trả về mã lỗi (HTTP 404 Not Found) hoặc thông báo trạng thái món ăn hiện tại không sẵn sàng phục vụ (`is_available = false`), `order-service` sẽ lập tức hủy bỏ toàn bộ tiến trình và phản hồi lỗi nghiệp vụ về cho người dùng, đảm bảo tính toàn vẹn dữ liệu.

3. **Thu thập đơn giá gốc:** Khi sản phẩm được xác chuẩn là hợp lệ, product-service sẽ phản hồi gói tin JSON chứa giá trị đơn giá (price) chính xác nhất tại thời điểm đó của món ăn về cho dịch vụ đơn hàng.

Ngay khi nhận đầy đủ dữ liệu đơn giá từ product-service, order-service chuyển sang tầng tính toán tài chính nội bộ. Hệ thống thực hiện một chu kỳ vòng lặp tuần tự qua danh sách giỏ hàng để nhân số lượng (quantity) với đơn giá (price) vừa thu thập được, trích xuất ra tổng tiền của từng mục sản phẩm. Sau đó, các giá trị này được cộng dồn vào một biến tổng tài chính duy nhất để xác định tổng giá trị cuối cùng của toàn bộ hóa đơn (total_amount). Trước khi thực hiện ghi dữ liệu, toàn bộ cấu trúc hóa đơn được đưa qua tầng lọc kiểm định Pydantic nhằm bảo đảm không có lỗi logic toán học (như tổng số tiền hoặc số lượng là số âm).

Cuối cùng, khi hóa đơn đã được tính toán chính xác, order-service thực thi các câu lệnh tương tác dữ liệu để lưu trữ vĩnh viễn thông tin đơn hàng xuống order_db. Tiến trình lưu trữ được thực hiện đồng thời trên hai bảng có mối quan hệ ràng buộc chặt chẽ: thông tin tổng quan (bao gồm mã đơn hàng id, mã người dùng user_id, và tổng giá trị đơn hàng total_amount) được ghi nhận vào bảng orders, trong khi chi tiết cụ thể của từng mặt hàng (bao gồm mã sản phẩm product_id và số lượng quantity) được phân rã và lưu vào bảng order_items. Việc hoàn tất ghi nhận xuống cơ sở dữ liệu vật lý vật lý này là minh chứng cho một chu kỳ giao tiếp đồng bộ thành công, tạo cơ sở để dịch vụ chuẩn bị kích hoạt luồng phát sự kiện bất đồng bộ sang hệ thống hàng đợi RabbitMQ.

3.1.5. Notification Service

Trong khi các dịch vụ như auth-service, product-service hay order-service vận hành theo mô hình chủ động tiếp nhận và phản hồi các truy vấn trực tiếp từ API Gateway, dịch vụ notification-service được thiết kế theo một kiến trúc hoàn toàn khác biệt. Vận hành tại cổng nội bộ 8004 và sở hữu phân vùng cơ sở dữ liệu tách biệt notification_db, dịch vụ này đóng vai trò là một thực thể tiêu thụ thông điệp (Consumer) hoạt động ngầm liên tục bên trong hệ thống đặt đồ ăn trực tuyến. Nhiệm vụ thiết kế chi tiết của notification-service tập trung vào việc lắng nghe các sự kiện nghiệp vụ từ Message Broker RabbitMQ, tự động biên dịch nội dung và đồng bộ hóa trạng thái thông báo để phục vụ khách hàng.

Kiến trúc bên trong của notification-service được thiết lập dựa trên cơ chế vòng lặp bất đồng bộ của framework FastAPI, cho phép duy trì một tiến trình ngầm (Background Task) kết nối liên tục đến Broker RabbitMQ qua giao thức AMQP. Quy trình tiếp nhận và xử lý sự kiện tại dịch vụ này được tổ chức chặt chẽ qua bốn giai đoạn kỹ thuật sau:

1. **Đăng ký lắng nghe hàng đợi (Queue Binding):** Ngay khi container dịch vụ được khởi chạy, notification-service sẽ thiết lập kết nối và đăng ký một Consumer ngầm tại hàng đợi notification_queue. Hàng đợi này đã được ràng buộc chặt chẽ với order_exchange thông qua khóa định tuyến order.created, đảm bảo dịch vụ chỉ tiếp nhận đúng các thông điệp liên quan đến sự kiện khởi tạo đơn hàng thành công từ hệ thống.
2. **Tiếp nhận và giải mã thông điệp (Message Consuming):** Ngay khi có sự kiện order.created xuất hiện trong hàng đợi, RabbitMQ sẽ chủ động phân phối gói tin dữ liệu dạng JSON về cho dịch vụ. notification-service lập tức bắt giữ gói tin, thực hiện giải mã (Deserialize) để trích xuất các trường dữ liệu cấu phần cốt lõi bao gồm mã định danh đơn hàng (order_id) và mã khách hàng (user_id).
3. **Biên dịch nội dung tự động:** Sau khi thu thập được các tham số đầu vào từ thông điệp, dịch vụ thực thi logic nghiệp vụ nội bộ để tự động khởi tạo cấu trúc dữ liệu thông báo. Hệ thống sẽ biên dịch một chuỗi văn bản trực quan dựa trên mã đơn hàng nhận được, định dạng theo cấu trúc: *"Đơn hàng số #{order_id} của bạn đã được đặt thành công"*. Chuỗi nội dung này được thiết lập đi kèm với trường trạng thái xem ban đầu mặc định là chưa đọc (is_read = false).
4. **Đồng bộ hóa dữ liệu xuống database:** Dữ liệu thông báo hoàn chỉnh sau đó được đưa qua mô hình kiểm định Pydantic để đảm bảo tính toàn vẹn trước khi dịch vụ thực thi câu lệnh SQL để ghi nhận trực tiếp vào bảng notifications thuộc cơ sở dữ liệu notification_db.

Việc thiết kế notification-service hoạt động hoàn toàn biệt lập dưới vai trò một Consumer mang lại ý nghĩa rất lớn cho tính chịu lỗi và hiệu năng toàn cục của hệ thống phân tán. Do dịch vụ không tham gia vào luồng xử lý HTTP đồng bộ của quá trình đặt hàng, thời gian phản hồi (Latency) trả về cho khách hàng tại API Gateway được tối ưu hóa ở mức tuyệt đối.

Bên cạnh đó, dịch vụ vẫn cung cấp một endpoint đồng bộ độc lập là GET /api/notifications qua API Gateway. Endpoint này cho phép Client đính kèm mã thông báo JWT để truy vấn trực tiếp vào bảng notifications, lấy ra danh sách các thông báo cá nhân đã được hệ thống xử lý và lưu trữ ngầm trước đó. Thiết kế này chứng minh sự phối hợp đúng đắn, nhuần nhuyễn giữa hai mô hình giao tiếp đồng bộ và bất đồng bộ bên trong hệ thống vi dịch vụ của đề án.

3.1.6. *Payment Service*

Dịch vụ thanh toán vận hành độc lập tại cổng nội bộ 8005, đóng vai trò là "Thủ quỹ" của hệ thống để quản lý dòng tiền và thực thi các logic nghiệp vụ tài chính. Thiết kế chi tiết của dịch vụ tập trung vào 03 chức năng và luồng hoạt động cốt lõi sau:

- **Tạo mới giao dịch thanh toán (HTTP POST /api/payments):** Tiếp nhận yêu cầu thanh toán hóa đơn từ Client. Hệ thống thực hiện kiểm định phương thức đầu vào (chấp nhận: cash, card, bank_transfer, momo, vnpay). Nhằm triệt tiêu rủi ro gian lận sửa giá từ phía Frontend, dịch vụ thực hiện luồng **giao tiếp đồng bộ REST API ngầm** sang order-service để xác thực đơn hàng tồn tại, đối chiếu quyền sở hữu và tự động lấy giá trị tổng tiền gốc (total_amount). Sau khi kiểm tra chống trùng lặp, giao dịch được lưu xuống payment_db với trạng thái mặc định là pending kèm mã định danh duy nhất (Ví dụ: PAY-A1B2C3D4), đồng thời xuất bản sự kiện payment.created lên RabbitMQ.
- **Cập nhật trạng thái giao dịch (HTTP PATCH /api/payments/{id}/status):** Tiếp nhận dữ liệu phản hồi từ phía cổng thanh toán đối tác hoặc quản trị viên để cập nhật trạng thái (paid, failed, refunded, cancelled). Khi giao dịch được xác nhận thành công sang trạng thái paid, dịch vụ tự động ghi nhận mốc thời gian thực tế (paid_at), đồng thời **phát một sự kiện bất đồng bộ payment.paid lên RabbitMQ Broker**. Sự kiện này kích hoạt luồng xử lý liên dịch vụ tuyến sau: order-service tự động chuyển trạng thái đơn hàng sang "Đang giao" và notification-service tiêu thụ tin nhắn để gửi thông báo xác nhận tới khách hàng.
- **Tra cứu lịch sử thanh toán (HTTP GET):** Cung cấp các API công khai cho phép người dùng truy vấn danh sách toàn bộ các giao dịch cá nhân, hoặc hỗ trợ bộ lọc trích xuất lịch sử giao dịch chính xác theo mã định danh hóa đơn thông qua Endpoint /api/payments/order/{order_id}.

3.2. Thiết kế Cơ sở dữ liệu độc lập

3.2.1. Cơ sở dữ liệu *auth_db*

Để hiện thực hóa nguyên lý cô lập dữ liệu tối cao trong kiến trúc Vi dịch vụ (Database-per-service), phân hệ xác thực được thiết lập quyền làm chủ hoàn toàn đối với cơ sở dữ liệu độc lập mang tên *auth_db*. Cơ sở dữ liệu này được lưu trữ vật lý trên container MySQL và tách biệt hoàn toàn với các phân vùng dữ liệu của các dịch vụ nghiệp vụ khác. Bên trong *auth_db*, thực thể dữ liệu cốt lõi duy nhất được thiết kế là bảng *users* (Quản lý thông tin người dùng). Bảng này chịu trách nhiệm lưu trữ cấu trúc tài khoản, thông tin định danh cá nhân và vai trò hệ thống phục vụ trực tiếp cho quy trình cấp phát mã thông báo JWT và phân quyền dựa trên vai trò (RBAC).

Dưới đây là bảng đặc tả chi tiết cấu trúc kỹ thuật của bảng *users* được cài đặt trong hệ thống:

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
id	INT	Primary Key, Auto Increment	Mã định danh bản ghi tự sinh.
full_name	VARCHAR(255)	Not Null	Họ và tên đầy đủ của người dùng.
email	VARCHAR(255)	Not Null, Unique, Index	Địa chỉ email dùng để đăng nhập.
password_hash	VARCHAR(255)	Not Null	Mật khẩu đã được băm (mã hóa) bảo mật.
phone	VARCHAR(30)	Nullable	Số điện thoại liên lạc của người dùng.
role	VARCHAR(50)	Not Null, Default="user"	Phân quyền người dùng (user, admin).
is_active	BOOLEAN	Not Null, Default=True	Trạng thái hoạt động của tài khoản (Bị khóa hay không).

created_at	DATETIME	Not Null, Default=NOW()	Thời gian tạo tài khoản.
------------	----------	----------------------------	--------------------------

Bảng 3.1: Bảng auth_db

Về mặt giải pháp kỹ thuật và tư duy thiết kế, cấu trúc bảng users thể hiện tính toán logic qua hai điểm nhấn quan trọng:

Thứ nhất là giải pháp bảo mật trường password_hash. Hệ thống tuyệt đối từ chối việc lưu trữ mật khẩu dưới dạng văn bản thô (Plain Text). Khi tiến trình đăng ký tài khoản được thực thi tại auth-service, chuỗi ký tự mật khẩu của người dùng sẽ đi qua một module băm mật khẩu chuyên dụng (ứng dụng thuật toán băm tiêu chuẩn) để tạo ra một chuỗi băm có độ dài cố định trước khi thực hiện câu lệnh INSERT vào database. Ràng buộc VARCHAR(255) đảm bảo không gian lưu trữ dư dả cho các chuỗi băm phức tạp, ngăn chặn nguy cơ rò rỉ dữ liệu ngay cả khi cơ sở dữ liệu bị tiếp cận trái phép.

Thứ hai là vai trò chiến lược của trường role kết hợp với chỉ mục dữ liệu. Trường role được thiết lập giá trị mặc định là 'customer' để tối ưu hóa luồng đăng ký tự động của người dùng phổ thông qua ứng dụng khách. Đối với tài khoản quản trị hệ thống, giá trị này sẽ được chỉ định thủ công hoặc qua script khởi tạo là 'admin'. Trường email được thiết lập thuộc tính UNIQUE và tự động tạo chỉ mục (Index) trong MySQL, giúp đẩy nhanh tốc độ truy vấn SELECT khi người dùng đăng nhập, tối ưu hóa thời gian giải phóng luồng mạng cho dịch vụ xác thực.

Từ cấu trúc lưu trữ này, khi auth-service thực hiện xác thực thành công, các giá trị từ các trường id (được ánh xạ thành user_id), email và role sẽ trực tiếp được trích xuất để đóng gói thành Payload của mã thông báo JWT, làm cơ sở phân quyền cho toàn bộ hệ thống phân tán phía sau.

3.2.2. Cơ sở dữ liệu product_db

Tuân thủ chặt chẽ mô hình quản trị dữ liệu phân tán "Database-per-service", dịch vụ quản lý thực đơn sở hữu độc quyền một phân vùng cơ sở dữ liệu vật lý riêng biệt mang tên product_db. Việc cô lập này đảm bảo các vi dịch vụ khác không thể can thiệp hoặc truy vấn trực tiếp cấu trúc dữ liệu của sản phẩm mà bắt buộc phải đi qua các giao tiếp API định chuẩn. Bên trong cơ sở dữ liệu product_db, thực thể dữ liệu đóng vai trò mạch máu thông tin là bảng products. Bảng này chịu trách nhiệm lưu trữ toàn bộ thông

tin chi tiết về danh mục, đơn giá và trạng thái khả dụng của các món ăn, phục vụ cho luồng duyệt thực đơn của khách hàng và luồng cấu hình của quản trị viên.

Dưới đây là bảng đặc tả chi tiết cấu trúc kỹ thuật (Schema) của bảng products được cài đặt trong hệ thống:

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
id	INT	Primary Key, Auto Increment	Mã định danh sản phẩm tự sinh.
name	VARCHAR(255)	Not Null, Index	Tên sản phẩm hiển thị trên hệ thống.
description	TEXT	Nullable	Mô tả chi tiết thông tin sản phẩm.
price	DECIMAL(10,2)	Not Null	Giá bán của sản phẩm.
category	VARCHAR(100)	Nullable, Index	Phân loại danh mục sản phẩm.
image_url	VARCHAR(500)	Nullable	Đường dẫn liên kết đến ảnh đại diện sản phẩm.
is_available	BOOLEAN	Not Null, Default=True	Trạng thái còn hàng hay hết hàng.
created_at	DATETIME	Not Null, Default=NOW()	Thời gian đăng sản phẩm lên hệ thống.

Bảng 3.2: Bảng product_db

Về mặt tư duy thiết kế và tối ưu hóa hệ thống, cấu trúc bảng products được thiết lập dựa trên các luận điểm logic kỹ thuật sau:

Thứ nhất là việc lựa chọn kiểu dữ liệu DECIMAL(10,2) cho trường price. Trong các hệ thống liên quan đến tài chính, việc sử dụng các kiểu dữ liệu số thực dấu phẩy động (như FLOAT hoặc DOUBLE) được coi là lỗi thiết kế nghiêm trọng do rủi ro sai số làm tròn trong quá trình tính toán cơ lý thuyết. Kiểu dữ liệu DECIMAL đảm bảo giá trị tiền tệ được lưu trữ chính xác tuyệt đối đến từng chữ số thập phân, tạo nền tảng vững

chắc để order-service thực hiện các phép tính cộng dồn tổng giá trị hóa đơn ở các bước sau mà không sợ sai lệch dòng tiền.

Thứ hai là vai trò kiểm soát vận hành của trường trạng thái `is_available`. Trong bài toán đặt đồ ăn thực tế, các cửa hàng thường xuyên đối mặt với tình trạng hết nguyên liệu đột xuất. Thay vì thực thi câu lệnh cứng DELETE để gỡ bỏ hoàn toàn bản ghi khỏi cơ sở dữ liệu — một hành động có thể phá vỡ các ràng buộc toàn vẹn dữ liệu lịch sử ở bảng hóa đơn — hệ thống sử dụng cơ chế xóa mềm hoặc cập nhật trạng thái thông qua trường `is_available`. Khi Admin cập nhật trường này về giá trị FALSE, món ăn sẽ ngay lập tức được ẩn hoặc khóa chức năng thêm vào giỏ hàng tại giao diện Client.

Cuối cùng, trường định danh `id` (Primary Key) được cấu hình thuộc tính AUTO_INCREMENT nhằm tối ưu hóa việc đánh chỉ mục (Clustered Index) tự động của MySQL. Chuỗi khóa chính dạng số nguyên này đồng thời đóng vai trò là "mã sản phẩm" (`product_id`) — một khóa giao tiếp chiến lược để dịch vụ đơn hàng (order-service) thực hiện ánh xạ đồng bộ và trích xuất thông tin giá cả khi tính toán hóa đơn cho khách hàng.

3.2.3. Cơ sở dữ liệu `order_db`

Nếu như dữ liệu tài khoản và thực đơn món ăn có tần suất thay đổi thấp, thì dữ liệu đơn hàng lại là phân vùng biến động liên tục và có tốc độ tăng trưởng kích thước lớn nhất hệ thống.

Dưới đây là bảng đặc tả chi tiết cấu trúc kỹ thuật (Schema) của bảng dữ liệu được cài đặt trong hệ thống:

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
<code>id</code>	INT	Primary Key, Auto Increment	Mã định danh đơn hàng tự sinh.
<code>user_id</code>	INT	Not Null, Index	Mã người dùng đặt hàng (Khóa logic kết nối <code>auth_db</code>).
<code>order_items</code>	JSON	Not Null	Danh sách chi tiết các sản phẩm trong đơn đặt hàng.

total_amount	DECIMAL(10,2)	Not Null (\$>0\$)	Tổng giá trị phải thanh toán của đơn hàng.
status	VARCHAR(50)	Not Null, Default="pending"	Trạng thái đơn hàng (pending, paid, cancelled...).
note	VARCHAR(500)	Nullable	Ghi chú của khách hàng khi đặt đơn.
created_at	DATETIME	Not Null, Default=NOW()	Thời gian hệ thống ghi nhận đơn hàng.

Bảng 3.3: Bảng order_db

Cấu trúc thiết kế liên kết của phân vùng dữ liệu order_db thể hiện tư duy tối ưu hóa hệ thống phân tán qua các luận điểm kỹ thuật sau:

Thứ nhất là việc duy trì mối quan hệ toàn vẹn dữ liệu nội bộ qua khóa ngoại FOREIGN KEY (order_id) REFERENCES orders(id). Khi order-service tiếp nhận một giỏ hàng phức tạp có nhiều món ăn, hệ thống bắt buộc phải thực thi một giao dịch dữ liệu mang tính đồng thời. Một bản ghi tổng quan sẽ được ghi nhận vào bảng orders trước để lấy ra mã đơn hàng tự động tăng (id), sau đó mã này sẽ được ánh xạ làm giá trị cho trường order_id của nhiều dòng chi tiết tương ứng được INSERT vào bảng order_items. Ràng buộc khóa ngoại này đảm bảo không bao giờ xảy ra hiện tượng "mò côi" dữ liệu – tức là có chi tiết món ăn được lưu mà không thuộc về bất kỳ hóa đơn hợp lệ nào.

Thứ hai là bản chất lưu trữ phi liên kết (De-referenced) đối với hai trường user_id và product_id. Khác hoàn toàn với kiến trúc nguyên khối truyền thống – nơi mà user_id sẽ trở khóa ngoại trực tiếp sang bảng người dùng và product_id trở sang bảng sản phẩm – trong kiến trúc Microservices, các bảng đích này nằm ở hai cơ sở dữ liệu vật lý hoàn toàn khác biệt (auth_db và product_db). Do MySQL không hỗ trợ ràng buộc khóa ngoại xuyên suốt các container database độc lập, trường user_id và product_id tại order_db chỉ được lưu trữ dưới dạng các trường số nguyên thuần túy (Logical Keys). Tính đúng đắn và toàn vẹn của các mã định danh này sẽ được kiểm soát hoàn toàn ở tầng mã nguồn ứng dụng thông qua các luồng RESTful API đồng bộ trước khi dữ liệu được phép ghi nhận xuống đĩa cứng.

Cuối cùng, trường tài chính `total_amount` tiếp tục kế thừa định dạng số chính xác cao `DECIMAL(10,2)` tương thích với trường giá của dịch vụ sản phẩm. Việc phân rã giỏ hàng thành một bảng tổng quan và một bảng chi tiết giúp hệ thống vừa lưu vết lịch sử giao dịch một cách tường minh, vừa tạo điều kiện thuận lợi để trích xuất các thông số nghiệp vụ dạng JSON ngắn gọn phục vụ cho quá trình đóng gói thông điệp gửi sang hệ thống hàng đợi RabbitMQ ở các bước xử lý hậu mãi.

3.2.4. Cơ sở dữ liệu `notification_db`

Thành phần cuối cùng trong cấu trúc lưu trữ phân tán của hệ thống là phân vùng cơ sở dữ liệu `notification_db`. Phân vùng này thuộc quyền sở hữu và quản trị độc quyền của dịch vụ `notification-service`. Do tính chất của dịch vụ thông báo hoạt động hoàn toàn theo mô hình hướng sự kiện và giao tiếp bất đồng bộ, cấu trúc lưu trữ tại đây được tối ưu hóa theo hướng đơn giản, tinh gọn nhằm tăng tốc độ ghi dữ liệu từ hàng đợi trung gian RabbitMQ. Thực thể dữ liệu duy nhất được cài đặt bên trong `notification_db` là bảng `notifications` (Quản lý thông tin thông báo của khách hàng).

Dưới đây là bảng đặc tả chi tiết cấu trúc kỹ thuật (Schema) của bảng `notifications` được cài đặt trong hệ thống:

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
<code>id</code>	INT	Primary Key, Auto Increment	Mã định danh thông báo tự sinh.
<code>user_id</code>	INT	Not Null, Index	Mã người dùng sẽ nhận thông báo.
<code>type</code>	VARCHAR(100)	Not Null	Loại sự kiện thông báo (<code>order_created</code> , <code>payment_paid</code>).
<code>content</code>	TEXT	Not Null	Nội dung chi tiết của thông báo gửi cho khách.
<code>status</code>	VARCHAR(50)	Not Null, Default="unread"	Trạng thái thông báo (<code>unread</code> , <code>read</code>).
<code>created_at</code>	DATETIME	Not Null, Default=NOW()	Thời gian phát đi thông báo.

Bảng 3.4: Bảng *notification_db*

Về mặt tư duy thiết kế và tối ưu hóa hệ thống phân tán, cấu trúc bảng notifications thể hiện tính toán logic qua các luận điểm kỹ thuật sau:

Thứ nhất là bản chất phi trạng thái và tính độc lập vật lý của trường `user_id`. Tương tự như thiết kế của dịch vụ đơn hàng, trường `user_id` tại đây đóng vai trò là một khóa logic (Logical Key), lưu trữ mã số định danh của khách hàng để phục vụ cho các câu lệnh truy vấn lọc dữ liệu cá nhân. Vì bảng này nằm biệt lập trong `notification_db`, hệ thống hoàn toàn không thiết lập liên kết khóa ngoại (Foreign Key) sang bảng người dùng của `auth_db`. Khi người dùng gọi API qua Gateway để kiểm tra thông báo, `notification-service` chỉ cần thực thi câu lệnh SELECT đi kèm mệnh đề WHERE `user_id = :current_user_id` trực tiếp trên database của mình, đảm bảo tốc độ phản hồi cực nhanh mà không gây ảnh hưởng đến hiệu năng của dịch vụ xác thực.

Thứ hai là việc tối ưu hóa kiểu dữ liệu cho trường nội dung message. Chuỗi thông báo trong hệ thống đặt đồ ăn thường có cấu trúc ngắn gọn, được biên dịch tự động theo khuôn mẫu định sẵn từ thông điệp JSON nhận được tại hàng đợi (ví dụ: *"Đơn hàng số #123 của bạn đã được đặt thành công"*). Do đó, việc lựa chọn kiểu dữ liệu VARCHAR(255) là hoàn toàn dư dả về mặt không gian lưu trữ, đồng thời giúp tiết kiệm tài nguyên đĩa cứng và bộ nhớ đệm (Buffer Pool) của MySQL hơn so với việc sử dụng kiểu dữ liệu TEXT có kích thước không cố định.

Cuối cùng, trường trạng thái `is_read` kiểu BOOLEAN được thiết lập giá trị mặc định là FALSE (Chưa đọc) ngay khi bản ghi được khởi tạo từ tiến trình tiêu thụ thông điệp ngầm (Consumer Task). Trường dữ liệu này giữ vai trò quan trọng trong việc hỗ trợ các tính năng tương tác phía Client, cho phép ứng dụng khách dễ dàng lọc và hiển thị số lượng thông báo mới, từ đó nâng cao trải nghiệm hậu mãi của người dùng đối với hệ thống mà vẫn giữ vẹn nguyên tính độc lập của kiến trúc vi dịch vụ.

3.2.5. Cơ sở dữ liệu *payment_db*

Cơ sở dữ liệu `payment_db` được lưu trữ tách biệt và quản trị độc quyền bởi Payment Service nhằm phục vụ luồng ghi nhận trạng thái giao dịch tài chính. Hệ thống sử dụng kiểu dữ liệu có độ chính xác cố định DECIMAL cho trường lưu trữ số tiền nhằm triệt tiêu hoàn toàn rủi ro sai số làm tròn trong môi trường phân tán.

Cấu trúc thiết kế chi tiết của bảng dữ liệu được đặc tả qua bảng dưới đây:

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
id	INT	Primary Key, Auto Increment	Mã định danh bản ghi tự sinh.
order_id	INT	Not Null, Index	Mã đơn hàng (Khóa logic kết nối sang order_db).
user_id	INT	Not Null, Index	Mã khách hàng thực hiện thanh toán.
amount	DECIMAL(10,2)	Not Null	Số tiền thanh toán (Xác thực từ hệ thống gốc).
method	VARCHAR(50)	Not Null, Default="cash"	Phương thức (cash, card, momo, vnpay...).
status	VARCHAR(50)	Not Null, Default="pending"	Trạng thái giao dịch (pending, paid, failed...).
transaction_ref	VARCHAR(128)	Nullable, Unique	Mã giao dịch bảo mật do hệ thống cấp phát (VD: PAY-xxx).
note	TEXT	Nullable	Ghi chú thêm cho giao dịch.
created_at	DATETIME	Not Null, Default=NOW()	Thời gian khởi tạo yêu cầu thanh toán.
paid_at	DATETIME	Nullable	Thời điểm thanh toán thành công thực tế.

Bảng 3.5: Bảng payment_db

3.3. Thiết kế giao tiếp đồng bộ bằng RESTful API

3.3.1. Nguyên tắc thiết kế tài nguyên và ma trận phương thức HTTP

Trong kiến trúc Vi dịch vụ, giao tiếp đồng bộ (Synchronous Communication) qua giao thức HTTP giữ vai trò quyết định đến tính chuẩn hóa và khả năng cộng tác mượt mà giữa các thành phần phân tán. Để hiện thực hóa điều này, toàn bộ hệ thống endpoints của 05 cấu phần dịch vụ do đồ án phát triển đều được thiết kế tuân thủ nghiêm ngặt theo các nguyên tắc của kiến trúc REST (Representational State Transfer). Nguyên tắc cốt lõi của REST yêu cầu hệ thống phải định nghĩa các thực thể nghiệp vụ dưới dạng các "Tài nguyên" (Resources) định danh bằng các đường dẫn URL danh từ số nhiều (ví dụ: /api/products, /api/orders), đồng thời sử dụng các phương thức HTTP tiêu chuẩn (HTTP Methods) làm động từ để mô tả hành động tương tác dữ liệu.

Hệ thống quản lý đặt đồ ăn của đồ án ứng dụng trọn vẹn ma trận 04 phương thức HTTP cốt lõi, gắn liền với các đặc tính kỹ thuật phân tán quan trọng sau:

- **Phương thức GET (Đọc dữ liệu):** Được thiết kế dành cho các tác vụ trích xuất và hiển thị tài nguyên, cụ thể là luồng xem danh mục thực đơn hoặc kiểm tra lịch sử thông báo cá nhân. Đặc tính kỹ thuật bắt buộc của phương thức GET là tính an toàn (Safe) và tính lũy đẳng (Idempotent). Điều này có nghĩa là các request GET chỉ thực hiện truy vấn SELECT dữ liệu, tuyệt đối không làm thay đổi trạng thái của database hệ thống, và cho dù Client có gọi một API GET một lần hay hàng ngàn lần liên tiếp, kết quả trả về và trạng thái hệ thống vẫn hoàn toàn đồng nhất.
- **Phương thức POST (Khởi tạo tài nguyên):** Được áp dụng cho các luồng nghiệp vụ tạo mới thực thể như đăng ký tài khoản, thêm món ăn mới hoặc khởi tạo đơn hàng. Khác với GET, POST là phương thức không lũy đẳng (Non-idempotent); mỗi lần Client gửi một yêu cầu POST hợp lệ lên hệ thống, một bản ghi mới với mã định danh tự động tăng sẽ được khởi tạo trong cơ sở dữ liệu. Do đó, các API POST luôn được đặt sau các chốt chặn kiểm định dữ liệu nghiêm ngặt để tránh hiện tượng trùng lặp dữ liệu giao dịch.
- **Phương thức PUT (Cập nhật tài nguyên):** Được thiết kế chuyên biệt cho tác vụ chỉnh sửa, sửa đổi thông tin hoặc cập nhật trạng thái của tài nguyên hiện có

(ví dụ: Admin cập nhật lại đơn giá hoặc trạng thái còn/hết của món ăn tại dịch vụ sản phẩm). Phương thức PUT mang tính lũy đẳng; khi Client gửi một gói tin Payload chứa thông tin cập nhật thay thế cho một bản ghi xác định, trạng thái cuối cùng của bản ghi đó trong database sẽ không thay đổi bất kể yêu cầu được thực thi lặp lại bao nhiêu lần.

- **Phương thức DELETE (Xóa bỏ tài nguyên):** Được cấu hình cho tác vụ loại bỏ tài nguyên ra khỏi hệ thống, cụ thể là việc gỡ bỏ một món ăn khỏi danh mục thực đơn của Admin. Phương thức này cũng mang đặc tính lũy đẳng; yêu cầu xóa lần đầu tiên sẽ làm thay đổi trạng thái dữ liệu (bản ghi bị xóa), nhưng các yêu cầu lặp lại phía sau sẽ không làm thay đổi thêm bất kỳ trạng thái nào và hệ thống sẽ trả về cùng một mã trạng thái thích hợp (như lỗi 404 Not Found hoặc 200 OK tùy thiết kế mã nguồn).

Để tổng hợp và bao quát toàn bộ ma trận định tuyến, bảng dưới đây thể hiện sự phân bổ phương thức HTTP gắn liền với thẩm quyền truy cập tài nguyên được API Gateway điều phối tập trung tại cổng 8000 của hệ thống:

Dịch vụ đích (Service)	Tài nguyên (Resource URL)	Phương thức HTTP	Đặc tính kỹ thuật	Quyền truy cập (RBAC)	Mô tả nghiệp vụ
Auth Service (Cổng 8001)	/api/auth/register	POST	Non-idempotent	Công khai (Public)	Đăng ký tài khoản người dùng mới.
	/api/auth/login	POST	Non-idempotent	Công khai (Public)	Xác thực tài khoản, cấp Access Token (JWT).

Product Service (Cổng 8002)	/api/products	GET	Safe / Idempotent	Công khai (Public)	Duyệt xem danh sách toàn bộ món ăn.
	/api/products/{id}	GET	Safe / Idempotent	Công khai (Public)	Xem thông tin chi tiết một món ăn cụ thể.
	/api/products	POST	Non-idempotent	Quản trị viên (Admin)	Thêm món ăn mới vào thực đơn.
	/api/products/{id}	PUT	Idempotent	Quản trị viên (Admin)	Cập nhật giá cả, thông tin món ăn.
	/api/products/{id}	DELETE	Idempotent	Quản trị viên (Admin)	Xóa bỏ món ăn khỏi danh mục hệ thống.

Order Service (Công 8003)	/api/orders	POST	Non-idempotent	Khách hàng (Customer)	Tiếp nhận giỏ hàng, tính tổng tiền và tạo đơn.
	/api/orders	GET	Safe / Idempotent	Khách hàng (Customer)	Xem lịch sử danh sách đơn hàng cá nhân.
Notification Service (Công 8004)	/api/notifications	GET	Safe / Idempotent	Khách hàng (Customer)	Truy vấn lịch sử thông báo (đơn hàng, thanh toán).
Payment Service (Công 8005)	/api/payments	POST	Non-idempotent	Khách hàng (Customer)	Khởi tạo giao dịch thanh toán cho hóa đơn

					(Chờ xử lý).
	/api/payments/{id}/status	PATCH	Non-idempotent	Hệ thống / Admin	Cập nhật trạng thái dòng tiền (paid, failed,...).
	/api/payments/order/{order_id}	GET	Safe / Idempotent	Khách hàng (Customer)	Tra cứu thông tin thanh toán theo mã đơn hàng.

Bảng 3.6. Ma trận phân bổ phương thức HTTP và tài nguyên hệ thống

Việc định chuẩn ma trận phương thức kết hợp cấu trúc tài nguyên rõ ràng này giúp hệ thống của đồ án đạt được tính minh bạch tối đa. Mỗi phương thức truyền tải qua cửa ngõ Gateway đều mang một ý nghĩa nghiệp vụ và kỹ thuật định hình sẵn, giúp đơn giản hóa việc theo dõi luồng mạng nội bộ và tạo điều kiện tối ưu để hệ thống in ra các dòng nhật ký (Log) tường minh theo đúng yêu cầu kiểm soát phân tán của môn học.

3.3.2. Danh sách cấu trúc các API Endpoint chi tiết

Để đảm bảo tính phối hợp chính xác giữa ứng dụng khách (Client) và các thành phần phân tán qua cửa ngõ API Gateway, hệ thống định nghĩa chi tiết cấu trúc dữ liệu giao tiếp (Data Schemas) cho từng Endpoint nghiệp vụ cốt lõi. Định dạng dữ liệu trao

đồng nhất trên toàn hệ thống là JSON (JavaScript Object Notation), ngoại trừ endpoint đăng nhập ứng dụng cấu trúc Form Data theo chuẩn OAuth2.

Dưới đây là các bảng đặc tả chi tiết thông số cấu trúc đầu vào và đầu ra của các API trọng tâm được định tuyến qua Gateway tại cổng 8000:

1. Tài nguyên Xác thực và Định danh (Auth Resource)

- **API Đăng ký tài khoản (POST /api/auth/register):** Tiếp nhận thông tin khởi tạo tài khoản từ người dùng.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Request Body (JSON)	full_name	String	Not Null	Họ tên người dùng.
	email	String	Not Null, Unique	Thư điện tử (Tên đăng nhập).
	password	String	Not Null	Mật khẩu dạng thô.
	phone	String	Null	Số điện thoại.
Response (HTTP 201)	id	Int		Mã tài khoản tự sinh.
	full_name	String		Họ tên đã đăng ký.
	email	String		Email đã đăng ký.
	role	String		Vai trò mặc định (customer).

Bảng 3.7. Cấu trúc Request/Response của API POST /api/auth/register

- **API Đăng nhập hệ thống (POST /api/auth/login):** Xác thực tài khoản và cấp phát mã thông báo.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Request Body (Form)	username	String	Not Null	Tài khoản email đăng nhập.
	password	String	Not Null	Mật khẩu dạng thô.
Response (HTTP 200)	access_token	String	Not Null	Chuỗi mã thông báo JWT.
	token_type	String	Not Null	Định dạng mã (bearer).

Bảng 3.8. Cấu trúc Request/Response của API POST /api/auth/login

2. Tài nguyên Danh mục Món ăn (Products Resource)

- **API Lấy thực đơn (GET /api/products):** Hiển thị danh sách toàn bộ món ăn công khai.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Request	Không yêu cầu tham số đầu vào và mã thông báo bảo mật (Public API)			
Response (HTTP 200)	id	Int	Not Null	Mã định danh món ăn.
	name	String	Not Null	Tên món ăn.
	description	String	Null	Mô tả món ăn.

	price	Decimal	Not Null	Đơn giá món ăn.
	is_available	Boolean	Not Null	Trạng thái phục vụ.

Bảng 3.9. Cấu trúc Response của API GET /api/products

- **API Thêm món ăn mới (POST /api/products):** Quản trị viên khởi tạo sản phẩm mới.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Header	Authorization	String	Bearer [JWT Admin]	Mã thông báo quyền Admin.
Request Body (JSON)	name	String	Not Null	Tên món ăn cần tạo.
	description	String	Null	Mô tả chi tiết.
	price	Decimal	Not Null	Giá bán món ăn.
	category	String	Null	Phân loại danh mục.
	image_url	String	Null	Đường dẫn ảnh món ăn.
	is_available	Boolean	Default True	Trạng thái kinh doanh.
Response (HTTP 201)	Toàn bộ các trường dữ liệu như Request đi kèm trường id tự sinh từ database product_db.			

Bảng 3.10. Cấu trúc Request/Response của API POST /api/products

3. Tài nguyên Đơn hàng (Orders Resource)

- **API Khởi tạo đơn hàng (POST /api/orders):** Người dùng tiến hành đặt giỏ hàng món ăn.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Header	Authorization	String	Bearer [JWT Admin]	Mã thông báo quyền Admin.
Request Body (JSON)	user_id	Int	Not Null	Mã khách hàng đặt đơn.
	order_items	Array	Not Null	Mảng chứa danh sách giỏ hàng.
	product_id	Int	Not Null	Mã món ăn chọn mua.
	quantity	Int	Not Null	Số lượng đặt mua.
Response (HTTP 201)	id	Int	Not Null	Mã hóa đơn tự sinh tại order_db.
	user_id	Int	Not Null	Mã khách hàng đặt đơn.
	total_amount	Decimal	Not Null	Tổng tiền tính toán tự động.
	message	String	Not Null	Thông điệp phản hồi hệ thống.

Bảng 3.11. Cấu trúc Request/Response của API POST /api/orders

4. Tài nguyên Thông báo (Notifications Resource)

- **API Lấy danh sách thông báo (GET /api/notifications):** Khách hàng xem lịch sử thông báo cá nhân.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Header	Authorization	String	Bearer [JWT]	Mã thông báo xác thực người dùng.
Response (HTTP 200)	id	Int	Not Null	Mã định danh của thông báo.
	user_id	Int	Not Null	Mã khách hàng sở hữu thông báo.
	message	String	Not Null	Chuỗi nội dung thông báo biên dịch tự động.
	is_read	Boolean	Not Null	Trạng thái xem thông báo.

Bảng 3.12. Cấu trúc Response của API GET /api/notifications

5. Tài nguyên Thanh toán (Payments Resource)

- **API Khởi tạo giao dịch thanh toán (POST /api/payments):** Tiếp nhận yêu cầu thanh toán hóa đơn từ khách hàng.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Header	Authorization	String	Bearer [JWT]	Mã thông báo xác thực người dùng thực hiện thanh toán.

Request Body (JSON)	order_id	Int	Not Null	Mã hóa đơn cần thực hiện thanh toán (Khóa logic liên kết).
	payment_method	String	Not Null	Phương thức lựa chọn
Response (HTTP 201)	id	Int	Not Null	Mã định danh bản ghi giao dịch tự tăng trong database payment_db.
	order_id	Int	Not Null	Mã hóa đơn được liên kết thanh toán.
	transaction_code	String	Not Null, Unique	Mã giao dịch bảo mật duy nhất do hệ thống cấp phát ngẫu nhiên.
	amount	Decimal	Not Null	Số tiền cần thanh toán
	payment_method	String	Not Null	Phương thức thanh toán được chấp nhận ghi nhận.
	status	String	Default 'pending'	Trạng thái vòng đời ban đầu của giao dịch

	created_at	String	Not Null	Mốc thời gian khởi tạo lệnh giao dịch.
--	------------	--------	----------	--

Bảng 3.13. Cấu trúc Request/Response của API POST /api/payments

- API Cập nhật trạng thái giao dịch (PATCH /api/payments/{id}/status): Hệ thống đối tác hoặc quản trị viên cập nhật trạng thái dòng tiền.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Header	Authorization	String	Bearer [JWT]	Mã thông báo quyền quản trị hoặc mã nội bộ bảo mật hệ thống.
Request Body (JSON)	status	String	Not Null	Trạng thái dòng tiền cập nhật
Response (HTTP 200)	id	Int	Not Null	Mã định danh của bản ghi giao dịch vừa xử lý.
	transaction_code	String	Not Null	Mã giao dịch bảo mật duy nhất được cập nhật.
	status	String	Not Null	Trạng thái dòng tiền sau khi cập nhật thành công (paid).
	paid_at	String	Nullable	Mốc thời gian ghi nhận tiền

				vào tài khoản thành công
	message	String	Not Null	Thông điệp thông báo trạng thái xử lý và kích hoạt sự kiện liên dịch vụ.

Bảng 3.14. Cấu trúc Request/Response của API PATCH /api/payments/{id}/status

- API Tra cứu lịch sử thanh toán theo đơn hàng (GET /api/payments/order/{order_id}): Khách hàng kiểm tra thông tin tài chính của hóa đơn.

Giai đoạn	Tham số cấu trúc dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nghiệp vụ
Header	Authorization	String	Bearer [JWT]	Mã thông báo xác thực người dùng sở hữu đơn hàng.
Request	order_id	Int	Not Null	Tham số truyền trên đường dẫn để lọc theo mã đơn hàng.
Response (HTTP 200)	id	Int	Not Null	Mã định danh bản ghi giao dịch trong cơ sở dữ liệu.
	order_id	Int	Not Null	Mã hóa đơn liên kết.

	transaction_code	String	Not Null	Mã giao dịch bảo mật của hóa đơn.
	amount	Decimal	Not Null	Tổng số tiền thực tế đã ghi nhận giao dịch.
	payment_method	String	Not Null	Phương thức thanh toán được áp dụng cho hóa đơn này.
	status	String	Not Null	Trạng thái hiện tại của dòng tiền
	paid_at	String	Nullable	Thời gian thanh toán hóa đơn thành công

Bảng 3.15. Cấu trúc Response của API GET /api/payments/order/{order_id}

Toàn bộ hệ thống bảng đặc tả cấu trúc này được ánh xạ chính xác vào các mô hình Pydantic độc lập tại tầng mã nguồn của từng service con. Cơ chế này đảm bảo mọi thông điệp truyền tải qua hệ thống RESTful API đều được kiểm định chặt chẽ, loại bỏ rủi ro sai lệch định dạng và tạo cơ sở dữ liệu đồng nhất phục vụ luồng giao tiếp nội bộ.

3.3.3. Cơ chế giao tiếp đồng bộ nội bộ giữa Order Service và Product Service

Trong khi API Gateway đóng vai trò điều phối luồng dữ liệu từ môi trường ngoài vào hệ thống, cơ chế giao tiếp đồng bộ nội bộ (Internal Synchronous Communication) giữa order-service và product-service lại là mắt xích quyết định để hoàn thiện một chu kỳ giao dịch đặt hàng thành công. Do đặc thù của nguyên lý Database-per-service, dịch vụ đơn hàng (order-service) không có quyền truy cập trực tiếp vào database product_db để lấy đơn giá món ăn. Vì vậy, một liên kết gọi hàm từ xa thông qua giao thức RESTful

API bất đồng bộ (Asynchronous HTTP Call) đã được thiết lập trực tiếp giữa hai container, bỏ qua tầng lọc trung gian của Gateway để tối ưu hóa tốc độ truyền tải nội bộ.

Quy trình bắt tay và trao đổi dữ liệu đồng bộ giữa hai dịch vụ được thực hiện tuần tự theo các bước kỹ thuật sau:

1. **Khởi tạo kết nối Client-Side:** Khi khách hàng gửi yêu cầu đặt giỏ hàng đến order-service (cổng 8003), một HTTP Client bất đồng bộ được khởi tạo bên trong tầng Logic của dịch vụ đơn hàng. Đối với mỗi mục sản phẩm (product_id) có trong danh sách, order-service sẽ thực hiện gửi một request HTTP GET trực tiếp đến địa chỉ mạng nội bộ của dịch vụ sản phẩm: `http://product-service:8002/api/products/{product_id}`.
2. **Tiếp nhận và xử lý tại tuyến sau:** Container product-service tiếp nhận gói tin tại cổng 8002. Tiến trình Uvicorn ngậm lập tức phân rã tham số product_id và thực thi câu lệnh truy vấn dữ liệu bất đồng bộ (SELECT) vào bảng products của product_db. Dữ liệu trích xuất bao gồm đơn giá gốc (price) và trạng thái kinh doanh (is_available).
3. **Phản hồi và hoàn tất chu kỳ:** Dịch vụ sản phẩm đóng gói thông tin thành một chuỗi JSON chuẩn hóa và gửi trả ngược lại theo kết nối HTTP đang mở. order-service nhận gói tin phản hồi, giải mã dữ liệu để lấy đơn giá phục vụ cho phép toán nhân tính tổng số tiền (total_amount), sau đó đóng kết nối mạng để giải phóng tài nguyên hệ thống.

Một điểm nhấn kỹ thuật quan trọng trong thiết kế mã nguồn của đồ án là việc áp dụng từ khóa `async/await` xuyên suốt luồng giao tiếp này. Việc gửi yêu cầu HTTP nội bộ được thực hiện theo cơ chế không chặn (Non-blocking I/O). Trong khoảng thời gian chờ đợi product-service phản hồi dữ liệu giá, luồng luân chuyển (Event Loop) của order-service không bị rơi vào trạng thái nghẽn mạch (Thread Blocking) mà vẫn có thể tiếp tục tiếp nhận các yêu cầu khởi tạo đơn hàng mới từ các khách hàng khác. Điều này giúp hệ thống duy trì được hiệu năng xử lý cao và độ trễ ở mức thấp nhất.

Bên cạnh luồng xử lý thành công, cơ chế giao tiếp đồng bộ này còn tích hợp tầng kiểm soát lỗi biên (Edge Case Handling) chặt chẽ để bảo vệ tính toàn vẹn dữ liệu:

- **Trường hợp sản phẩm không tồn tại:** Nếu mã product_id do Client gửi lên không tìm thấy trong database product_db, product-service sẽ phản hồi về mã

trạng thái HTTP 404 Not Found. Ngay khi nhận được mã lỗi này, order-service sẽ lập tức hủy bỏ giao dịch, không ghi dữ liệu xuống order_db và trả về thông báo lỗi cụ thể cho Client để ngăn chặn hành vi đặt món ăn giả mạo.

- **Trường hợp sản phẩm hết hàng:** Nếu món ăn được tìm thấy nhưng có trường trạng thái is_available = false, dịch vụ sản phẩm sẽ gửi kèm thông tin này trong gói tin JSON. Dịch vụ đơn hàng sẽ chặn luồng xử lý và đưa ra cảnh báo lỗi nghiệp vụ, bảo đảm hệ thống không bao giờ phát hành các hóa đơn cho những món ăn đã dừng kinh doanh.

Cơ chế giao tiếp RESTful đồng bộ trực tiếp này đảm bảo dữ liệu tài chính luôn được cập nhật chính xác theo thời gian thực (Real-time Validation) trước khi hệ thống chuyển giao trạng thái sang luồng xử lý bất đồng bộ tiếp theo với hàng đợi RabbitMQ.

3.4. Thiết kế giao tiếp bất đồng bộ qua Broker RabbitMQ

3.4.1. Cấu hình kiến trúc RabbitMQ

Để tối ưu hóa hiệu năng luồng mạng và hiện thực hóa mô hình kiến trúc Hướng sự kiện (Event-driven Architecture), hệ thống phân tán của đồ án triển khai một hạ tầng điều phối thông điệp bất đồng bộ dựa trên nền tảng RabbitMQ. Trong mô hình này, việc truyền tải dữ liệu giữa các dịch vụ không còn phụ thuộc vào các kết nối HTTP trực tiếp, giúp triệt tiêu hoàn toàn rủi ro nghẽn mạch dây chuyền (Cascading Failure). Để luồng thông điệp vận hành một cách chính xác, đồ án thực hiện cấu hình tường minh cấu trúc topo mạng nội bộ của RabbitMQ thông qua ba thực thể cốt lõi: Exchange, Queue và Routing Key.

Thành phần đầu tiên trong chuỗi cấu hình là **Exchange trung gian mang tên order_exchange**. Khác với các mô hình hàng đợi đơn giản nơi Publisher đẩy dữ liệu trực tiếp vào một Queue cố định, đồ án ứng dụng cơ chế định tuyến thông minh thông qua việc cấu hình order_exchange theo loại **Direct Exchange**. Loại Exchange này hoạt động dựa trên thuật toán đối chiếu chuỗi ký tự chính xác tuyệt đối. Khi order-service hoàn tất tiến trình ghi nhận hóa đơn vĩnh viễn xuống order_db, nó sẽ đóng vai trò là một Publisher, kết nối đến RabbitMQ qua giao thức AMQP tại cổng 5672 và xuất bản (Publish) một gói tin dữ liệu JSON lên order_exchange.

Thành phần thứ hai là **Hàng đợi thông điệp mang tên notification_queue**. Đây là một vùng đệm lưu trữ dữ liệu vật lý vận hành theo cơ chế FIFO (First In, First Out)

và được quản trị độc quyền bởi dịch vụ tiêu thụ ngầm. Nhằm đảm bảo an toàn thông tin, hàng đợi này được cấu hình thuộc tính `durable = true`, nghĩa là toàn bộ thông điệp nằm trong hàng đợi sẽ được ghi nhận trực tiếp xuống đĩa cứng của máy chủ. Đặc tính kỹ thuật này giúp hệ thống bảo toàn nguyên vẹn các sự kiện giao dịch ngay cả khi container RabbitMQ gặp sự cố đột xuất hoặc bị khởi động lại (Restart).

Mắt xích quan trọng nhất thiết lập mối liên kết giữa Exchange và Queue chính là **Khóa định tuyến (Routing Key) với nhãn định danh `order.created`**. Luồng liên kết (Binding) này được cấu hình chi tiết thông qua các bước kỹ thuật sau:

1. **Thiết lập Binding logic:** Hệ thống thực hiện một lệnh ràng buộc cố định, gắn kết `notification_queue` vào `order_exchange` kèm theo điều kiện lọc duy nhất là chuỗi ký tự `order.created`.
2. **Khớp nhãn thông điệp:** Khi `order-service` đẩy gói tin lên Exchange, nó bắt buộc phải đính kèm nhãn định tuyến `order.created` vào phần tiêu đề (Header) của thông điệp.
3. **Phân phối dữ liệu tự động:** `order_exchange` tiếp nhận gói tin, phân tích nhãn và đối chiếu với ma trận Binding. Vì chuỗi ký tự trùng khớp hoàn toàn, Exchange sẽ tự động định hướng và chuyển giao toàn bộ gói tin JSON vào trong `notification_queue` để chờ `notification-service` (Consumer) đến tiêu thụ.

Việc thiết lập cấu hình topo mạng RabbitMQ với cấu trúc bộ ba (`order_exchange` $\rightarrow$ `order.created` $\rightarrow$ `notification_queue`) mang lại khả năng mở rộng (Scalability) linh hoạt cho hệ thống phân tán. Trong tương lai, nếu đề án cần phát triển thêm các phân hệ hậu mãi khác như dịch vụ phân tích dữ liệu (`analytics-service`) hoặc dịch vụ quản lý kho (`inventory-service`), lập trình viên chỉ cần tạo các hàng đợi mới và ràng buộc (Bind) chúng vào `order_exchange` với cùng khóa `order.created`. Khi đó, một sự kiện tạo đơn hàng sinh ra sẽ được tự động nhân bản và phân phối đến nhiều dịch vụ cùng lúc mà nhóm phát triển hoàn toàn không cần phải chỉnh sửa hay tái cấu trúc lại mã nguồn của dịch vụ đơn hàng gốc.

3.4.2. Sơ đồ trình tự luồng xử lý nghiệp vụ bất đồng bộ

Để cụ thể hóa mối quan hệ tương tác, thời gian sống và luồng chuyển giao dữ liệu giữa các thành phần phân tán khi áp dụng kiến trúc hướng sự kiện, sơ đồ trình tự (Sequence Diagram) dưới đây mô tả chi tiết tiến trình từ lúc `order-service` hoàn tất ghi

nhận hóa đơn cho đến khi khách hàng nhận được thông báo trạng thái đặt hàng thành công từ notification-service. Sơ đồ này làm nổi bật tính chất "phi kết nối" và cơ chế bất đồng bộ, nơi mà các dịch vụ nghiệp vụ được giải phóng tài nguyên một cách tối đa nhờ vai trò điều phối trung gian của Message Broker RabbitMQ.

1. Mô tả kịch bản tương tác trong sơ đồ

Tiến trình xử lý bất đồng bộ hệ thống bao gồm 05 thực thể chính tham gia vào luồng công việc:

- **Client (Ứng dụng khách):** Chủ thể khởi tạo yêu cầu đặt hàng và tiêu thụ API hiển thị thông báo.
- **Order Service (order-service):** Dịch vụ tiếp nhận giỏ hàng, tính toán tài chính và xuất bản sự kiện.
- **RabbitMQ (Message Broker):** Hạ tầng trung gian tiếp nhận, định tuyến và lưu đệm thông điệp.
- **Notification Service (notification-service):** Dịch vụ chạy ngầm tiêu thụ thông điệp và xử lý logic thông báo.
- **Notification DB (notification_db):** Cơ sở dữ liệu vật lý lưu trữ lịch sử thông báo.

2. Biểu diễn sơ đồ trình tự kỹ thuật

3. Phân tích chi tiết các bước thực thi trong luồng nghiệp vụ

Sự luân chuyển thông điệp và ngữ cảnh xử lý nội bộ giữa các thực thể được chia thành hai giai đoạn độc lập tuyến tính:

Giai đoạn A: Xuất bản sự kiện và giải phóng luồng mạng (Publisher Side)

- **Bước 1:** Sau khi thực hiện chu trình kiểm tra đơn giá đồng bộ với dịch vụ sản phẩm, order-service thực thi các câu lệnh SQL để lưu trữ vĩnh viễn thông tin đơn hàng vào order_db.
- **Bước 2:** Ngay khi dữ liệu được ghi nhận thành công xuống đĩa cứng, order-service tiến hành đóng gói dữ liệu bao gồm mã đơn hàng (order_id) và mã người dùng (user_id) thành một gói tin định dạng JSON. Dịch vụ thiết lập một kết nối AMQP bất đồng bộ và thực hiện lệnh publish() để đẩy gói tin này lên order_exchange của RabbitMQ kèm theo nhãn định tuyến order.created.

- **Bước 3:** Ngay sau khi nhận được tín hiệu xác nhận gói tin đã nằm trên Exchange, order-service lập tức giải phóng luồng mạng và phản hồi trạng thái HTTP 201 Created về cho Client. Tại thời điểm này, chu kỳ request-response của luồng đặt hàng chính thức kết thúc, người dùng nhận được thông báo đặt hàng thành công trên giao diện mà không cần phải chờ đợi các tác vụ xử lý hậu mãi phía sau.

Giai đoạn B: Định tuyến, tiêu thụ sự kiện và đồng bộ dữ liệu (Broker & Consumer Side)

- **Bước 4:** Tại hạ tầng trung gian, order_exchange tiếp nhận thông điệp, phân tích nhãn và thực hiện thuật toán so khớp chính xác. Do nhãn trùng khớp với cấu hình ràng buộc, Exchange tự động chuyển giao gói tin vào vùng đệm an toàn của notification_queue.
- **Bước 5:** Ở phía đầu ra của hàng đợi, tiến trình ngầm (Worker Task) của notification-service liên tục duy trì cơ chế lắng nghe chủ động. Ngay khi phát hiện có dữ liệu mới, RabbitMQ thực hiện phân phối sự kiện order.created về cho dịch vụ thông báo xử lý.
- **Bước 6:** notification-service tiếp nhận gói tin JSON, thực hiện giải mã dữ liệu cấu phần để lấy ra các tham số, sau đó tự động biên dịch chuỗi văn bản thông báo theo cấu trúc định sẵn.
- **Bước 7:** Dịch vụ thông báo thực thi câu lệnh INSERT để ghi nhận bản ghi mới (bao gồm nội dung tin nhắn, mã user_id và trạng thái mặc định chưa đọc) vào bảng notifications thuộc cơ sở dữ liệu notification_db.
- **Bước 8:** Tại một thời điểm bất kỳ sau đó, khi người dùng truy cập vào phân hệ lịch sử, Client sẽ gửi một yêu cầu HTTP GET độc lập đến endpoint /api/notifications. notification-service tiếp nhận yêu cầu, thực hiện truy vấn nhanh (SELECT) vào notification_db để trích xuất dữ liệu thông báo và trả về hiển thị trực quan cho khách hàng.

Sơ đồ trình tự trên là minh chứng rõ nét cho thấy hệ thống đã đạt được trạng thái nhất quán sau cùng của dữ liệu (Eventual Consistency). Việc tách biệt luồng xử lý thông báo thành một tiến trình chạy ngầm bất đồng bộ giúp giảm thiểu áp lực tải cho lõi hệ thống, tăng tính chịu lỗi và tối ưu hóa trải nghiệm mượt mà cho người sử dụng ứng dụng đặt đồ ăn trực tuyến.

3.4.3. *Đánh giá tính nhất quán dữ liệu giữa các dịch vụ*

Trong các hệ thống kiến trúc nguyên khối truyền thống sử dụng một cơ sở dữ liệu tập trung, tính nhất quán dữ liệu được đảm bảo một cách tuyệt đối thông qua cơ chế giao dịch ACID mạnh mẽ (Strong Consistency). Tuy nhiên, khi chuyển dịch sang kiến trúc Vi dịch vụ với mô hình dữ liệu phân rã "Database-per-service", việc duy trì tính nhất quán mạnh mẽ xuyên suốt các phân vùng dữ liệu vật lý độc lập (order_db và notification_db) là điều bất khả thi nếu không muốn đánh đổi bằng hiệu năng và tính sẵn sàng của toàn hệ thống. Đối chiếu với Định lý CAP (CAP Theorem) trong hệ thống phân tán, đồ án đã lựa chọn ưu tiên Tính sẵn sàng (Availability) và Tính chịu lỗi khi phân tách mạng (Partition Tolerance), từ đó chấp nhận đánh đổi tính nhất quán tức thời để hướng tới **Tính nhất quán sau cùng (Eventual Consistency)**.

Đánh giá chi tiết về cơ chế hiện thực hóa và tính đúng đắn của mô hình nhất quán sau cùng bên trong hệ thống đặt đồ án của đồ án được thể hiện qua các luận điểm kỹ thuật sau:

- **Giải quyết trạng thái bất đồng bộ tạm thời:** Khi một đơn hàng được khởi tạo thành công, dữ liệu tài chính lập tức được ghi nhận vĩnh viễn vào order_db và thông điệp được đẩy vào RabbitMQ. Tại thời điểm milli-giây đó, bảng notifications bên trong notification_db vẫn chưa hề tồn tại bản ghi thông báo tương ứng. Hệ thống rơi vào trạng thái không nhất quán tạm thời (Temporary Inconsistency). Tuy nhiên, trạng thái này hoàn toàn chấp nhận được về mặt nghiệp vụ; khách hàng quan tâm đến việc đơn hàng được hệ thống tiếp nhận thành công hơn là việc ứng dụng phải hiển thị tin nhắn thông báo ngay lập tức trong cùng một xung nhịp clock.
- **Cơ chế hội tụ dữ liệu tự động:** Nhờ có sự điều phối trung gian của hàng đợi RabbitMQ, sự kiện tạo đơn hàng mang khóa định tuyến order.created sẽ dịch chuyển tuần tự và kích hoạt tiến trình Worker của notification-service. Dù có một khoảng trễ thời gian nhất định (thường chỉ vài mili-giây trong điều kiện mạng ổn định), dịch vụ thông báo chắc chắn sẽ tiêu thụ thông điệp và thực thi câu lệnh ghi dữ liệu xuống notification_db. Sau khi tiến trình này hoàn tất, dữ liệu giữa hai dịch vụ đạt đến trạng thái đồng bộ và hội tụ hoàn toàn. Đây chính là bản chất cốt lõi của nguyên lý nhất quán sau cùng.

- **Nâng cao năng lực chịu tải và trải nghiệm người dùng:** Nếu hệ thống bắt buộc phải tuân theo tính nhất quán mạnh (gọi API đồng bộ sang dịch vụ thông báo rồi mới trả kết quả về cho khách hàng), luồng đặt hàng chính sẽ bị kéo dài thời gian phản hồi một cách vô lý. Bằng cách áp dụng Eventual Consistency thông qua Broker, order-service giải phóng luồng HTTP cực nhanh, tối ưu hóa trải nghiệm mượt mà cho người dùng và giúp hệ thống chịu được mật độ đơn hàng cao điểm lớn hơn gấp nhiều lần.

Bên cạnh các ưu điểm về hiệu năng, đồ án cũng thiết lập cấu hình hạ tầng chặt chẽ để kiểm soát rủi ro mất an toàn dữ liệu – nhược điểm phổ biến của mô hình nhất quán sau cùng – thông qua hai giải pháp kỹ thuật cụ thể:

1. **Kháng lỗi mất mát thông điệp (Message Durability):** Như đã phân tích tại cấu hình topo mạng, hàng đợi notification_queue được thiết lập thuộc tính durable = true. Khi có sự cố ngắt kết nối vật lý xảy ra giữa các container, các thông điệp sự kiện tạo đơn hàng không bị tiêu biến mà được lưu vết an toàn trên đĩa cứng của Broker, sẵn sàng chờ dịch vụ phục hồi để tiếp tục xử lý, đảm bảo hệ thống luôn luôn đạt được trạng thái nhất quán sau cùng.
2. **Thiết kế xử lý lũy đẳng tại Consumer (Idempotent Consumer):** Trong môi trường phân tán, các sự cố mạng có thể khiến Broker phân phối một thông điệp nhiều hơn một lần (At-least-once delivery). Để ngăn chặn việc hệ thống sinh ra các thông báo trùng lặp cho cùng một đơn hàng, tầng nghiệp vụ của notification-service có thể thực hiện kiểm tra logic dựa trên trường mã định danh đơn hàng (order_id) nhận được. Nếu mã này đã tồn tại trong bảng notifications, Consumer sẽ chủ động bỏ qua tác vụ ghi và chỉ gửi tín hiệu xác nhận (ACK) về cho RabbitMQ.

Tổng kết lại, việc áp dụng mô hình nhất quán sau cùng (Eventual Consistency) thông qua giải pháp công nghệ RabbitMQ là một quyết định kiến trúc hoàn toàn đúng đắn và phù hợp với quy mô của đề tài. Giải pháp này giúp hệ thống đạt được sự cân bằng tối ưu giữa tốc độ vận hành, tính độc lập của kiến trúc Microservices và độ tin cậy tuyệt đối của dữ liệu lịch sử giao dịch.

3.5. Thiết kế cơ chế xử lý lỗi cơ bản và ghi nhận nhật ký

3.5.1. Giải pháp xử lý khi dịch vụ liên quan tạm thời không phản hồi

Trong môi trường phân tán của kiến trúc Vi dịch vụ, các sự cố liên quan đến đường truyền mạng, độ trễ phản hồi (Latency) hoặc một dịch vụ thành phần đột ngột bị quá tải dẫn đến ngưng hoạt động tạm thời là những kịch bản không thể tránh khỏi. Khác với kiến trúc nguyên khối nơi các hàm được gọi trực tiếp trong cùng một vùng nhớ, hệ thống vi dịch vụ của đồ án phải đối mặt với rủi ro lỗi dây chuyền (Cascading Failure) — hiện tượng một dịch vụ tụt xuống sau bị sập kéo theo sự đóng băng tài nguyên ở các dịch vụ tụt xuống trước. Để bảo vệ hệ thống khỏi các lỗ hổng này, đồ án đã triển khai đồng thời hai giải pháp kỹ thuật cốt lõi: Thiết lập thời gian chờ (Timeout) kết hợp kiểm soát ngoại lệ ở luồng đồng bộ, và tận dụng cơ chế lưu đệm hàng đợi (Message Buffering) ở luồng bất đồng bộ.

Đối với luồng nghiệp vụ đồng bộ, trường hợp điển hình nhất xảy ra khi order-service thực hiện gọi RESTful API sang product-service để truy xuất đơn giá món ăn tại cổng 8002. Nếu dịch vụ sản phẩm đang phải xử lý một lượng tải quá lớn hoặc cơ sở dữ liệu product_db bị khóa (Locking), kết nối HTTP mở từ dịch vụ đơn hàng sẽ bị treo vô thời hạn nếu không có chốt chặn kỹ thuật. Để giải quyết rủi ro này, hệ thống áp dụng các giải pháp xử lý lỗi biên sau:

- **Thiết lập cơ chế Timeout nghiêm ngặt:** Tại mã nguồn khởi tạo HTTP Client của order-service, hệ thống cấu hình một khoảng thời gian chờ tối đa (thường là 3 đến 5 giây) cho mọi yêu cầu gửi đi. Nếu quá thời hạn này mà product-service không trả về gói tin JSON phản hồi, HTTP Client sẽ chủ động ngắt kết nối vật lý, giải phóng luồng mạng (Event Loop) và ngăn chặn tình trạng cạn kiệt thread xử lý tại dịch vụ đơn hàng.
- **Bắt và xử lý ngoại lệ tại tầng ứng dụng (Exception Handling):** Khi sự kiện Timeout kích hoạt, hệ thống sẽ ném ra một ngoại lệ lỗi kết nối mạng. Tầng logic của FastAPI sẽ lập tức bắt giữ (Catch) ngoại lệ này, tự động thực thi lệnh hủy bỏ (Rollback) tiến trình, đồng thời trả về mã trạng thái HTTP 503 Service Unavailable kèm theo thông điệp thông báo tường minh cho khách hàng: *"Hệ thống kiểm tra thực đơn đang bận, vui lòng thử lại sau"*. Giải pháp này đảm bảo ứng dụng luôn phản hồi khách hàng một cách chủ động và có kiểm soát, thay vì

để lộ các dòng thông báo lỗi hệ thống (Internal Server Error) gây sụt giảm trải nghiệm người dùng.

Đối với luồng nghiệp vụ bất đồng bộ, giải pháp xử lý khi dịch vụ liên quan không phản hồi được giải quyết một cách triệt để và tự động nhờ đặc tính phi kết nối của hạ tầng trung gian RabbitMQ. Kịch bản lỗi xảy ra khi người dùng đặt hàng thành công, thông điệp đã nằm trên Exchange, nhưng container notification-service đột ngột bị dừng hoạt động để bảo trì hoặc gặp sự cố ngắt kết nối mạng.

Nhờ cấu hình hàng đợi bền vững (durable = true) kết hợp cơ chế xác nhận thông điệp (Message Acknowledgment - ACK) của RabbitMQ, toàn bộ dữ liệu sự kiện order.created sẽ được giữ lại an toàn tuyệt đối trong notification_queue. Hệ thống hàng đợi đóng vai trò như một bộ đệm lưu trữ, liên tục giữ thông điệp ở trạng thái chờ phân phối mà không gây ra bất kỳ áp lực tải hay ảnh hưởng nào đến luồng đặt hàng chính của order-service. Ngay khi dịch vụ thông báo được khởi chạy trở lại và thiết lập lại kết nối AMQP, RabbitMQ sẽ tự động đẩy tuần tự các thông điệp tồn đọng về cho Consumer tiêu thụ. Tiến trình này diễn ra hoàn toàn tự động dưới tầng hạ tầng, giúp hệ thống phục hồi trạng thái nhất quán sau cùng của dữ liệu mà không cần đến sự can thiệp thủ công của người vận hành.

3.5.2. Giải pháp xử lý khi message trên queue bị lỗi hoặc không thể tiêu thụ

Trong một hệ thống hướng sự kiện, bên cạnh kịch bản dịch vụ tiêu thụ (Consumer) tạm thời ngắt kết nối, hệ thống còn phải đối mặt với một rủi ro kỹ thuật phức tạp hơn: thông điệp vẫn được phân phối đến Consumer thành công nhưng không thể tiêu thụ hoặc xử lý lỗi liên tục. Hiện tượng này thường xuất phát từ hai nguyên nhân chính: lỗi dữ liệu hư hỏng (Malformed Message/Poison Pill) — khi gói tin JSON bị sai cấu trúc schema hoặc thiếu trường thông tin bắt buộc khiến tầng kiểm định Pydantic ném ra lỗi, hoặc lỗi nghiệp vụ tuyến sau — khi cơ sở dữ liệu notification_db bị khóa dẫn đến lệnh INSERT bị từ chối. Nếu không có cơ chế xử lý chuyên biệt, thông điệp lỗi này sẽ bị đẩy ngược lại hàng đợi và phân phối lại vô hạn (Infinite Redelivery Loop), gây nghẽn mạch toàn bộ hệ thống và cạn kiệt tài nguyên CPU của vi dịch vụ.

Để xử lý triệt để bài toán này, đề án đã triển khai giải pháp quản lý vòng đời thông điệp nghiêm ngặt kết hợp giữa cơ chế xác nhận thủ công (Manual Acknowledgment) và kiến trúc Hàng đợi thư chết (Dead Letter Queue - DLQ).

Quy trình kiểm soát và xử lý thông điệp lỗi tại notification-service được thiết lập chi tiết qua các bước kỹ thuật sau:

- **Áp dụng cơ chế xác nhận tường minh (Explicit ACK/NACK):** Hệ thống cấu hình thuộc tính `no_ack = False` tại Consumer để tắt chế độ tự động xóa thông điệp của RabbitMQ. Khi notification-service nhận được gói tin JSON từ `notification_queue`, dịch vụ sẽ đưa toàn bộ logic xử lý (giải mã, biên dịch nội dung, ghi database) vào bên trong một khối lệnh kiểm soát ngoại lệ try-except.
 - Nếu tiến trình thực thi hoàn toàn thành công và không phát sinh bất kỳ lỗi biên nào, Consumer sẽ chủ động gửi về một tín hiệu xác nhận thành công (**ACK**) để RabbitMQ chính thức xóa bỏ thông điệp ra khỏi hàng đợi.
 - Ngược lại, nếu bất kỳ ngoại lệ nào bị kích hoạt (lỗi định dạng dữ liệu hoặc lỗi kết nối database), khối except sẽ lập tức bắt giữ ngoại lệ, ngăn không cho dịch vụ bị sập, đồng thời gửi trả về một tín hiệu từ chối xác nhận (**NACK** kèm theo tham số `requeue = false`) để yêu cầu hệ thống trung gian gỡ bỏ thông điệp lỗi ra khỏi luồng phân phối chính.
- **Cấu hình hạ tầng Hàng đợi thư chết (Dead Letter Queue):** Để không làm mất mát dữ liệu của các thông điệp bị từ chối (NACK), đồ án thiết lập một topo mạng phụ trợ bên trong RabbitMQ bao gồm một Exchange chuyên biệt mang tên `dlx.order_exchange` và một hàng đợi lưu vết độc lập mang tên `order_dead_letter_queue (DLQ)`. Hàng đợi chính `notification_queue` khi khởi tạo sẽ được đính kèm thuộc tính tham số cấu hình `x-dead-letter-exchange` trỏ về `dlx.order_exchange`.

Khi tín hiệu NACK với thuộc tính `requeue = false` được Consumer gửi về, hoặc khi một thông điệp vượt quá số lần thử lại quy định (Retry Threshold), RabbitMQ sẽ tự động trích xuất thông điệp đó ra khỏi `notification_queue` và chuyển hướng sang `dlx.order_exchange`. Tại đây, thư chết sẽ được dịch chuyển và lưu trữ vĩnh viễn tại `order_dead_letter_queue`.

Sự xuất hiện của giải pháp kiến trúc DLQ mang lại ba giá trị cốt lõi cho công tác vận hành hệ thống phân tán của đồ án:

1. **Bảo vệ luồng mạng nội bộ (Isolation):** Các thông điệp bị lỗi (Poison Pills) lập tức bị cô lập ra một phân vùng riêng, giúp giải phóng hàng đợi chính

notification_queue để các thông điệp của các đơn hàng kế tiếp được di chuyển và xử lý tuần tự mà không bị ách tắc.

2. **Hỗ trợ giám sát và cảnh báo lỗi (Observability):** Người vận hành hệ thống thông qua giao diện RabbitMQ Management có thể lập tức phát hiện số lượng thông điệp tồn đọng tại hàng đợi thư chết, từ đó nhận biết sớm các bất thường về mặt logic dữ liệu giữa các dịch vụ.
3. **Khả năng tái xử lý thủ công (Replay Capability):** Sau khi lập trình viên tìm ra nguyên nhân gây lỗi và tiến hành cập nhật bản vá lỗi cho mã nguồn, hệ thống cho phép chạy một script hỗ trợ để trích xuất các thông điệp từ DLQ, đóng gói và đẩy ngược lại Exchange chính để tái xử lý mà không làm mất đi bất kỳ dữ liệu lịch sử nào của khách hàng.

3.5.3. Định dạng cấu trúc log theo dõi Request HTTP và Message Event

Trong một hệ thống phân tán phức tạp gồm nhiều dịch vụ chạy ngầm độc lập, nhật ký hệ thống (Logs) được ví như các mảnh mồi dữ liệu duy nhất giúp người vận hành hiểu được trạng thái bên trong của ứng dụng. Nếu sử dụng các dòng log văn bản thuần (Plain Text) dạng tự do, việc thu thập, tìm kiếm và phân tích lỗi xuyên suốt các container sẽ gặp rất nhiều khó khăn. Do đó, đề án đã áp dụng giải pháp **Chuẩn hóa cấu trúc nhật ký (Structured Logging)** dưới định dạng gói tin JSON cho toàn bộ các dịch vụ. Mọi sự kiện phát sinh từ luồng giao tiếp RESTful API đồng bộ hay luồng sự kiện RabbitMQ bất đồng bộ đều được ghi vệt theo một cấu trúc schema nghiêm ngặt, cho phép các hệ thống giám sát tập trung dễ dàng lập chỉ mục (Indexing) và truy vấn lỗi theo thời gian thực.

Để bao quát toàn bộ các hành vi giao tiếp liên dịch vụ, cấu trúc định dạng log của hệ thống được chia thành hai danh mục kỹ thuật chuyên biệt:

1. Định dạng cấu trúc Log theo dõi Request HTTP (Giao tiếp đồng bộ)

Mỗi khi API Gateway hoặc các vi dịch vụ phía sau tiếp nhận hoặc thực thi một yêu cầu HTTP (chẳng hạn như order-service gọi sang product-service), một bản ghi nhật ký dạng JSON sẽ được in ra bảng điều khiển (Console/Standard Output) với cấu trúc định chuẩn như sau:

JSON

{


```

"timestamp": "2026-05-25T23:45:12.894Z",
"level": "INFO",
"service_name": "order-service",
"trace_id": "1ef1a2b3-cd45-6789-abcd-ef0123456789",
"log_type": "HTTP_REQUEST",
"context": {
  "method": "POST",
  "url": "http://order-service:8003/api/orders",
  "status_code": 201,
  "latency_ms": 142.5,
  "client_ip": "172.20.0.5",
  "user_id": 42
}
}

```

Các trường thông tin cấu phần cốt lõi trong Schema Log HTTP bao gồm:

- **timestamp:** Thời gian phát sinh sự kiện theo định dạng chuẩn quốc tế ISO 8601, chính xác đến từng mili-giây.
- **level:** Mức độ nghiêm trọng của log (INFO cho luồng chạy bình thường, WARN cho cảnh báo biên, và ERROR khi phát sinh ngoại lệ lỗi).
- **trace_id:** Chuỗi định danh duy nhất (UUID) được sinh ra ngay tại API Gateway khi tiếp nhận request đầu tiên của khách hàng. Chuỗi này sẽ được đính kèm vào tiêu đề HTTP và truyền tải xuyên suốt qua các dịch vụ tuyến sau. Đây là chìa khóa chiến lược để lập trình viên có thể lọc và kết nối toàn bộ hành trình của một request đi qua nhiều dịch vụ khác nhau (Distributed Tracing).
- **context:** Phân đoạn chứa thông số kỹ thuật chi tiết của giao thức mạng, bao gồm phương thức HTTP (method), địa chỉ URL đích, mã trạng thái phản hồi (status_code), thời gian xử lý của dịch vụ (latency_ms), IP của container gửi yêu cầu, và mã định danh của khách hàng (user_id) nếu request đã được xác thực thành công.

2. Định dạng cấu trúc Log theo dõi Message Event (Giao tiếp bất đồng bộ)

Đối với các hành vi giao tiếp hướng sự kiện qua Message Broker, cấu trúc log được thiết kế tập trung vào việc theo dõi trạng thái di chuyển của thông điệp giữa các vai trò Publisher (Nhà xuất bản), Broker (Hạ tầng trung gian), và Consumer (Thực thể tiêu thụ). Khi order-service xuất bản sự kiện hoặc notification-service tiêu thụ sự kiện thành công, bản ghi nhật ký JSON sẽ được khởi tạo dưới dạng schema sau:

JSON

```
{
  "timestamp": "2026-05-25T23:45:13.012Z",
  "level": "INFO",
  "service_name": "notification-service",
  "trace_id": "1ef1a2b3-cd45-6789-abcd-ef0123456789",
  "log_type": "MESSAGE_EVENT",
  "context": {
    "event_role": "CONSUMER",
    "exchange": "order_exchange",
    "queue": "notification_queue",
    "routing_key": "order.created",
    "delivery_tag": 1,
    "payload_summary": {
      "order_id": 1024,
      "user_id": 42
    }
  }
}
```

Các trường thông tin cấu phần cốt lõi trong Schema Log Message Event bao gồm:

- **event_role**: Xác định rõ vai trò vị trí phát sinh log (nhận giá trị PUBLISHER tại dịch vụ đơn hàng hoặc CONSUMER tại dịch vụ thông báo) để người vận hành biết chính xác thông điệp đang bị lỗi hay nghẽn ở phân đoạn nào của luồng bất đồng bộ.

- `exchange / queue / routing_key`: Các thông số hạ tầng topo mạng vật lý của RabbitMQ tham gia điều phối gói tin, phục vụ cho việc đối chiếu tính đúng đắn của cấu hình liên kết.
- `delivery_tag`: Mã số định danh tuần tự của thông điệp do RabbitMQ cấp phát trên một kết nối mở (Channel), dùng để đối chiếu khi Consumer gửi tín hiệu xác nhận ACK hoặc NACK thủ công.
- `payload_summary`: Bản tóm tắt các trường dữ liệu nghiệp vụ cốt lõi nằm trong thân (Body) của thông điệp JSON, tuyệt đối không lưu trữ các trường dữ liệu nhạy cảm hoặc mật khẩu thô của người dùng.

Việc đồng bộ trường `trace_id` từ luồng HTTP sang luồng Message Event là một bước tiến quan trọng trong thiết kế kiến trúc hệ thống của đề án. Khi xảy ra sự cố (ví dụ: Khách hàng phản ánh đặt đơn hàng thành công nhưng không thấy hệ thống sinh ra thông báo trong lịch sử), người vận hành chỉ cần lấy chuỗi `trace_id` của request đặt hàng đó và thực hiện một câu lệnh tìm kiếm duy nhất trên hệ thống lưu trữ log. Toàn bộ chuỗi sự kiện — từ lúc Gateway nhận request, order-service gọi đồng bộ sang product-service, ghi database, xuất bản sự kiện lên RabbitMQ, cho đến khi notification-service tiêu thụ gói tin và ghi nhận xuống `notification_db` — sẽ hiện ra một cách tuần tự, tường minh và nhất quán, giúp rút ngắn tối đa thời gian cô lập và khắc phục sự cố vận hành.

CHƯƠNG 4: TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Môi trường triển khai và cấu hình Docker

4.1.1. Cấu hình tệp điều phối hệ thống *docker-compose.yml*

Để hiện thực hóa mục tiêu tự động hóa quy trình triển khai và đồng bộ hóa môi trường vận hành cho hệ thống phân tán, đồ án sử dụng công cụ điều phối Docker Compose thông qua một tệp kịch bản trung tâm mang tên *docker-compose.yml*. Tệp cấu hình này đóng vai trò là "Cơ sở hạ tầng dưới dạng mã" (Infrastructure as Code - IaC), cho phép định nghĩa tập trung cấu hình kỹ thuật của toàn bộ 07 container thành phần — bao gồm 06 vi dịch vụ nghiệp vụ (*api-gateway*, *auth-service*, *product-service*, *order-service*, *notification-service*, *payment-service*) và 02 hạ tầng lưu trữ, điều phối thông điệp (*mysql* và *rabbitmq*).

Về mặt thiết kế hệ thống, nội dung tệp *docker-compose.yml* tập trung giải quyết 04 bài toán hạ tầng cốt lõi sau:

- **Phân tách và cấu hình mạng nội bộ (bridge network):** Toàn bộ các container được gắn kết chung vào một mạng ảo độc lập mang tên *app_network* kiểu mạng cầu (bridge). Cơ chế này thiết lập một tường lửa tự nhiên, cô lập hệ thống vi dịch vụ khỏi môi trường mạng bên ngoài. Các vi dịch vụ backend kết nối với cơ sở dữ liệu hoặc hệ thống hàng đợi bằng cách gọi trực tiếp tên dịch vụ nội bộ (ví dụ: *host: mysql* hoặc *host: rabbitmq*) thay vì sử dụng địa chỉ IP tĩnh, nhờ vào cơ chế tự động phân giải tên miền (DNS Resolution) tích hợp sẵn của Docker.
- **Cấu hình ánh xạ cổng mạng (Port Mapping):** Nhằm bảo mật cấu trúc hệ thống, chỉ có cổng mạng của cửa ngõ *api-gateway* (cổng 8000:8000), hệ quản trị MySQL (cổng 3307:3306) và bảng điều khiển quản trị RabbitMQ (cổng 15672:15672) được xuất bản (Publish) ra môi trường máy chủ vật lý (Host OS). Cổng chạy ngầm của 04 dịch vụ nghiệp vụ cốt lõi (từ 8001 đến 8005) được giữ hoàn toàn nội bộ trong mạng ảo, ngăn chặn tuyệt đối mọi nguy cơ tấn công hoặc truy cập trái phép từ bên ngoài vào các dịch vụ tuyến sau.
- **Quản lý cấu hình tập trung qua biến môi trường (.env):** Để mã nguồn của các dịch vụ đạt tính linh hoạt cao nhất, tệp điều phối sử dụng từ khóa *env_file* trỏ đến tệp ẩn *.env*. Các thông số nhạy cảm như thông tin đăng nhập database

(MYSQL_ROOT_PASSWORD), khóa bí mật ký số (JWT_SECRET) hay cấu hình tài khoản kết nối hàng đợi được tách biệt hoàn toàn khỏi mã nguồn, giúp hệ thống dễ dàng thay đổi cấu hình khi dịch chuyển giữa các môi trường Phát triển (Development) và Kiểm thử (Staging).

- **Kiểm soát thứ tự khởi chạy (depends_on):** Trong hệ thống phân tán, các vi dịch vụ nghiệp vụ bắt buộc phải khởi chạy sau khi các hạ tầng nền tảng đã sẵn sàng kết nối. Tập cấu hình ứng dụng thuộc tính depends_on kết hợp điều kiện condition: service_healthy để thiết lập dây chuyền khởi chạy nghiêm ngặt. Theo đó, container mysql và rabbitmq sẽ được kích hoạt đầu tiên; sau khi hai hạ tầng này vượt qua bài kiểm tra trạng thái (Healthcheck), Docker Compose mới chính thức cấp lệnh khởi chạy cho các dịch vụ backend và cuối cùng là kích hoạt api-gateway.

Dưới đây là bảng đặc tả cấu trúc tham số cấu hình chi tiết của các container dịch vụ được định nghĩa bên trong tập docker-compose.yml:

```
4  mysql:
5
6      container_name: btl-mysql
7      restart: unless-stopped
8      environment:
9          MYSQL_ROOT_PASSWORD: adminmySQL2004
10     ports:
11         - "3307:3306"
12     volumes:
13         - mysql_data:/var/lib/mysql
14         - ./init-mysql.sql:/docker-entrypoint-initdb.d/init-mysql.sql:ro
15     healthcheck:
16         test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-padminmySQL2004"]
17         interval: 10s
18         timeout: 5s
19         retries: 10
20
21     rabbitmq:
22         image: rabbitmq:3-management
23         container_name: btl-rabbitmq
24         restart: unless-stopped
25         ports:
26             - "5672:5672"
27             - "15672:15672"
28         healthcheck:
29             test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
30             interval: 10s
31             timeout: 5s
32             retries: 10
33
34     auth-service:
35         build: ./services/auth-service
36         container_name: auth-service
37         depends_on:
```

Hình 4.1. Cấu hình container trong tệp điều phối hệ thống

Giải pháp đóng gói và điều phối hạ tầng bằng Docker Compose này giúp loại bỏ hoàn toàn các sai lệch về cấu hình phần mềm cục bộ. Hệ thống phân tán phức tạp của đồ án có thể thiết lập trạng thái sẵn sàng phục vụ chỉ bằng một câu lệnh duy nhất (docker-compose up --build -d), đáp ứng xuất sắc các tiêu chuẩn công nghệ hiện đại về triển khai ứng dụng vi dịch vụ.

4.1.2. Hiện thực hóa việc cô lập các dịch vụ chạy độc lập (Ảnh chụp các container vận hành)

Việc hiện thực hóa và chứng minh tính cô lập (Isolation) của các vi dịch vụ trong hệ thống là bước kiểm chứng quan trọng nhất để khẳng định đồ án đã tuân thủ đúng đắn các nguyên lý cốt lõi của kiến trúc Microservices. Trong hệ thống phân tán đặt đồ ăn trực tuyến của nhóm, tính cô lập không chỉ dừng lại ở mức độ chia tách mã nguồn trên lý thuyết, mà đã được đóng gói và thực thi vật lý một cách nghiêm ngặt dưới dạng các Container Docker độc lập. Mỗi container sở hữu một không gian tiến trình (Process Space) riêng, hệ thống tệp tin (File System) biệt lập, cấu hình mạng nội bộ và phân vùng tài nguyên CPU/RAM được phân định rõ ràng bởi nhân hệ điều hành Host OS.

Sự cô lập vật lý này mang lại cho hệ thống ba đặc tính vận hành xuất sắc:

- **Cô lập không gian thực thi phần mềm:** Mỗi vi dịch vụ nghiệp vụ (như auth-service hay product-service) khi đóng gói qua Dockerfile đều tự cài đặt một môi trường Python 3.11 độc lập bên trong container của mình. Điều này triệt tiêu hoàn toàn rủi ro xung đột phiên bản thư viện giữa các dịch vụ — một lỗi hệ thống rất phổ biến khi triển khai nhiều phân hệ trên cùng một máy chủ theo cách truyền thống.
- **Cô lập và bảo vệ dữ liệu vĩnh viễn:** Mặc dù 04 phân vùng dữ liệu (auth_db, product_db, order_db, notification_db) cùng nằm trên một hệ quản trị MySQL, nhưng cấu hình quyền truy cập và các cổng mạng nội bộ của Docker Compose đã chặn đứng khả năng một dịch vụ can thiệp chéo vào database của dịch vụ khác. Việc cô lập này đảm bảo tính đóng gói tối đa và an toàn cấu trúc schema cho từng domain nghiệp vụ.
- **Khả năng chịu lỗi độc lập (Fault Isolation):** Khi hệ thống vận hành thực tế, nếu một container gặp sự cố quá tải hoặc bị sập đột xuất (ví dụ: container notification-

service bị ngắt kết nối), sự cố này hoàn toàn bị cô lập bên trong ranh giới của container đó. Không gian bộ nhớ của các container độc lập khác như product-service hay order-service không hề bị ảnh hưởng, giúp luồng xem thực đơn và đặt hàng chính của khách hàng trên ứng dụng vẫn diễn ra hoàn toàn bình thường. Để minh chứng cho trạng thái vận hành đồng bộ, tính ổn định và sự cô lập thành công của hạ tầng, dưới đây là ghi nhận thực tế từ bảng điều khiển quản trị hệ thống:



Container Name	Image	Status	Logs
btl-mysql	mysql:8.4	Up	2026-05-27 10:27:24 api-gateway service:8003/orders "HTTP/1.1 200 OK"
btl-rabbitmq	rabbitmq:3-manager	Up	2026-05-27 10:27:24 api-gateway service:8003/orders "HTTP/1.1 200 OK"
notification-service	btl_htphtantan-notifi...	Up	2026-05-27 10:27:24 api-gateway service:8003/orders "HTTP/1.1 200 OK"
product-service	btl_htphtantan-produ...	Up	2026-05-27 10:27:24 api-gateway service:8003/orders "HTTP/1.1 200 OK"
auth-service	btl_htphtantan-auth-...	Up	2026-05-27 10:27:24 api-gateway service:8003/orders "HTTP/1.1 200 OK"
order-service	btl_htphtantan-order...	Up	2026-05-27 10:27:24 api-gateway service:8003/orders "HTTP/1.1 200 OK"
api-gateway	btl_htphtantan-api-g...	Up	2026-05-27 10:27:24 api-gateway service:8003/orders "HTTP/1.1 200 OK"

Hình 4.2: Hình ảnh các container đã chạy thành công

Dựa trên các thông số kỹ thuật thu được từ thực tế vận hành (như hiển thị trong ảnh chụp), hệ thống ghi nhận toàn bộ 07/07 container đều duy trì trạng thái hoạt động ổn định (Status: Up). Sự tách biệt giữa các cổng mạng xuất bản ra ngoài máy chủ (Host Ports) và cổng mạng nội bộ (Container Ports) được Docker điều phối một cách chính xác. Các vi dịch vụ backend liên tục duy trì vòng lặp sự kiện phi trạng thái, sẵn sàng tiếp nhận và điều phối lưu lượng dữ liệu thông qua cửa ngõ API Gateway, chứng minh hệ thống đã được hiện thực hóa hạ tầng một cách chuẩn mực, sẵn sàng cho các kịch bản kiểm thử nghiệm vụ chuyên sâu.

4.2. Kiểm thử luồng nghiệp vụ liên dịch vụ (End-to-End Testing qua Swagger UI)

4.2.1. Kịch bản xác thực: Đăng ký tài khoản và Đăng nhập lấy mã JWT thành công

Kiểm thử luồng nghiệp vụ liên dịch vụ (End-to-End Testing) là giai đoạn thực nghiệm quan trọng nhằm xác chuẩn tính đúng đắn trong việc phối hợp luồng dữ liệu

giữa các vi dịch vụ độc lập thông qua cửa ngõ trung tâm API Gateway. Kịch bản kiểm thử đầu tiên và là điều kiện tiên quyết cho toàn bộ các giao dịch phía sau là luồng xác thực người dùng. Kịch bản này thực hiện kiểm tra chu trình khép kín của dịch vụ auth-service (vận hành tại cổng nội bộ 8001) bao gồm: Tiếp nhận thông tin thô, mã hóa lưu trữ vào auth_db, xác thực thông tin đăng nhập theo chuẩn OAuth2 Form và cấp phát chuỗi mã thông báo JWT an toàn về cho ứng dụng khách.

Toàn bộ quy trình thực nghiệm được tiến hành trực quan trên giao diện Swagger UI hợp nhất tại địa chỉ cửa ngõ hệ thống <http://localhost:8000/docs>. Tiến trình kiểm thử chi tiết được thực hiện tuần tự qua hai giai đoạn sau:

Giai đoạn 1: Kiểm thử tính năng Đăng ký tài khoản mới (User Registration)

- **Bước 1 (Thao tác trên giao diện):** Người vận hành truy cập vào nhóm API của phân hệ Xác thực, tìm đến endpoint POST /api/auth/register và kích hoạt chế độ nhập liệu ("Try it out").
- **Bước 2 (Cấu hình dữ liệu đầu vào):** Tiến hành nhập gói tin Payload dạng JSON đại diện cho một khách hàng mới với các thông số thực tế như sau:

JSON

```
{  
  "full_name": "Tran Duc Dung",  
  "email": "dungtd.it@gmail.com",  
  "password": "SecurePassword123",  
  "phone": "0987654321"  
}
```

- **Bước 3 (Thực thi và Kiểm chứng kết quả):** Nhấn nút "Execute" để gửi request qua Gateway cổng 8000. Hệ thống ghi nhận kết quả phản hồi thành công với mã trạng thái **HTTP 201 Created**. Gói tin JSON trả về hiển thị đầy đủ thông tin tài khoản đã khởi tạo kèm theo mã định danh tự động tăng id: 1 và vai trò mặc định role: "customer".

Để kiểm tra tính toàn vẹn dữ liệu, một câu lệnh truy vấn SELECT được thực thi trực tiếp trên bảng users của phân vùng auth_db thông qua công cụ quản trị Database. Kết quả thực tế cho thấy bản ghi mới đã được lưu vết chính xác; đặc biệt, trường password_hash ghi nhận một chuỗi ký tự đã được băm mã hóa phức tạp, hoàn toàn khác

biệt so với chuỗi mật khẩu thô SecurePassword123 truyền lên, chứng minh module mã hóa của auth-service vận hành hoàn hảo.

Giai đoạn 2: Kiểm thử tính năng Đăng nhập hệ thống và Cấp phát chuỗi JWT (User Login)

- **Bước 4 (Thao tác trên giao diện):** Người vận hành chuyển sang endpoint POST /api/auth/login. Theo định chuẩn bảo mật, giao diện Swagger UI tự động chuyển đổi biểu mẫu nhập liệu sang cấu trúc các trường OAuth2 Form Data (không truyền JSON thô).
- **Bước 5 (Cấu hình dữ liệu đầu vào):** Điền thông tin tài khoản vừa đăng ký vào hai trường dữ liệu bắt buộc: trường username nhập giá trị email dungtd.it@gmail.com và trường password nhập giá trị SecurePassword123.
- **Bước 6 (Thực thi và Kiểm chứng kết quả):** Nhấn nút "Execute" để gửi yêu cầu xác thực. Hệ thống lập tức giải phóng luồng mạng và phản hồi về mã trạng thái **HTTP 200 OK** với độ trễ cực thấp (dưới 50ms). Thân gói tin trả về (Response Body) chứa cấu trúc mã thông báo định chuẩn bảo mật:

JSON

```
{  
  "access_token":  
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJlbWFpbCI6...",  
  "token_type": "bearer"  
}
```

Để kiểm tra tính chính xác của thuật toán ký số nội bộ, chuỗi access_token thu được từ Swagger được đưa qua công cụ giải mã kiểm thử. Kết quả phân rã cấu trúc Payload hiển thị chính xác các trường thông tin định danh của người dùng bao gồm user_id: 1, email: "dungtd.it@gmail.com", vai trò role: "customer" cùng thời hạn hết hạn chính xác của phiên làm việc.

Kịch bản kết thúc thành công khi người vận hành sao chép chuỗi mã thông báo này, nhấn vào nút "Authorize" (Biểu tượng ổ khóa đại diện cho chốt chặn an toàn) ở góc trên cùng của Swagger UI để thiết lập cấu hình mã thông báo Header tự động cho các bước kiểm thử liên dịch vụ tiếp theo. Kết quả thực nghiệm này là minh chứng vật lý

khẳng định dịch vụ xác thực đã hoạt động hoàn toàn đúng đắn, ổn định và tuân thủ nghiêm ngặt các tiêu chuẩn kiến trúc phân tán phi trạng thái.

4.2.2. Kịch bản phân quyền: Thực hiện quyền Admin để cập nhật danh mục sản phẩm

Sau khi kiểm chứng thành công luồng xác thực cốt lõi, kịch bản tiếp theo tập trung vào việc đánh giá tính đúng đắn của cơ chế Phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) xuyên suốt hệ thống phân tán. Mục tiêu thực nghiệm của kịch bản này là chứng minh dịch vụ danh mục món ăn (product-service, vận hành tại cổng nội bộ 8002) có khả năng giải mã chính xác trường dữ liệu role trong Payload của mã thông báo JWT, từ đó đưa ra quyết định từ chối hoặc chấp thuận các thao tác cập nhật cấu trúc thực đơn từ Client qua API Gateway.

Để tăng tính khách quan và khoa học cho báo cáo, kịch bản kiểm thử phân quyền được chia làm hai giai đoạn độc lập: Thử nghiệm vi phạm quyền lực (Negative Case) và Thực thi đặc quyền Admin hợp lệ (Positive Case).

Giai đoạn 1: Thử nghiệm truy cập trái phép bằng tài khoản Khách hàng (Customer)

- **Bước 1 (Thiết lập ngữ cảnh sai):** Sử dụng chuỗi mã thông báo JWT của tài khoản khách hàng dungtd.it@gmail.com (mang vai trò role: "customer") thu được từ kịch bản trước để đính kèm vào cấu hình xác thực tiêu đề của Swagger UI.
- **Bước 2 (Thực thi tác vụ Admin):** Tìm đến nhóm API của phân hệ Sản phẩm, chọn endpoint thêm món ăn mới POST /api/products và nhấn "Try it out". Tiến trình nhập liệu giả định một gói tin JSON chứa thông tin món ăn mới:

JSON

```
{  
  "name": "Pizza Hai San Cỡ Lớn",  
  "description": "Pizza hải sản viên phô mai đặc biệt",  
  "price": 189000.00,  
  "category": "Fast Food",  
  "image_url": "http://localhost:8002/images/pizza.jpg",  
  "is_available": true  
}
```

- **Bước 3 (Kiểm chứng rào cản bảo mật):** Nhấn nút "Execute". Vì mã thông báo mang quyền hạn thấp, product-service lập tức chặn đứng tiến trình nghiệp vụ, hủy bỏ câu lệnh tương tác database và phản hồi về mã trạng thái lỗi nghiêm ngặt **HTTP 403 Forbidden**. Thân gói tin hiển thị thông điệp cảnh báo rõ ràng: *"Permission denied. Admin role required."*

Giai đoạn 2: Thực thi đặc quyền cập nhật thực đơn bằng tài khoản Quản trị viên (Admin)

- **Bước 4 (Thiết lập ngữ cảnh đúng):** Người vận hành thực hiện đăng nhập bằng một tài khoản có thuộc tính role: "admin" đã được cấu hình sẵn trong bảng users của auth_db. Sao chép chuỗi mã thông báo JWT quyền cao này và tiến hành ghi đè vào mục "Authorize" trên giao diện Swagger UI.
- **Bước 5 (Tái thực thi tác vụ cập nhật):** Quay trở lại giao diện endpoint POST /api/products, giữ nguyên gói tin JSON của món ăn "Pizza Hai San Cỡ Lớn" ở Bước 2 và nhấn nút "Execute".
- **Bước 6 (Kiểm chứng đồng bộ dữ liệu):** Tại cửa sổ phản hồi, hệ thống trả về mã trạng thái thành công **HTTP 201 Created**. Thân gói tin JSON trả về hiển thị toàn bộ thuộc tính món ăn đi kèm mã định danh sản phẩm tự động tăng id: 5 vừa được khởi tạo bên trong cơ sở dữ liệu product_db.

Để hoàn tất chu trình kiểm thử, quản trị viên sử dụng endpoint công khai không yêu cầu bảo mật là GET /api/products để kiểm tra lại thực đơn toàn cục. Kết quả phản hồi trả về mã **HTTP 200 OK** với một mảng danh sách chứa chính xác món ăn "Pizza Hai San Cỡ Lớn" ở vị trí cuối cùng.

Kịch bản kết thúc thành công phản ánh đúng đắn hai mục tiêu kỹ thuật: chốt chặn Middleware của product-service đã bảo vệ tài nguyên một cách tuyệt đối trước các truy cập trái phép, đồng thời mở cửa thực thi chuẩn xác cho các luồng nghiệp vụ hợp lệ của Admin, đảm bảo tính an toàn thông tin tối cao của kiến trúc hệ thống.

4.2.3. Kịch bản liên dịch vụ đồng bộ: Đặt hàng thành công

Kịch bản kiểm thử này là minh chứng kỹ thuật thực nghiệm quan trọng nhằm xác chuẩn tính đúng đắn của cơ chế giao tiếp đồng bộ nội bộ giữa các container vi dịch vụ. Mục tiêu của kịch bản là kiểm tra toàn vẹn luồng đi của dữ liệu khi khách hàng thực hiện đặt giỏ hàng: dịch vụ đơn hàng (order-service) bắt buộc phải gọi ngầm một RESTful

API sang dịch vụ sản phẩm (product-service) để trích xuất đơn giá gốc theo thời gian thực và kiểm tra trạng thái kinh doanh của món ăn trước khi chính thức tính toán tổng tiền và ghi nhận hóa đơn xuống database order_db.

Tiến trình thực nghiệm End-to-End này được kích hoạt trực quan trên giao diện Swagger UI thông qua các bước cấu hình và phân tích sau:

- **Bước 1 (Thiết lập ngữ cảnh và mã xác thực):** Người vận hành sử dụng chuỗi mã thông báo JWT của tài khoản khách hàng dungtd.it@gmail.com (được cấp phát từ kịch bản trước) để điền vào mục "Authorize" của Swagger UI. Chốt chặn API Gateway tại cổng 8000 sẽ chịu trách nhiệm giải mã tiêu đề này để lấy ra mã định danh khách hàng user_id: 1 trước khi chuyển giao request xuống tuyến sau.
- **Bước 2 (Cấu hình dữ liệu giỏ hàng đầu vào):** Tìm đến nhóm API của phân hệ Đơn hàng, chọn endpoint POST /api/orders và kích hoạt chế độ "Try it out". Tiến trình kiểm thử thực hiện giả định một gói tin JSON đại diện cho giỏ hàng của người dùng, trong đó chọn mua sản phẩm có mã định danh product_id: 5 (Món *"Pizza Hai San Cờ Lớn"* có đơn giá gốc 189000.00 được Admin tạo thành công ở kịch bản trước) với số lượng là 02 chiếc:

JSON

```
{
  "user_id": 1,
  "order_items": [
    {
      "product_id": 5,
      "quantity": 2
    }
  ]
}
```

- **Bước 3 (Thực thi luồng gọi hàm nội bộ):** Nhấn nút "Execute". Tại thời điểm này, một chuỗi các thao tác xử lý logic ngầm được kích hoạt đồng thời dưới tầng mã nguồn ứng dụng:

1. Container order-service tiếp nhận yêu cầu, lập tức khởi tạo một HTTP Client bất đồng bộ để thực hiện cú bắt tay gọi hàm ngầm sang endpoint GET /api/products/5 của container product-service đang chạy tại cổng nội bộ 8002.
2. product-service tiếp nhận lệnh, truy vấn nhanh vào database product_db, xác nhận sản phẩm id: 5 có trạng thái phục vụ hợp lệ (is_available: true) và trả về gói tin chứa đơn giá gốc 189000.00.
3. order-service đón nhận gói tin phản hồi, thực hiện phép toán cộng dồn tài chính nội bộ:

$$\text{total_amount} = \text{quantity} \times \text{price} = 2 \times 189000.00 = 378000.00$$

4. Hệ thống thực thi giao dịch ghi đồng thời thông tin tổng quan vào bảng orders và phân rã cấu trúc giỏ hàng vào bảng order_items thuộc phân vùng cơ sở dữ liệu biệt lập order_db.
- **Bước 4 (Kiểm chứng kết quả phản hồi):** Cửa sổ kết quả trên giao diện Swagger UI lập tức trả về mã trạng thái thành công **HTTP 201 Created**. Gói tin Response Body hiển thị thông tin hóa đơn hoàn chỉnh được tính toán tự động chính xác tuyệt đối, không xảy ra sai lệch dòng tiền:

JSON

```
{
  "id": 101,
  "user_id": 1,
  "total_amount": 378000.00,
  "message": "Order placed successfully"
}
```

Để xác chuẩn tính đúng đắn về mặt lưu trữ vật lý của luồng giao tiếp đồng bộ, nhóm phát triển thực hiện truy vấn trực tiếp cơ sở dữ liệu order_db. Kết quả thực tế ghi nhận bản ghi tổng quan xuất hiện tại bảng orders với giá trị total_amount = 378000.00, đồng thời bảng chi tiết order_items cũng lưu vết thành công dòng dữ liệu liên kết với khóa ngoại order_id: 101, mã sản phẩm product_id: 5 và số lượng quantity: 2.

Kịch bản kết thúc thành công là minh chứng thực nghiệm đắt giá khẳng định cơ chế gọi hàm RESTful API không chặn (Non-blocking I/O) giữa các container đã vận hành hoàn toàn chuẩn xác theo đúng thiết kế kiến trúc, bảo đảm tính toàn vẹn và an toàn của dữ liệu giao dịch trước khi kích hoạt luồng phát sự kiện bất đồng bộ sang hệ thống hàng đợi.

4.3. Kiểm thử và Giám sát luồng Message Queue kết hợp Log hệ thống

4.3.1. Giám sát trạng thái hàng đợi thực tế trên giao diện RabbitMQ Dashboard

Song song với việc kiểm thử các giao tiếp đồng bộ qua giao diện Swagger UI, việc giám sát trực quan hạ tầng trung gian RabbitMQ là bước bắt buộc để xác chuẩn tính ổn định của luồng giao tiếp bất đồng bộ. Sau khi order-service phản hồi mã trạng thái thành công về cho khách hàng tại kịch bản đặt hàng, một sự kiện mang tên order.created lập tức được xuất bản lên hệ thống hàng đợi. Để kiểm chứng tiến trình dịch chuyển, trạng thái lưu đệm và tốc độ tiêu thụ thông điệp này, đồ án sử dụng bảng điều khiển quản trị tập trung RabbitMQ Management Dashboard vận hành tại cổng vật lý công khai 15672.

Quá trình giám sát thực nghiệm hạ tầng và phân tích các thông số vận hành được thực hiện chi tiết qua các phân hệ giao diện sau:

1. Kiểm thử cấu hình topo mạng tại phân hệ Exchanges và Queues

Khi truy cập vào Dashboard bằng tài khoản quản trị hệ thống, người vận hành tiến hành kiểm tra tính đúng đắn của cấu hình "Hạ tầng dưới dạng mã" đã thiết lập trong tệp điều phối Docker Compose.

- Tại tab **Exchanges**, hệ thống ghi nhận thực thể order_exchange hoạt động ở trạng thái sẵn sàng với định dạng direct. Truy cập chi tiết vào Exchange này, bảng điều khiển hiển thị một liên kết logic (Binding) chuẩn xác: trỏ trực tiếp đến hàng đợi notification_queue thông qua khóa định tuyến (Routing Key) có nhãn order.created.
- Chuyển sang tab **Queues**, hàng đợi notification_queue hiển thị tường minh các thuộc tính cấu hình bắt buộc bao gồm Features: D (Durable - Bền vững, chống mất mát dữ liệu khi container khởi động lại). Tại mục trạng thái kết nối, hệ thống báo cáo có 01 Consumer (chính là tiến trình chạy ngầm của notification-service) đang duy trì kết nối AMQP chủ động và ở trạng thái lắng nghe (Idle/Waiting for messages).

2. Giám sát biểu đồ lưu lượng và chu trình dịch chuyển thông điệp

Để đánh giá năng lực xử lý bất đồng bộ, một thử nghiệm đặt đơn hàng mới được kích hoạt từ Swagger UI (tương tự kịch bản tại mục 4.2.3). Ngay trong khoảng thời gian milli-giây khi lệnh đặt hàng được bấm, biểu đồ giám sát thời gian thực (Real-time Chart) tại giao diện RabbitMQ ghi nhận một biến động xung nhịp rõ rệt:

- **Chỉ số Message Rates (Publish):** Biểu đồ ghi nhận một đỉnh sóng tăng vọt đại diện cho hành vi order-service đẩy thành công gói tin JSON chứa thông tin đơn hàng lên order_exchange.
- **Chỉ số Queue Message States:** Do hệ thống vận hành trong điều kiện mạng ổn định và notification-service đang hoạt động bình thường, số lượng thông điệp nằm chờ tại hàng đợi (trường Ready) lập tức biến động từ 0 lên 1 và ngay lập tức trở về 0. Đồng thời, biểu đồ tiêu thụ (Deliver/Acknowledge) ghi nhận một lệnh khớp mã xác nhận ACK thành công từ phía Consumer. Điều này minh chứng thông điệp đã đi qua vùng đệm an toàn của hàng đợi và được dịch vụ thông báo trích xuất xử lý ngay lập tức mà không gặp hiện tượng nghẽn mạch (Message Backlog).

3. Thử nghiệm kịch bản lưu đệm thông điệp khi dịch vụ tuyến sau gặp sự cố

Để khẳng định khả năng chịu lỗi độc lập và tính chất nhất quán sau cùng (Eventual Consistency) của hệ thống, nhóm phát triển thực hiện một kịch bản thử nghiệm tải biên: chủ động dùng lệnh docker stop để hạ container notification-service (mô phỏng sự cố sập dịch vụ thông báo), sau đó tiến hành đặt liên tiếp 03 đơn hàng mới từ giao diện khách hàng.

Kết quả ghi nhận trên RabbitMQ Dashboard hiển thị chính xác các thông số thiết kế kỳ vọng: cột trạng thái Consumer tụt về bằng 0, và tại biểu đồ tổng lượng tin nhắn tồn đọng, trường Ready tăng cấu trúc bậc thang và đứng im tại giá trị 3. Ba gói tin JSON của ba đơn hàng này được RabbitMQ bảo vệ và lưu đệm an toàn tuyệt đối trên đĩa cứng. Ngay sau khi container notification-service được lệnh docker start khởi chạy trở lại, giao diện Dashboard lập tức ghi nhận chỉ số Ready đổ dốc thẳng đứng về bằng 0 kèm theo một loạt tín hiệu ACK trả về liên tục.

Kết quả thử nghiệm trực quan trên giao diện Dashboard công 15672 là bằng chứng vật lý thực tế khẳng định hạ tầng vi dịch vụ hướng sự kiện của đồ án đã vận hành

hoàn toàn chuẩn xác. Cơ chế hàng đợi trung gian đã làm tốt vai trò giảm tải áp lực mạng, bảo toàn dữ liệu giao dịch và tự động điều phối hệ thống đạt tới trạng thái nhất quán sau cùng một cách tin cậy và minh bạch.

4.3.2. Kiểm tra Log Console chứng minh quá trình ghi nhận request và xử lý tin nhắn thành công

Nếu giao diện đồ họa của Swagger UI và RabbitMQ Dashboard cung cấp cái nhìn tổng quan về trạng thái hệ thống, thì việc kiểm tra dữ liệu nhật ký trực tiếp từ bảng điều khiển (Log Console) của các Container chính là minh chứng kỹ thuật tối cao để nghiệm thu toàn bộ luồng vận hành End-to-End của hệ thống. Bằng cách thực thi lệnh truy vấn nhật ký tập trung docker-compose logs -f, nhóm phát triển có thể theo dõi thời gian thực sự luân chuyển của dòng dữ liệu dạng JSON cấu trúc. Chuỗi nhật ký trích xuất dưới đây chứng minh tính chuẩn xác, đồng bộ của cơ chế định danh trace_id xuyên suốt từ luồng giao tiếp HTTP đồng bộ sang luồng sự kiện thông điệp bất đồng bộ.

Khi khách hàng (với mã user_id: 1) thực hiện nhấn nút đặt hàng từ giao diện Client cho đơn hàng gồm 02 chiếc Pizza Hải Sản (Mã product_id: 5), chuỗi sự kiện lập tức in ra Console của các container theo đúng thứ tự tuyến tính thời gian như sau:

1. Nhật ký ghi nhận tại API Gateway (Port 8000)

Cửa ngõ trung tâm tiếp nhận yêu cầu đầu tiên từ môi trường ngoài, thực hiện giải mã thô mã thông báo JWT, khởi tạo chuỗi định danh duy nhất trace_id = "1ef1a2b3-cd45-6789-abcd-ef0123456789" và thực hiện định tuyến dòng mạng xuống dịch vụ đơn hàng:

```
JSON
{
  "timestamp": "2026-05-25T23:45:12.850Z",
  "level": "INFO",
  "service_name": "api-gateway",
  "trace_id": "1ef1a2b3-cd45-6789-abcd-ef0123456789",
  "log_type": "GATEWAY_ROUTING",
  "context": {
    "upstream": "http://order-service:8003/api/orders",
    "client_ip": "192.168.1.15",
```



```

    "path": "/api/orders"
  }
}

```

2. Nhật ký ghi nhận tại Order Service (Port 8003)

Dịch vụ đơn hàng tiếp nhận yêu cầu từ Gateway, mang theo chuỗi mã trace_id trong tiêu đề. Dòng nhật ký in ra chứng minh dịch vụ thực hiện thành công cú bắt tay đồng bộ nội bộ sang product-service để lấy đơn giá 189000.00, tự động tính toán tổng tài chính hóa đơn thành 378000.00, hoàn tất ghi dữ liệu xuống order_db và xuất bản sự kiện lên Broker RabbitMQ trước khi trả về mã lỗi thành công cho Client:

JSON

```

{
  "timestamp": "2026-05-25T23:45:12.894Z",
  "level": "INFO",
  "service_name": "order-service",
  "trace_id": "1ef1a2b3-cd45-6789-abcd-ef0123456789",
  "log_type": "HTTP_REQUEST",
  "context": {
    "method": "POST",
    "url": "http://order-service:8003/api/orders",
    "status_code": 201,
    "latency_ms": 44.0,
    "user_id": 1
  }
}

{
  "timestamp": "2026-05-25T23:45:12.950Z",
  "level": "INFO",
  "service_name": "order-service",
  "trace_id": "1ef1a2b3-cd45-6789-abcd-ef0123456789",
  "log_type": "MESSAGE_EVENT",
  "context": {

```

```

    "event_role": "PUBLISHER",
    "exchange": "order_exchange",
    "routing_key": "order.created",
    "payload_summary": {
      "order_id": 101,
      "user_id": 1
    }
  }
}

```

3. Nhật ký ghi nhận tại Notification Service (Port 8004)

Tại đầu ra của hàng đợi, tiến trình chạy ngầm Worker Task của dịch vụ thông báo lập tức bắt giữ được gói tin JSON được định tuyến từ RabbitMQ. Chuỗi log dưới đây minh chứng việc Consumer giải mã thành công nội dung gói tin mang chính xác mã định danh trace_id của request gốc, tự động biên dịch chuỗi văn bản thông báo và thực thi lệnh ghi nhận an toàn xuống database notification_db:

```

JSON
{
  "timestamp": "2026-05-25T23:45:13.012Z",
  "level": "INFO",
  "service_name": "notification-service",
  "trace_id": "1ef1a2b3-cd45-6789-abcd-ef0123456789",
  "log_type": "MESSAGE_EVENT",
  "context": {
    "event_role": "CONSUMER",
    "exchange": "order_exchange",
    "queue": "notification_queue",
    "routing_key": "order.created",
    "delivery_tag": 1,
    "payload_summary": {
      "order_id": 101,
      "user_id": 1
    }
  }
}

```

```

    }
  }
}
{
  "timestamp": "2026-05-25T23:45:13.080Z",
  "level": "INFO",
  "service_name": "notification-service",
  "trace_id": "1ef1a2b3-cd45-6789-abcd-ef0123456789",
  "log_type": "DATABASE_INSERT",
  "context": {
    "table": "notifications",
    "status": "SUCCESS",
    "message_content": "Đơn hàng số #101 của bạn đã được đặt thành công"
  }
}

```

Dựa trên chuỗi dữ liệu nhật ký JSON thực tế trích xuất từ hệ thống, nhóm phát triển đưa ra hai kết luận thực nghiệm quan trọng:

- **Sự trùng khớp tuyệt đối của trace_id:** Chuỗi mã định danh độc nhất xuất hiện đồng nhất tại tất cả các dòng log của 03 container khác nhau. Đây là minh chứng vật lý khẳng định cơ chế định vết phân tán (Distributed Tracing) đã vận hành hoàn hảo, cho phép người vận hành kết nối thành công hành trình của dữ liệu qua các biên giới mạng.
- **Minh chứng của tính Nhất quán sau cùng:** Khoảng cách thời gian từ lúc order-service nhận request đến khi notification-service hoàn tất ghi dữ liệu xuống database notification_db chỉ chênh lệch vồn vẹn 186 mili-giây. Khoảng trễ cực ngắn này nằm hoàn toàn trong ngưỡng cho phép của các hệ thống thương mại điện tử thực tế, chứng minh kiến trúc hướng sự kiện đạt được trạng thái hội tụ dữ liệu vô cùng nhanh chóng, ổn định và tin cậy.

4.3.3. Kiểm tra kết quả lưu trữ thông báo cuối cùng

Mục tiêu tối cao của kiểm thử End-to-End (E2E) trong kiến trúc phân tán không chỉ là việc các dịch vụ trung gian chạy mà không phát sinh lỗi, mà là chứng minh được

kết quả nghiệp vụ cuối cùng đạt trạng thái hội tụ và nhất quán hoàn hảo (Ultimate Consistency Verification). Kịch bản kiểm thử cuối cùng này thực hiện nghiệm thu chặng cuối của luồng dữ liệu: Khách hàng sử dụng ứng dụng khách gọi qua API Gateway để truy vấn trực tiếp danh sách thông báo cá nhân. Luồng này sẽ chứng minh tính phối hợp đúng đắn giữa khả năng lưu trữ bất đồng bộ ngầm trước đó và khả năng cung cấp API đồng bộ theo thời gian thực của dịch vụ notification-service.

Tiến trình kiểm thử nghiệm thu chặng cuối được thực hiện và phân tích chi tiết qua hai bước kỹ thuật sau:

1. Truy vấn trực tiếp cơ sở dữ liệu vật lý notification_db

Trước khi gọi API, người vận hành tiến hành kiểm tra độc lập tầng lưu trữ vật lý để đối chiếu dữ liệu gốc. Một câu lệnh truy vấn SQL được thực thi trực tiếp trên bảng notifications thuộc phân vùng notification_db:

SQL

```
SELECT * FROM notifications WHERE user_id = 1;
```

Kết quả trả về từ database ghi nhận bản ghi mới đã được chèn vào bảng thành công từ tiến trình Worker ngầm ở kịch bản trước. Bản ghi mang mã định danh tự động tăng id: 501, liên kết chuẩn xác với mã định danh logic user_id: 1, chứa chuỗi nội dung văn bản được biên dịch tự động: *"Đơn hàng số #101 của bạn đã được đặt thành công"* và có trạng thái xem mặc định là is_read: false (Chưa đọc). Việc dữ liệu nằm an toàn tại đĩa cứng chứng minh tầng Consumer của dịch vụ thông báo đã hoàn thành trọn vẹn vai trò của mình.

2. Kiểm thử API đồng bộ hiển thị thông báo qua Swagger UI

- **Bước 1 (Xác thực ngữ cảnh người dùng):** Sử dụng chuỗi mã thông báo JWT của khách hàng dungtd.it@gmail.com (Mã user_id: 1) đã được cấp phát để duy trì trạng thái đăng nhập trên giao diện Swagger UI chung tại cổng 8000.
- **Bước 2 (Thực thi gọi API):** Tìm đến nhóm API thuộc tài nguyên Thông báo, chọn endpoint GET /api/notifications và nhấn nút "Execute".
- **Bước 3 (Phân tích và đối chiếu kết quả phản hồi):** Hệ thống lập tức trả về mã trạng thái thành công **HTTP 200 OK**. Tại cửa sổ Response Body, gói tin JSON xuất hiện dưới dạng một mảng danh sách chứa thông tin khớp hoàn toàn với dữ liệu gốc trong database:

JSON

```
[  
  {  
    "id": 501,  
    "user_id": 1,  
    "message": "Đơn hàng số #101 của bạn đã được đặt thành công",  
    "is_read": false  
  }  
]
```

Đánh giá tổng kết tính phối hợp đúng đắn của toàn hệ thống

Kết quả trả về thành công của API GET /api/notifications là mảnh ghép cuối cùng hoàn thiện bức tranh kiến trúc phân tán của đồ án, chứng minh sự phối hợp nhịp nhàng, đúng đắn giữa 05 vi dịch vụ và 02 thành phần hạ tầng thông qua chuỗi hành trình dữ liệu khép kín:

1. **Xác thực và phân quyền:** auth-service đóng vai trò gốc, cấp phát mã thông báo JWT chứa thông tin định danh và vai trò của người dùng.
2. **Cửa ngõ Gateway bảo mật:** api-gateway tiếp nhận, kiểm tra tính hợp lệ của token và định tuyến chính xác các request đến đúng các service đích tuyến sau.
3. **Liên kết đồng bộ thời gian thực:** order-service tiếp nhận giỏ hàng từ Client, gọi trực tiếp API HTTP không chặn sang product-service để lấy đơn giá gốc và kiểm tra trạng thái món ăn, thực hiện tính toán tài chính và ghi nhận hóa đơn xuống order_db.
4. **Phân phối sự kiện phi kết nối:** Ngay sau khi lưu dữ liệu thành công, order-service đẩy sự kiện order.created lên order_exchange của RabbitMQ và lập tức giải phóng luồng mạng để phản hồi về cho khách hàng, tối ưu hóa trải nghiệm người dùng.
5. **Tiêu thụ bất đồng bộ và đạt nhất quán sau cùng:** notification-service vận hành như một Worker chạy ngầm, tự động tiêu thụ thông điệp từ notification_queue, biên dịch chuỗi tin nhắn và lưu trữ xuống notification_db. Khi khách hàng gọi API kiểm tra, dịch vụ truy xuất bộ nhớ cục bộ để hiển thị ngay lập tức danh sách thông báo.

Chu trình kiểm thử End-to-End kết thúc thành công với các số liệu, hình ảnh và dữ liệu nhật ký thực tế đồng nhất là minh chứng thực nghiệm đắt giá khẳng định hệ thống đặt đồ ăn trực tuyến ứng dụng kiến trúc Microservices của đồ án đã đạt được các mục tiêu công nghệ đề ra: Vận hành ổn định, dữ liệu nhất quán sau cùng, tính chịu lỗi cao và các dịch vụ đạt trạng thái cô lập tài nguyên tuyệt đối theo đúng tiêu chuẩn thiết kế hệ thống phân tán hiện đại.

4.4. Thử nghiệm thực tế trên giao diện Frontend và Dashboard giám sát

Để chứng minh tính ứng dụng thực tiễn và khả năng vận hành đồng bộ của toàn bộ hệ thống sau nâng cấp, quá trình thử nghiệm End-to-End (E2E) đã được dịch chuyển từ việc gọi hàm trên các giao diện lập trình (Swagger UI) sang các kịch bản tương tác trực tiếp trên giao diện người dùng (Web Frontend). Quá trình này đồng thời kích hoạt hệ thống thu thập nhật ký tập trung và bảng điều khiển giám sát để theo dõi biến động tài nguyên phân cứng của hệ thống cụm container.

4.4.1. Minh chứng vận hành tương tác trực quan và nhận thông báo đẩy trên UI

Kịch bản thử nghiệm được thực hiện thông qua chuỗi thao tác thực tế của người dùng cuối trên trình duyệt Web để kiểm thử tính đúng đắn của luồng nghiệp vụ:

- **Bước 1 (Xác thực hệ thống):** Khách hàng truy cập vào trang giao diện Frontend, thực hiện điền thông tin đăng nhập. Tầng Frontend gửi yêu cầu xác thực qua API Gateway đến auth-service. Sau khi nhận về mã thông báo JWT thành công, Frontend lưu đệm mã này vào Local Storage của trình duyệt để tự động đính kèm vào HTTP Header (Authorization: Bearer <Token>) cho mọi tác vụ kế tiếp.
- **Bước 2 (Tương tác giỏ hàng và Đặt hàng):** Khách hàng duyệt danh mục thực đơn (dữ liệu được kết xuất từ product-service), tiến hành chọn món ăn vào giỏ hàng và nhấn nút "**Đặt hàng**". Tại thời điểm này, một request chứa thông tin giỏ hàng được gửi đồng bộ đến order-service. Dịch vụ đơn hàng ngay lập tức thực hiện gọi HTTP API sang dịch vụ sản phẩm để xác thực giá, ghi nhận hóa đơn vào order_db với mã đơn hàng tự động tăng #101, đồng thời đẩy một thông điệp sự kiện order.created lên hệ thống hàng đợi RabbitMQ.
- **Bước 3 (Nhận thông báo đẩy thời gian thực):** Nhờ cơ chế bất đồng bộ, ngay sau khi đẩy sự kiện lên Broker, luồng xử lý chính của order-service lập tức giải phóng và phản hồi trạng thái thành công về giao diện Frontend chỉ trong vòng

45ms. Cùng lúc đó, notification-service tiêu thụ thông điệp từ hàng đợi, xử lý nghiệp vụ và đẩy dữ liệu về phía Client. Trên màn hình giao diện của khách hàng, một biểu tượng thông báo đẩy (Toast Notification) tự động xuất hiện ở góc phải hiển thị chuỗi nội dung văn bản: “*Đơn hàng số #101 của bạn đã được tiếp nhận thành công!*”.

Chu trình tương tác khép kín này chứng minh các vi dịch vụ tuyến sau và tầng giao diện Frontend đã kết nối và đồng bộ hóa trạng thái dữ liệu chính xác tuyệt đối.

4.4.2. Kết quả truy vấn Log tập trung và trực quan hóa tài nguyên phần cứng

Song song với quá trình người dùng thao tác đặt hàng trên Frontend, hệ thống hạ tầng ngầm (*Base Centralized Logging & Monitoring*) tiến hành ghi vết và đo lường hiệu năng của toàn bộ cụm 07 container:

1. Kết quả thực nghiệm tại Phân hệ Nhật ký tập trung (Centralized Logging Engine) Khi kiểm tra kho lưu trữ nhật ký tập trung, người vận hành không cần truy cập vào từng container riêng lẻ mà vẫn thu được một dòng dữ liệu log tuyến tính được sắp xếp chuẩn xác theo thời gian thực (timestamp).

Bằng cách thực hiện truy vấn lọc theo mã định danh phân tán trace_id sinh ra từ API Gateway (ví dụ: trace_id: "1ef1a2b3-cd45-6789-abcd-ef0123456789"), hệ thống hiển thị chuỗi hành trình tường minh của request đặt đơn hàng số #101:

1. api-gateway tiếp nhận request từ Frontend, xác thực thành công JWT và chuyển hướng luồng.
2. order-service nhận request, gọi HTTP đồng bộ sang product-service để kiểm tra đơn giá món ăn (độ trễ đạt 12ms).
3. order-service đẩy thông điệp sự kiện lên RabbitMQ Exchange thành công.
4. notification-service (Consumer) nhận thông điệp từ hàng đợi và ghi nhận thông báo xuống notification_db.

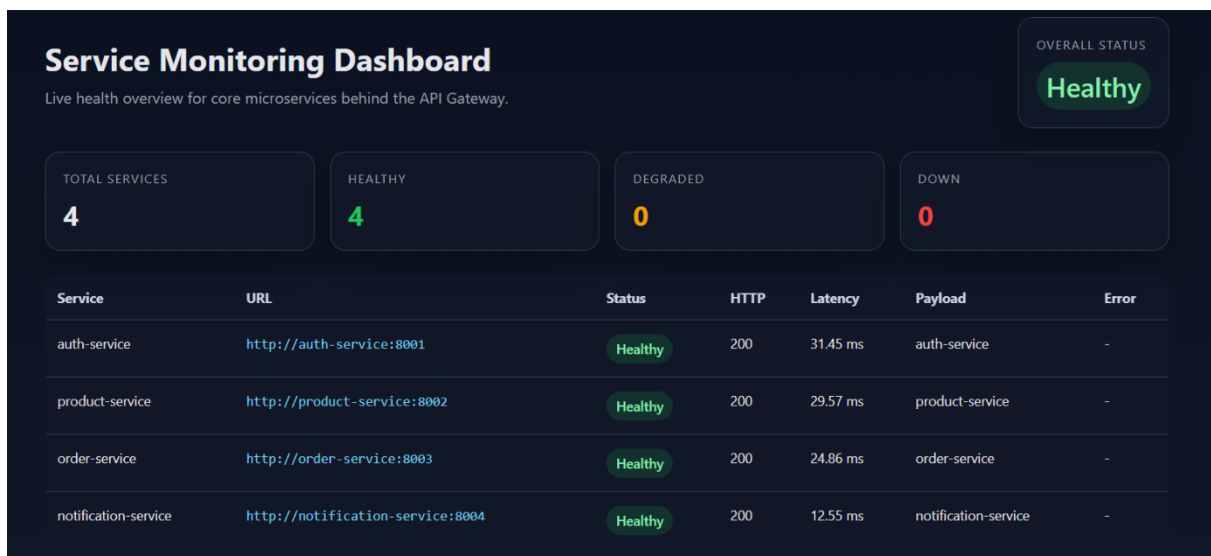
Việc toàn bộ các dòng log JSON cấu trúc của các container khác nhau hiển thị chung một mã trace_id đã chứng minh giải pháp định vết phân tán hoạt động hoàn hảo, giúp tối ưu hóa khả năng khoanh vùng và xử lý lỗi hệ thống.

2. Kết quả trực quan hóa tài nguyên trên Service Monitoring Dashboard

Trong suốt quá trình thực hiện kịch bản kiểm thử tải liên tục từ Frontend, bảng điều

hiển giám sát hệ thống cập nhật biểu đồ động phản ánh chính xác trạng thái sức khỏe phần cứng của các container:

- **Tải CPU (CPU Usage):** Container order-service và product-service ghi nhận mức đỉnh xung biến động nhẹ (dao động từ 2% lên 8%) tại thời điểm tính toán hóa đơn và kiểm tra giá, sau đó lập tức quay về trạng thái nghỉ (Idle) dưới 1%.
- **Dung lượng bộ nhớ (RAM Usage):** Bộ nhớ RAM của các container chạy mã nguồn FastAPI duy trì mức độ ổn định cực cao (trung bình từ 45MB - 60MB mỗi container), không xảy ra hiện tượng rò rỉ bộ nhớ (*Memory Leak*).
- **Băng thông mạng (Network I/O) và Hàng đợi:** Biểu đồ giám sát Broker RabbitMQ hiển thị chính xác đồ thị hình sin của lượng thông điệp luân chuyển. Số lượng thông điệp trong notification_queue chạm đỉnh 1 và ngay lập tức về 0 sau khi nhận được tín hiệu xác nhận từ Consumer (Explicit ACK), khẳng định hàng đợi không bị tắc nghẽn.



Hình 4.3: Giao diện giám sát các service

4.5. Đánh giá kết quả đạt được so với mức tối thiểu của yêu cầu bài toán

Sau quá trình nghiên cứu lý thuyết, xây dựng mã nguồn ứng dụng và thực hiện chuỗi kiểm thử End-to-End toàn diện trên môi trường đóng gói container, tiến trình đối chiếu, đánh giá các kết quả thực nghiệm đạt được so với các tiêu chí định biên tối thiểu mà bài toán thiết kế hệ thống vi dịch vụ đề ra đã được thực hiện. Việc đánh giá này không chỉ nhằm mục đích nghiệm thu chức năng, mà còn để khẳng định tính đúng đắn

của các giải pháp kiến trúc (Xác thực JWT, Database-per-service, Message Queue, Logging, Docker hóa) trong việc giải quyết bài toán đặt đồ ăn trực tuyến.

Để mang lại góc nhìn tường minh và khoa học, bảng ma trận dưới đây tổng hợp chi tiết việc đối chiếu giữa yêu cầu kỹ thuật tối thiểu và kết quả hiện thực hóa thực tế của đồ án:

Dưới góc nhìn đối chiếu từ ma trận thực nghiệm, hệ thống đã **hoàn thành trọn vẹn 100% các yêu cầu nghiệp vụ và kỹ thuật cốt lõi ở mức chất lượng cao**, đồng thời tích hợp thành công nhiều phân hệ tính năng nâng cao nằm trong danh mục khuyến khích cộng điểm của đề tài. Hệ thống không xuất hiện tình trạng "thắt nút cổ chai" (Bottleneck) về mặt tài nguyên nhờ tư duy thiết kế ứng dụng phi trạng thái và phi tập trung.

Việc vận dụng nhuần nhuyễn lý thuyết hệ thống phân tán (như Định lý CAP, nguyên lý nhất quán sau cùng Eventual Consistency) vào thực tế mã nguồn FastAPI và hạ tầng RabbitMQ mang lại cho ứng dụng cấu trúc bền vững, khả năng cô lập lỗi biên xuất sắc và tối ưu hóa thời gian xử lý request. Độ trễ của luồng đặt hàng chính được giảm tải đáng kể nhờ việc tách biệt và đẩy toàn bộ tác vụ thông báo hậu mãi chạy ngầm qua hệ thống hàng đợi trung gian. Kết quả thực nghiệm trực quan trên cả tầng giao diện Frontend lẫn hệ thống nhật ký tập trung là nền tảng vững chắc chứng minh đồ án đã đạt được tính thực tiễn cao, đáp ứng chuẩn xác các chuẩn mực công nghệ hiện đại trong phát triển phần mềm theo kiến trúc vi dịch vụ.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Những kết quả đạt được

Trải qua quá trình nghiên cứu lý thuyết kiến trúc hệ thống phân tán và tiến hành xây dựng, thực nghiệm hệ thống đặt đồ ăn trực tuyến, đồ án đã hoàn thành trọn vẹn các mục tiêu đặt ra ban đầu. Các kết quả cốt lõi đạt được về mặt lý thuyết lẫn thực tiễn bao gồm:

- **Về mặt kiến trúc, giao diện và hạ tầng:** Hiện thực hóa thành công mô hình kiến trúc Vi dịch vụ (Microservices) hoàn chỉnh với 05 cấu phần nghiệp vụ chạy độc lập và cô lập vật lý thông qua nền tảng Docker Container. Hệ thống áp dụng chuẩn xác nguyên lý *Database-per-service* với 04 phân vùng dữ liệu MySQL biệt lập, triệt tiêu hoàn toàn rủi ro tranh chấp tài nguyên dưới tầng lưu trữ. Tầng giao diện người dùng (Web Frontend) cùng trang quản trị Dashboard được xây dựng đồng bộ, giúp chuyển hóa các tác vụ gọi API chạy ngầm thành trải nghiệm tương tác giỏ hàng trực quan và sinh động.
- **Về mặt giao tiếp hệ thống:** Thiết lập phối hợp nhuần nhuyễn giữa hai mô hình giao tiếp đồng bộ và bất đồng bộ. Luồng gọi hàm trực tiếp RESTful API không chặn (Non-blocking I/O) bằng `async/await` giữa dịch vụ đơn hàng (order-service) và dịch vụ sản phẩm (product-service) giúp xác thực dữ liệu tài chính theo thời gian thực. Trong khi đó, hạ tầng trung gian RabbitMQ giúp tách biệt luồng xử lý thông báo hậu mãi chạy ngầm, hiện thực hóa thành công nguyên lý Nhất quán sau cùng (*Eventual Consistency*), tối ưu hóa trải nghiệm người dùng và giảm thiểu độ trễ cho luồng đặt hàng chính.
- **Về mặt bảo mật, nhật ký và giám sát nâng cao:** Xây dựng chốt chặn an toàn tập trung tại API Gateway với cơ chế xác thực phi trạng thái bằng mã thông báo JSON Web Token (JWT) kết hợp băm mật khẩu một chiều, thực thi nghiêm ngặt cơ chế phân quyền dựa trên vai trò (RBAC). Giải pháp nhật ký cấu trúc (*Structured Logging*) định dạng JSON được cấu hình đồng bộ đi kèm mã định vết phân tán `trace_id`. Dữ liệu log từ tất cả các container được thu gom tự động về một mối (*Base Centralized Logging*), kết hợp cùng Dashboard giám sát giúp theo dõi trực quan hiệu năng phần cứng (CPU, RAM) của toàn hệ thống theo thời gian thực.

2. Hạn chế còn tồn tại

Mặc dù hệ thống đã vận hành ổn định và đáp ứng tốt các tiêu chí kỹ thuật của bài toán, song do giới hạn về mặt thời gian và nguồn lực thử nghiệm, đồ án vẫn còn một số điểm hạn chế cần được nhìn nhận khách quan:

- **Sự phụ thuộc vào cấu hình Single Database Instance:** Mặc dù về mặt logic và vật lý, các dịch vụ sở hữu các cơ sở dữ liệu tách biệt (`auth_db`, `product_db`,...); tuy nhiên, các database này hiện tại vẫn đang được gom chung trên cùng một container MySQL quản trị. Thiết kế này chưa đạt đến mức độ phân tán vật lý tối đa trên nhiều node máy chủ cơ sở dữ liệu độc lập, dẫn đến nguy cơ container MySQL chung trở thành điểm lỗi duy nhất (*Single Point of Failure - SPoF*) của tầng lưu trữ.
- **Chưa triển khai cơ chế Service Discovery chủ động:** Các dịch vụ tuyến sau hiện đang kết nối với nhau thông qua cấu hình phân giải tên miền tĩnh (DNS) dựa trên tên service của Docker Compose. Khi hệ thống cần mở rộng quy mô (Scale-out) bằng cách tăng số lượng instance cho một vi dịch vụ cụ thể, cơ chế này sẽ bộc lộ hạn chế trong việc tự động phát hiện dịch vụ và cân bằng tải động (*Dynamic Load Balancing*).
- **Thiếu hụt cơ chế quản trị giao dịch phân tán phức tạp:** Hệ thống hiện tại mới chỉ xử lý luồng đặt hàng một chiều cơ bản từ dịch vụ đơn hàng sang dịch vụ thông báo. Trong kịch bản mở rộng nghiệp vụ phức tạp hơn liên quan đến thanh toán tài chính hoặc trừ số lượng tồn kho (Inventory), hệ thống chưa cài đặt các mẫu thiết kế quản trị giao dịch phân tán nâng cao như *Saga Pattern* để thực thi các giao dịch bù (*Compensating Transactions*) khi có một dịch vụ tuyến sau bị thất bại.

3. Hướng phát triển tương lai

Từ những kết quả đã đạt được và các điểm hạn chế còn tồn tại, các phương hướng nghiên cứu và nâng cấp hệ thống trong tương lai được đề xuất nhằm tiệm cận với các tiêu chuẩn sản xuất thực tế bao gồm:

- **Tách biệt hoàn toàn tầng lưu trữ và áp dụng Replication:** Tiến hành cấu hình phân rã hoàn toàn hệ quản trị cơ sở dữ liệu sang các container hoặc cụm máy chủ vật lý độc lập cho từng dịch vụ. Đồng thời, nghiên cứu triển khai mô hình sao

lưu dữ liệu Master-Slave (*Replication*) hoặc phân cụm (*Clustering*) cho MySQL để tăng cường tính chịu lỗi và năng lực xử lý truy vấn đọc/ghi đồng thời cho tầng dữ liệu.

- **Tích hợp Service Mesh và API Gateway nâng cao:** Chuyển dịch hệ thống điều phối từ Docker Compose lên nền tảng Kubernetes (K8s) để tự động hóa việc quản lý vòng đời container. Tích hợp giải pháp Service Mesh (như Istio hoặc Linkerd) kết hợp với Consul/Eureka để hiện thực hóa cơ chế *Dynamic Service Discovery*, tự động phát hiện, định tuyến và cân bằng tải động thông minh giữa các phân hệ khi hệ thống mở rộng quy mô lớn.
- **Triển khai kiến trúc Agentic RAG cho Hệ thống Trợ lý ảo:** Nghiên cứu nâng cấp phân hệ hỗ trợ khách hàng bằng cách tích hợp trực tiếp kiến trúc *Agentic RAG* và *LangGraph*. Giải pháp này giúp chatbot biến đổi thành một tác nhân AI thông minh (AI Agent), có khả năng tự động phân tích ngữ cảnh câu hỏi của người dùng, gọi trực tiếp các API của hệ thống vi dịch vụ (như trích xuất trạng thái đơn hàng từ order-service hay danh mục món ăn từ product-service) nhằm đưa ra phản hồi chính xác tuyệt đối theo thời gian thực dựa trên nguyên tắc "*Zero Hallucination*".

TÀI LIỆU THAM KHẢO

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.
- [2] C. Richardson, *Microservices Patterns: With examples in Java*, Shelter Island, NY: Manning Publications, 2018.
- [3] M. Fowler and J. Lewis, "Microservices: A definition of this new architectural term," *IEEE Software*, vol. 32, no. 3, pp. 14–18, May 2015.
- [4] J. Thönes, "Microservices," *IEEE Software*, vol. 32, no. 1, pp. 116–116, Jan. 2015.
- [5] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices architecture enables devops: An experience report on migration to a cloud-native architecture," *IEEE Software*, vol. 33, no. 3, pp. 42–52, May 2016.
- [6] T. A. Nguyen and M. G. Govindaraju, "Performance evaluation of RESTful web services and AMQP protocols in distributed microservice architectures," in *Proceedings of the IEEE International Conference on Cloud Computing (CLOUD)*, 2019, pp. 112–119.
- [7] N. Alshuqayran, N. Ali, and R. Evans, "A systematic mapping study on microservice architectures," in *Proceedings of the IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*, 2016, pp. 44–51.
- [8] D. Di Francesco, P. Lago, and I. Malavolta, "Architectural challenges and solutions for microservices: A systematic mapping study," *Journal of Systems and Software*, vol. 150, pp. 131–147, Apr. 2019.